Near-Floor Geometry Is Generic: Leverage Dispersion in Trained Overcomplete Codes

Anonymous authors
Paper under double-blind review

Abstract

An overcomplete code packs F features into $d < F$ dimensions, so linear readout of one feature picks up cross-talk from the others, bounded below by the rank–trace floor $W(F, d) = (F - d)/(d(F - 1))$. Proximity to this floor is read as evidence of an efficient arrangement. It is not: an i.i.d. code of the same shape attains it at every shape and load we measure (1.0000–1.0005 over 7 matched controls), and the reason is exact. For the gain-calibrated pseudoinverse the attainment ratio is $\frac{d}{F(F-d)}(\sum_i h_i^{-1} - F)$ with $\sum_i h_i = d$, so it equals one precisely when the leverage h_i is equalised across features: distance from the floor *is* leverage dispersion.

On 40 released decoder matrices — 24 sparse-autoencoder decoders plus 16 MLP down-projections of a hybrid model — all sit *above* their matched control (1.0055–1.2018), and all reach guaranteed affine failure 4 to 40 sparsity levels earlier: where a random code of the same shape is still safe, a released dictionary already has a feature no affine rule recovers. Autoencoders trained by the same recipe on a *randomly initialised* copy of the same model return to the floor (1.0038–1.0065) in all 10 paired layers, so the excess reflects what the model learned, not the act of fitting an autoencoder. Under controlled conditions the loss decides whether an interface exists at all: L^2 reaches $R_{\text{geom}} = 18.5635$ to 32.3131 with its frontier collapsed to 1, against 1.0002 for L^4, and that frontier follows a rate we registered before the widest run (10.79 predicted, 10.510 measured).

Two limits are stated rather than implied: frontier statements are worst cases, not typical-case claims; and because the probe class contains the network's own decoder, our decoder comparison bounds supervised fitting rather than affine expressiveness.

1 Introduction

An overcomplete code places F feature directions in $d < F$ dimensions. Such codes are the object of study in work on superposition and computation in superposition, and they are what a sparse autoencoder returns when it decomposes a language model's activations into a dictionary of feature directions. Whatever else one wants from such a code, a great deal of downstream practice reads features out of it *linearly*: linear probes, the decoder of the autoencoder itself, steering vectors, and any diagnostic built from inner products with feature directions.

Linear readout of an overcomplete code cannot be exact. With $F > d$ the directions cannot be orthogonal, so recovering one feature's coefficient picks up contributions from the others, and the average squared size of that cross-talk is bounded below by a quantity that depends only on the two counts,

$$W(F, d) \;=\; \frac{F - d}{d(F - 1)}.$$

Reporting how close a code comes to this rank–trace floor is therefore a natural way to ask whether it is arranged well for linear readout, and ratios of this kind have been read as evidence that training discovers efficient geometry.

What a reader should take from this paper. Proximity to the floor is not evidence of anything a code learned, because it is what an unstructured code gives for free; the quantity that distance from the floor actually measures is the *dispersion of leverage* across features; and measured this way, no released sparse-autoencoder dictionary we examined is near the floor — every one carries more cross-talk under the best linear readout it admits than a random code of its own shape does, while an autoencoder trained by the same recipe on a randomly initialised model is not. The practical consequence is that the excess is a property of the represented content rather than of the fitting procedure, and that it moves with depth but not with dictionary width.

1.1 Why proximity to the floor says less than it appears to

Our starting point is an identity, not an experiment. For a code $\Phi \in \mathbb{R}^{d\times F}$ read out by its gain-calibrated pseudoinverse, the ratio of achieved cross-talk to the floor is

$$R_{\mathrm{geom}}(\Phi) \;=\; \frac{d}{F(F-d)}\left(\sum_{i=1}^{F}\frac{1}{h_i} - F\right), \qquad h_i = \phi_i^{\top}(\Phi\Phi^{\top})^{-1}\phi_i,$$

and the leverage scores obey $\sum_i h_i = d$ exactly. Cauchy–Schwarz then gives $\sum_i h_i^{-1} \geq F^2/d$ with equality precisely when every h_i equals d/F, so $R_{\mathrm{geom}} \geq 1$ with equality exactly when leverage is equalised across features (Proposition 5). Distance from the floor is not a separate property that a code might or might not have alongside its leverage profile: *it is* that profile's dispersion, in a specific harmonic sense that we make precise and then use.

Two consequences follow immediately and are worth stating before any measurement. First, a code whose leverage is nearly uniform sits at the floor whether or not it encodes anything, which is why an i.i.d. Gaussian code attains it. Second, the ratio is driven by $\sum_i h_i^{-1}$, a harmonic quantity, so the spread of leverage can grow substantially without the ratio moving — a dissociation we observe and would otherwise have had to report as an anomaly.

1.2 What we measure, and on what

We evaluate the diagnostic on two kinds of object, and the division of labour between them is deliberate.

Released dictionaries (Section 5) test whether any of this transfers beyond a controlled setting. We take the decoder matrices of 40 published sparse autoencoders — five model families and three independent training recipes, with d from 576 to 4 096 and feature loads from 4.5× to 85.3× — and compute R_{geom}, the leverage distribution, and the failure threshold from the decoder alone. No model weights, no activations, no training, and no linear programme are involved, so the whole section costs seconds per dictionary and can be re-run by a reader.

Controlled campaigns (Section 6) supply what released artefacts cannot: an intervention. We train 400 small networks in which the loss, the width, the feature load and the training sparsity are set by us, so that the effect of each on the resulting code can be measured rather than inferred, and we retain every trained model so that later analyses need no retraining.

1.3 Contributions

1. **Distance from the rank–trace floor is leverage dispersion, exactly.** The identity is elementary and we claim no novelty for the mathematics; what is new is that it does predictive work. It explains why unstructured codes attain the floor, why trained dictionaries do not, and why dispersion can grow while the ratio stays flat. We verify it against the implementation on 57 measured codes, with a maximum deviation of 2e-16.
2. **Near-floor geometry is generic, and we measure the baseline rather than assert it.** 7 i.i.d. controls, matched in shape and operating sparsity to each dictionary, land at $R_{\mathrm{geom}} = 1.0000$ to 1.0005 across loads from 4.5× to 85.3×. Any claim that a learned code is close to optimal for linear readout needs this comparison, and it is not present in the literature we build on.

3. **No released dictionary we measured is at the floor.** All 40 of 40 sit above their own matched control, at $R_{\text{geom}} = 1.0055$ to 1.2018, with leverage dispersion 0.090 to 0.469 against the controls' 0.005 to 0.020. Whatever these dictionaries optimise, it is not the evenness of their own leverage.

4. **The same holds in sparsity units, which is the form a practitioner can use.** The diagnostic also returns the sparsity at which some feature is guaranteed to be unrecoverable by any affine rule. All 40 dictionaries reach it 4 to 40 levels earlier than the i.i.d. code of their own shape and operating sparsity: where a random code is still safe, a released dictionary already has a feature no affine rule recovers. This is a worst-case statement about one feature rather than an average over all of them, so its agreement with the ratio is not automatic — though both are functionals of the same leverage distribution, and neither is an external endpoint (Section 9).

5. **A matched randomly-initialised control locates the cause.** Autoencoders trained by an identical recipe on a randomly initialised copy of the same model return to the floor ($R_{\text{geom}} = 1.0038$ to 1.0065) in all 10 paired layers. The excess therefore reflects what the model learned, not the act of fitting a sparse autoencoder — a distinction an i.i.d. control cannot draw, because it does not share the training procedure.

6. **The loss decides whether a linear interface exists at all.** Across four widths, two feature loads and two training sparsities, L^2 training reaches $R_{\text{geom}} = 18.5635$ to 32.3131 with its affine recovery frontier collapsed to 1, while L^4 holds 1.0002 with a frontier that grows from 4.4873 to 10.510. This is the largest effect we report and the one that replicates most widely.

7. **The frontier follows a rate, tested out of sample, and the advantage it buys stops growing.** Fitted on three widths the frontier implied 10.79 at $d = 400$; we registered that number in the run configuration before the width was trained and measured 10.510. Over the same four widths the advantage over an untrained code narrows from 1.4195 to 1.1800, which we report as a limitation on the preceding claim rather than alongside it.

8. **An artefact that can be checked.** The weights of all 400 trained networks are released, along with the SHA-256 of every third-party dictionary analysed, a decision log recording what was registered before each campaign, and a register of nine defects found during this work together with what each invalidated. Section 8 states which readings we withdrew and why.

1.4 Scope

Three limits apply throughout and we state them here rather than at the end. Every statement about the affine recovery frontier is a worst case over feature amplitudes: it says what no affine rule can guarantee, not what fails in typical use. The comparison between a network's own decoder and a fitted affine probe (Section 6.3) measures what supervised fitting recovers rather than what an affine map can express, because the probe class contains the network's decoder — and we verify that the class membership is not hypothetical: the selection misses the network's own map, on the very validation objective it maximises, in every L^4 model above the tie width. We report the comparison for completeness and build nothing on it. And the controlled campaigns use one task family at widths up to $d = 400$, so they establish the mechanism, not its magnitude in a deployed model — which is precisely why the dictionary measurements are the other half of the paper.

2 Problem setting and notation

We write $\widetilde{O}(\cdot), \widetilde{\Omega}(\cdot), \widetilde{\Theta}(\cdot)$ to hide factors polynomial in $\log d$. Table 1 fixes the notation used throughout.

Two decoding questions are compared throughout. *Analog reconstruction* asks that the scores $z = Mb$ approximate b in a norm; *support recovery* asks only that $\mathbf{1}\{z_i \geq \theta\} = b_i$ for every i. The first is a statement about magnitudes and is never exactly satisfiable when $F > d$; the second is a statement about a decision boundary and can be satisfied exactly — by a thresholded linear score as much as by anything else, which is why that rule is measured here too rather than treating support recovery as the preserve of nonlinearity. Everything below is an elaboration of that asymmetry.

Table 1: Notation.

symbol	meaning
d	activation width (number of neurons)
F	number of features, $F > d$
$\Phi \in \mathbb{R}^{d\times F}$	code; column ϕ_i is the direction of feature i
$G \in \mathbb{R}^{F\times d}$	linear readout
$M = G\Phi \in \mathbb{R}^{F\times F}$	readout interface, $\operatorname{rank}(M) \le d$
$D = \operatorname{diag}(M_{11}, \dots, M_{FF})$	diagonal gains; $D^{-1}M$ is the calibrated interface
$A = M - I_F$	cross-talk matrix (zero diagonal after calibration)
$b \in \{0,1\}^F$	sparse Boolean state; $S = \operatorname{supp}(b)$, $s = \lvert S\rvert$
$\mu = \max_{i\neq j} \lvert\langle \phi_i, \phi_j\rangle\rvert$	coherence of a unit-norm code
$W(F,d) = \frac{F-d}{d(F-1)}$	mean-square Welch-type floor
$\widehat{W}$	measured mean squared off-diagonal cross-talk
θ	decoding threshold (default 1/2)
$p_{\mathrm{rec}}(s)$	probability of exact threshold recovery at sparsity s
s_{95}	largest s such that $p_{\mathrm{rec}} \ge 0.95$ for *every* tested $s' \le s$
$\Sigma = \Phi\Phi^\top$	frame operator (invertible iff Φ has full row rank)
$h_i = \phi_i^\top \Sigma^{-1} \phi_i$	leverage of feature i; $\sum_i h_i = d$
$W_\star(\Phi)$	the code's own floor: least mean-square cross-talk it admits
$R_{\mathrm{geom}}, R_{\mathrm{readout}}$	$W_\star(\Phi)/W(F,d)$ and $W(G,\Phi)/W_\star(\Phi)$
Φ_{-i}	Φ with column i deleted
$\alpha \in (0,1]$	smallest detectable active amplitude
$\kappa_i(\alpha)$	collision frontier of feature i; separable at s iff $\kappa_i > s$
$\delta_i(s), \gamma_i(s)$	distance between the two hulls, and the margin $\delta_i(s)/2$
m, m'	feature counts *inside imported statements*; m' plays the role of F

3 The linear-readout floor

Theorem 1 (Unit-diagonal cross-Gramian Welch floor)**.** *Let $F \ge 2$, let $\Phi \in \mathbb{R}^{d\times F}$ and $G \in \mathbb{R}^{F\times d}$, and set $M = G\Phi$. Suppose $M_{ii} = 1$ for all $i \in [F]$. Then*

$$\sum_{i\neq j} |M_{ij}|^2 \;\ge\; \frac{F(F-d)}{d}. \tag{1}$$

Consequently, if $F > d$,

$$\frac{1}{F(F-1)} \sum_{i\neq j} |M_{ij}|^2 \;\ge\; W(F,d) := \frac{F-d}{d(F-1)}, \tag{2}$$

and hence $\max_{i\neq j} |M_{ij}| \ge \sqrt{W(F,d)}$. In particular, if $F/d \to \infty$ then $\max_{i\neq j} |M_{ij}| \ge (1 - o(1))\, d^{-1/2}$.

Proof. Since $M = G\Phi$ with $G \in \mathbb{R}^{F\times d}$ and $\Phi \in \mathbb{R}^{d\times F}$ we have $\operatorname{rank}(M) \le d$, and the unit diagonal gives $\operatorname{tr}(M) = F$. For any matrix of rank at most d, $|\operatorname{tr}(M)| \le \|M\|_* \le \sqrt{d}\, \|M\|_F$ (Appendix N), so $F^2 \le d\, \|M\|_F^2$. Expanding $\|M\|_F^2 = F + \sum_{i\neq j} |M_{ij}|^2$ gives (1), and dividing by $F(F-1)$ gives (2). For the maximum, note that $\max_{i\neq j} |M_{ij}|^2 \ge \frac{1}{F(F-1)} \sum_{i\neq j} |M_{ij}|^2$, and take square roots. □

The argument is the standard rank–trace mechanism, cf. Datta et al. (2012); we give it in full only for self-containedness. The next corollary is what the diagnostic actually uses, because it removes the normalisation hypothesis.

Corollary 2 (Gain-normalised form)**.** *Let $F > d$ and let $M = G\Phi$ satisfy $\operatorname{rank}(M) \le d$ and $M_{ii} \neq 0$ for all i. Then*

$$\max_{i\neq j} \frac{|M_{ij}|}{|M_{ii}|} \;\ge\; \sqrt{\frac{F-d}{d(F-1)}}.$$

Proof. Apply Theorem 1 to $D^{-1}M$ with $D = \mathrm{diag}(M_{11}, \ldots, M_{FF})$: this matrix has rank at most d, unit diagonal, and entries M_{ij}/M_{ii}. □

Remark 3 (Why the gain normalisation is essential)**.** Without a diagonal normalisation the absolute off-diagonal entries of $G\Phi$ can be made arbitrarily small by shrinking G, and the floor appears to fail. A concrete instance: take Φ to be the three unit vectors at 120° in $\mathbb{R}^2$ — an equiangular tight frame — and $G = D\Phi^\top$ with $D = \mathrm{diag}(1/10, 1/20, 1/20)$. Then $G\Phi$ has diagonal $(1/10, 1/20, 1/20)$ and maximum off-diagonal entry $1/20$, ten times *below* $\sqrt{(F-d)/(d(F-1))} = 1/2$. There is no contradiction: this interface reads out its own coordinates with gains far below one, and after recalibrating each row to unit gain the maximum off-diagonal entry becomes exactly $1/2$ — the floor, attained with equality. Corollary 2 isolates the scale-invariant content: what is bounded below is the interference-to-gain ratio. This example is verified in the test suite, and Section F.2 shows that a gradient optimiser left free of the constraint exploits exactly this degree of freedom.

Proposition 4 (Equality within the tied family)**.** *Let $\Phi \in \mathbb{R}^{d\times F}$ have unit-norm columns and let $G = \Phi^\top$, so that $M = \Phi^\top\Phi$ has unit diagonal. Then $\sum_{i\neq j} |M_{ij}|^2 \geq F(F-d)/d$, with equality* if and only if $\Phi\Phi^\top = (F/d)I_d$, *that is, iff Φ is a unit-norm tight frame.*

Proof. The diagonal entries are $\|\phi_i\|^2 = 1$. By cyclicity, $\|M\|_F^2 = \mathrm{tr}((\Phi\Phi^\top)^2) = \|\Sigma\|_F^2$ with $\Sigma = \Phi\Phi^\top \succeq 0$ and $\mathrm{tr}\,\Sigma = F$. Cauchy–Schwarz on the eigenvalues of Σ gives $\|\Sigma\|_F^2 \geq (\mathrm{tr}\,\Sigma)^2/d = F^2/d$, with equality iff all d eigenvalues coincide, i.e. $\Sigma = (F/d)I_d$. Subtracting the diagonal contribution F gives the claim. □

Within this family — unit-norm columns read out by their own transpose — equality therefore characterises tight frames, consistent with Waldron (2003); Datta et al. (2012). We stress the scope: it is a statement about the tied Gram matrix, not about every rank-d unit-diagonal interface. Other readouts attain the same floor under different conditions, and the next proposition identifies the condition for the pseudoinverse. Whether learned codes find such configurations is an empirical question, answered in Section F.2.

Proposition 5 (The pseudoinverse ratio is a leverage statistic)**.** *Let $\Phi \in \mathbb{R}^{d\times F}$ have full row rank, let $G = \Phi^+$, and write $P = \Phi^+\Phi$ for the orthogonal projector onto the row space, with leverage scores $h_i = P_{ii} \in (0, 1]$. Then the gain-normalised interface $\widetilde{M} = D^{-1}P$ satisfies*

$$\sum_{i\neq j} \widetilde{M}_{ij}^2 \;=\; \sum_{i=1}^{F} \frac{1}{h_i} - F, \qquad \textit{so} \qquad r = \frac{d}{F(F-d)} \left(\sum_{i=1}^{F} \frac{1}{h_i} - F \right).$$

Since $\sum_i h_i = \mathrm{tr}\,P = d$, Cauchy–Schwarz gives $\sum_i h_i^{-1} \geq F^2/d$ with equality iff $h_i = d/F$ for every i. Hence $r \geq 1$, and $r = 1$ exactly when leverage is equalised across features.

Proof. P is symmetric idempotent, so row i obeys $\sum_j P_{ij}^2 = P_{ii} = h_i$ and $D_{ii} = h_i$. Dividing row i by h_i and removing the unit diagonal entry leaves $(h_i - h_i^2)/h_i^2 = h_i^{-1} - 1$; summing over i gives the identity. The inequality is Cauchy–Schwarz applied to $\sum_i h_i \cdot \sum_i h_i^{-1} \geq F^2$, and dividing by $F(F-1)$ and by $W(F,d) = (F-d)/(d(F-1))$ turns it into $r \geq 1$. □

This is worth stating because it deflates a natural over-reading of a ratio close to one under the pseudoinverse readout. Attaining the floor with $G = \Phi^+$ says that the code spreads its features evenly over the row space — nothing more. It is a statement about the geometry of the code, not about whether the code is a good representation for any task; a code can have perfectly equalised leverage and still be useless. We use this to scope the empirical claims of Section F.3.

3.1 The floor of the code itself

Theorem 1 bounds every rank-d unit-diagonal interface, so it is a statement about the *ensemble* of codes: no particular Φ need be able to attain it. That makes a measured ratio hard to read. If a trained code sits 0.06% above $W(F,d)$, is the readout excellent, or is the floor simply unreachable for this code and the readout merely adequate? The question is answered by computing the floor of the code itself, which is available in closed form.

Theorem 6 (Code-specific analog optimum)**.** *Let $\Phi \in \mathbb{R}^{d\times F}$ have full row rank, let $\Sigma = \Phi\Phi^\top \succ 0$ be its frame operator, and let $h_i = \phi_i^\top \Sigma^{-1}\phi_i$ be the leverage of feature i. Then for every i*

$$\min_{g\in\mathbb{R}^d} \Big\{ \sum_{j\neq i} (g^\top \phi_j)^2 \;:\; g^\top \phi_i = 1 \Big\} \;=\; \frac{1}{h_i} - 1, \tag{3}$$

attained uniquely at $g_i^\star = \Sigma^{-1}\phi_i/h_i$. Stacking the F minimisers as rows gives $G^\star = D_h^{-1}\Phi^+$ with $D_h = \mathrm{diag}(h_1,\dots,h_F)$: the row-wise optimum is the Moore–Penrose pseudoinverse after *gain calibration, and coincides with Φ^+ itself only when every $h_i = 1$. Consequently the smallest mean-square off-diagonal cross-talk this code admits is*

$$W_\star(\Phi) \;:=\; \frac{1}{F(F-1)} \sum_{i=1}^{F} \Big(\frac{1}{h_i} - 1 \Big) \;=\; \frac{\sum_i h_i^{-1} - F}{F(F-1)} \;\geq\; W(F,d), \tag{4}$$

with equality if and only if $h_i = d/F$ for every i.

Proof. Since $\sum_j (g^\top\phi_j)^2 = g^\top \Sigma g$, the constraint $g^\top\phi_i = 1$ turns the objective into $g^\top\Sigma g - 1$. Minimising a positive-definite quadratic under one linear equality is classical: the Lagrange conditions give $g = \lambda\Sigma^{-1}\phi_i$, the constraint fixes $\lambda = 1/h_i$, and the value is $\phi_i^\top\Sigma^{-1}\phi_i/h_i^2 = 1/h_i$. Subtracting the constrained direction's contribution, which is exactly 1, gives (3); strict convexity gives uniqueness. Summing over i and dividing by $F(F-1)$ gives (4). Finally $\sum_i h_i = \mathrm{tr}(\Sigma^{-1}\Phi\Phi^\top) = d$, so Cauchy–Schwarz gives $\sum_i h_i^{-1} \geq F^2/d$ with equality iff the h_i are all equal, and $(F^2/d - F)/(F(F-1)) = W(F,d)$. □

Remark 7 (Provenance)**.** Equation (3) is the minimum-variance distortionless-response (Capon) beamformer with the frame operator in place of a covariance Capon (1969), and $g_i^\star$ is the canonical dual frame vector rescaled to unit gain; both are classical. We claim no novelty in the optimisation. What the diagnostic uses is the reading of h_i as a leverage score, the resulting per-code floor (4), and the exact attribution of the gap in Proposition 8.

Proposition 8 (The excess is leverage dispersion, exactly)**.** *With $c = d/F$ the mean leverage,*

$$\sum_{i=1}^{F} \frac{1}{h_i} - \frac{F^2}{d} \;=\; \sum_{i=1}^{F} \frac{(h_i - c)^2}{c^2 h_i} \;\geq\; 0.$$

Hence $W_\star(\Phi)$ exceeds $W(F,d)$ by a weighted dispersion of the leverage scores, which vanishes precisely when leverage is uniform.

Proof. Expand the right-hand side: $c^{-2}\sum_i (h_i^2 - 2ch_i + c^2)/h_i = c^{-2}\sum_i h_i - 2c^{-1}F + \sum_i h_i^{-1}$. Substituting $\sum_i h_i = d$ and $c = d/F$ turns the first two terms into $F^2/d - 2F^2/d = -F^2/d$. □

This separates two questions that a single ratio against $W(F,d)$ conflates. Writing $W(G,\Phi)$ for the mean-square off-diagonal cross-talk actually achieved by a readout G,

$$\underbrace{\frac{W(G,\Phi)}{W(F,d)}}_{\text{what is usually reported}} \;=\; \underbrace{\frac{W_\star(\Phi)}{W(F,d)}}_{R_{\text{geom}}\text{: the code}} \;\times\; \underbrace{\frac{W(G,\Phi)}{W_\star(\Phi)}}_{R_{\text{readout}}\text{: the decoder}}, \tag{5}$$

and the two factors answer different questions. R_{geom} asks whether the code's leverage is spread evenly; $R_{\text{readout}} \geq 1$ asks whether the decoder is the best one *for that code*. In particular Proposition 5 says that the calibrated pseudoinverse has $R_{\text{readout}} = 1$ identically, so a published attainment ratio computed with $G = \Phi^+$ measures R_{geom} and nothing else. Section F.3 reports both factors.

Finally, the analog level reads a state distribution through one object only, which is what lets Section C compare it with the affine level under a single state model.

Proposition 9 (Distribution-weighted reconstruction error)**.** *Let $M = G\Phi$ have unit diagonal, let $A = M - I_F$, and let the state $b \in \mathbb{R}^F$ be drawn from any law with finite second moment $C = \mathbb{E}[bb^\top]$. Then $\mathbb{E}\,\|Ab\|_2^2 = \operatorname{tr}(A^\top A\, C)$. The analog level therefore depends on the state law only through C: two laws with the same second moment impose the same reconstruction error on every readout.*

Proof. $\mathbb{E}\,\|Ab\|_2^2 = \mathbb{E}\operatorname{tr}(A^\top Abb^\top) = \operatorname{tr}(A^\top A\, C)$ by linearity. □

Corollary 10 (No better-than-floor ε-linear readout)**.** *Let $F > d$ and suppose G satisfies $\|Gy_i - e_i\|_\infty \le \delta$ for singleton states y_i, with $\delta < 1/2$. Then $\delta/(1-\delta) \ge \sqrt{(F-d)/(d(F-1))}$; in particular $\delta = \Omega(d^{-1/2})$ when $F/d \to \infty$.*

Proof. Let $M = G[y_1, \dots, y_F]$. The hypothesis gives $|M_{ii} - 1| \le \delta$ and $|M_{ji}| \le \delta$ for $j \ne i$. Calibrating, $|\widetilde{M}_{ij}| \le \delta/(1-\delta)$, and Theorem 1 applies to $\widetilde{M}$. □

4 The affine level: a per-feature collision frontier

Sections 3 and B bound two criteria on random ensembles. Neither answers the question an interpretability practitioner actually asks about a *particular* trained code: for this feature, in this network, up to what sparsity can a linear probe read the feature's presence off the representation at all? This section gives that quantity exactly, as the value of one linear programme per feature.

Definition 11 (Affine separability of a feature)**.** Fix $\Phi \in \mathbb{R}^{d\times F}$, a feature i and a sparsity s. Write $\mathcal{A}_i(s)$ for the set of states Φb with $b \in \{0,1\}^F$, $b_i = 1$ and $|\operatorname{supp}(b)| \le s$, and $\mathcal{B}_i(s)$ for the same with $b_i = 0$. Feature i is *affinely separable at sparsity s* if there exist $w \in \mathbb{R}^d$ and $\theta \in \mathbb{R}$ with $w^\top x > \theta$ for every $x \in \mathcal{A}_i(s)$ and $w^\top x < \theta$ for every $x \in \mathcal{B}_i(s)$.

The states are Boolean here, and that is not a simplification we could drop. Allowing an active amplitude to be any positive number makes "i is active" an open condition, and an active state with $b_i \to 0$ converges to an inactive one; the two hulls then touch for every code and every sparsity, so nothing would ever be separable. A positive floor on the active amplitude is what makes the question well posed, and Corollary 15 is the graded statement with that floor made explicit.

An affine functional separates two sets exactly when it separates their convex hulls, so the question is whether those two hulls meet. The programme below decides it.

Theorem 12 (The collision frontier)**.** *For $z \in \mathbb{R}^{F-1}$, indexed by the features other than i, let $z_j^+ = \max(z_j, 0)$ and $z_j^- = \max(-z_j, 0)$, so that both parts are non-negative and $z = z^+ - z^-$. Let Φ_{-i} be Φ with column i deleted. Put*

$$\kappa_i \;:=\; \min_{z} \Big\{ \max\big(\textstyle\sum_j z_j^+,\; 1 + \sum_j z_j^-\big) \;:\; \Phi_{-i} z = \phi_i,\; \|z\|_\infty \le 1 \Big\}. \tag{6}$$

with $\kappa_i := +\infty$ when the constraints admit no z at all. Then feature i is affinely separable at every sparsity $s' \le s$ if and only if $\kappa_i > s$. When κ_i is finite the largest sparsity at which feature i is separable is $\lceil \kappa_i \rceil - 1$; when $\kappa_i = +\infty$ the feature is separable at every sparsity, and no collision exists to bound.

Proof. Separability fails exactly when $\operatorname{conv}\mathcal{A}_i(s)$ meets $\operatorname{conv}\mathcal{B}_i(s)$. By Proposition 14 below, applied to the coordinates other than i with budget $s-1$ on the active side and s on the inactive side, those hulls are $\Phi\{e_i + u : u \in [0,1]^{F-1},\ \mathbf{1}^\top u \le s-1\}$ and $\Phi\{v : v \in [0,1]^{F-1},\ \mathbf{1}^\top v \le s\}$ respectively — the relaxation from 0/1 coordinates to the box is exact, because the polytope is integral. So a collision is a pair u, v in those two boxes with $\Phi(e_i + u) = \Phi v$. We may take u and v to have disjoint supports, since a common component cancels from both sides and only relaxes the two sum constraints. Then $\Phi(e_i + u) = \Phi v$ reads $\Phi_{-i}(v - u) = \phi_i$, so with $z = v - u$ we have $z^+ = v$ and $z^- = u$. The budget on the inactive state is $\sum_j v_j \le s$, that on the active state is $1 + \sum_j u_j \le s$, and the entries lie in $[0,1]$, giving $\|z\|_\infty \le 1$. The two budgets are exactly $\sum_j z_j^+ \le s$ and $1 + \sum_j z_j^- \le s$, so a collision at some sparsity at most s exists if and only if the objective in (6) can be brought to s or below, that is, if and only if $\kappa_i \le s$. □

Algorithm 1 CollisionFrontier — the largest affinely separable sparsity, per feature

Require: code $\Phi \in \mathbb{R}^{d\times F}$; feature set $\mathcal{I} \subseteq [F]$; minimum amplitude $\alpha \in (0, 1]$
Ensure: $\kappa_i(\alpha)$ and the frontier $\lceil \kappa_i(\alpha) \rceil - 1$ for each $i \in \mathcal{I}$
1: **for** $i \in \mathcal{I}$ **do**
2: solve, in variables $u, v \in \mathbb{R}^{F-1}_{\geq 0}$ and $t \in \mathbb{R}$:
$\min\ t \quad \text{s.t.} \quad \Phi_{-i}(u - v) = \phi_i, \quad \alpha \mathbf{1}^\top u \leq t, \quad 1 + \alpha \mathbf{1}^\top v \leq t, \quad u, v \leq 1/\alpha$
3: $\kappa_i(\alpha) \leftarrow t^\star$; $s_i^{\max} \leftarrow \lceil \kappa_i(\alpha) \rceil - 1$
4: **end for**
5: **return** $\{(\kappa_i(\alpha), s_i^{\max})\}_{i\in\mathcal{I}}$

Three things improve on the corresponding exactly-s statement, which we record because the earlier formulation of this work used it. The threshold is s itself rather than $2\min(s, F - s) - 1$; the admissible sets nest in s, so monotonicity of the frontier holds by construction rather than needing a proof and a restriction to $s \leq F/2$; and $\kappa_i = 7.4$ now reads directly as "separable up to 7, lost at 8", which is what a capacity ought to mean. Computationally (6) is a single linear programme, stated as Algorithm 1: the max of two linear forms is handled by one auxiliary scalar, and one solve settles every sparsity at once rather than one feasibility problem per s.

Remark 13 (Why splitting z into two non-negative vectors loses nothing)**.** Algorithm 1 does not constrain u and v to have disjoint supports, so a feasible point need not have $u = z^+$ and $v = z^-$. It is still exact. If $u_j, v_j > 0$ for some j, subtracting $\min(u_j, v_j)$ from both leaves $u - v$ unchanged, keeps feasibility, and strictly decreases both $\mathbf{1}^\top u$ and $\mathbf{1}^\top v$, hence does not increase t. Every optimum therefore admits a canonical split, on which the two budget constraints are exactly those of (6) and the bounds $u, v \leq 1/\alpha$ are exactly $\|z\|_\infty \leq 1/\alpha$.

The cost is one linear programme in $2(F - 1) + 1$ variables with d equality constraints per feature, which is affordable for every feature at $d = 50$ and not at $d = 400$; how we choose a subset there, and why the resulting minimum is an upper bound, is stated in Section 8.

4.1 What the state distribution contributes

Definition 11 allowed graded activations in $[0, 1]$. It is natural to ask what happens under a genuine state law, and the answer is unusually clean: almost nothing survives.

Proposition 14 (Amplitudes wash out of the hulls)**.** *For $\alpha \in (0, 1]$ and $k \in \mathbb{N}$ let $\mathcal{U}_k^\alpha = \{u \in \mathbb{R}^F_{\geq 0} : u_j \in \{0\} \cup [\alpha, 1],\ |\operatorname{supp}(u)| \leq k\}$. Then $\operatorname{conv} \mathcal{U}_k^\alpha = \{u \in [0, 1]^F : \mathbf{1}^\top u \leq k\}$ for every α.*

Proof. The right-hand side is the unit-weight knapsack polytope, which is integral: its vertices are the 0/1 vectors with at most k ones. Each such vertex lies in $\mathcal{U}_k^\alpha$ for every $\alpha \leq 1$, so the right-hand side is contained in the hull; the reverse inclusion is immediate since $\mathcal{U}_k^\alpha$ satisfies both constraints. □

Corollary 15 (The frontier under a minimum amplitude)**.** *If active amplitudes lie in $[\alpha, 1]$ then feature i is separable at every sparsity $s' \leq s$ iff $\kappa_i(\alpha) > s$, where*

$$\kappa_i(\alpha) = \min_z \Big\{ \max\left(\alpha \textstyle\sum_j z_j^+,\ 1 + \alpha \sum_j z_j^-\right) \ :\ \Phi_{-i} z = \phi_i,\ \|z\|_\infty \leq 1/\alpha \Big\},$$

and $\kappa_i(\alpha)$ is non-decreasing in α. As $\alpha \to 0$ an active state converges to an inactive one, the hulls touch, and no uniform margin exists.

Remark 16 (The affine level is a support-level worst case, not a weighted average)**.** It would be convenient to call Corollary 15 a distribution-aware frontier. It is not, and the distinction matters. By Proposition 14 two state laws with the same support family and the same minimum amplitude give the *same* $\kappa_i(\alpha)$, however differently they weight the states in between. The affine level therefore reads exactly one scalar off the state law — the smallest detectable amplitude — whereas by Proposition 9 the analog level reads the full second moment C. That asymmetry is a property of the two criteria, not an artefact of our formulation:

separability is a statement about a worst case over a support family, and a worst case cannot be reweighted. A genuinely probabilistic version, guaranteeing separability for all but a δ fraction of states, is possible but needs machinery we do not open here.

4.2 Margin, certificate, and a dimensional ceiling

The verdict of Theorem 12 is binary. Two further quantities say how comfortably it holds and how it fails, and one says that no code can do well beyond a certain sparsity. The first two are stated for the exactly-s parameterisation in which they were derived and verified, where the admissible set carries the affine constraint $\mathbf{1}^\top z = 1$ and an ℓ_1 budget; we flag that rather than restate them without proof.

Proposition 17 (The robust margin is half the hull distance)**.** *Let $\delta_i(s) = \operatorname{dist}(\mathcal{A}_i(s), \mathcal{B}_i(s))$ be the Euclidean distance between the two hulls. The maximum-margin affine decoder for feature i has $\|w^\star\| = 1/\delta_i(s)$ and margin $\gamma_i(s) = 1/(2\,\|w^\star\|) = \delta_i(s)/2$. Consequently, with the optimal unit-norm w and its centred bias, every perturbed input $x + \eta$ with $\|\eta\|_2 < \gamma_i(s)$ still receives the correct label on every admissible support.*

Proof. The margin programme is $\min \|w\|$ subject to $\min_{u \in D} w^\top u \geq 1$ with $D = \mathcal{A}_i(s) - \mathcal{B}_i(s)$. Support-function duality gives $\max_{\|w\| \leq 1} \min_{u \in D} w^\top u = \operatorname{dist}(0, D) = \delta_i(s)$, whence $\|w^\star\| = 1/\delta_i(s)$. For the perturbation claim, the score moves by at most $\|w\|\,\|\eta\|$. □

The margin is therefore a certified noise tolerance in the units of the representation, not merely a scale, and $\delta_i(s) = 0$ holds exactly when the feature is not separable — the verdict and the margin are two readings of one object.

Proposition 18 (Failure comes with a finite, independently checkable certificate)**.** *If feature i is not separable at sparsity s, there are states $x_1^+, \ldots, x_m^+ \in \mathcal{A}_i(s)$, states $x_1^-, \ldots, x_n^- \in \mathcal{B}_i(s)$ and convex weights λ, μ with*

$$\sum_k \lambda_k x_k^+ \;=\; \sum_l \mu_l x_l^-. \tag{7}$$

No affine rule can label all of them correctly, since affinity would force the same point to lie strictly on both sides of θ. By Radon's theorem the obstruction can be compressed to at most $d + 2$ states.

Remark 19 (What the certificate is, and what it is not)**.** Equation (7) is a convex combination of active states equal to a convex combination of inactive ones. It is *not* a single pair of colliding supports, and describing it as one would be wrong: the obstruction is generically spread over several states on each side. This matters practically, because the certificate is what makes the negative verdict auditable. "The solver reported infeasible" is not evidence; a list of supports from which any third party can rebuild the states from Φ and recompute (7) is. Our implementation stores the supports rather than the cut vectors for exactly that reason, and re-verifies them in a process that never sees the solver.

Proposition 20 (A dimensional ceiling on the affine level)**.** *Call a set C of columns an* affine circuit *if it is minimally affinely dependent: $\sum_{j \in C} l_j \phi_j = 0$ with $\sum_{j \in C} l_j = 0$ and every $l_j \neq 0$. Let q be the smallest size of an affine circuit of Φ. Since any $d + 2$ points in $\mathbb{R}^d$ are affinely dependent, $q \leq d + 2$ whenever $F > d + 1$. Moreover the exactly-s collision radius satisfies $\rho_{\min} \leq q - 1 \leq d + 1$, and hence no code supports featurewise affine support recovery beyond sparsity $\lceil d/2 \rceil$.*

Proof. Given a circuit C, pick $i \in C$ with $|l_i|$ largest and set $z_j = -l_j/l_i$ for $j \in C \setminus \{i\}$ and $z_j = 0$ otherwise. Then $\Phi_{-i} z = \phi_i$; the affine constraint $\mathbf{1}^\top z = 1$ holds automatically, because $\sum_{j \in C} l_j = 0$ forces $\sum_{j \neq i} l_j = -l_i$; $\|z\|_\infty \leq 1$ by the choice of i; and $\|z\|_1 \leq q - 1$. Hence $\rho_i \leq q - 1$. The sparsity bound follows from the threshold $2\min(s, F - s) - 1$ of the exactly-s rule. □

This is the affine counterpart of Theorem 1: one bounds error from below, the other bounds sparsity from above, and both follow from the ambient dimension alone.

5 Released dictionaries are not near the floor

A sparse autoencoder's decoder is an overcomplete code in exactly the sense of Section 2: F unit-norm directions in $d < F$ dimensions, used in practice by taking inner products with feature directions. It is therefore an object the diagnostic applies to without adaptation, and it is one that people deploy. This section measures 40 of them.

The gauge, stated before anything is measured. Leverage is not invariant to rescaling a column, so a measurement of it is meaningless without a declared normalisation. Every decoder analysed here, and every control it is compared against, is normalised to unit column norms before anything is computed (`unit_cols` in `scripts/sae_diagnostic.py`). That is the same convention the theory of Section 3 assumes, and it is what makes the comparison against a shape-matched control a comparison of directions rather than of scales. Without it the excess we report could be manufactured: shrinking a handful of columns of an i.i.d. code, directions untouched, moves its ratio further than any dictionary in this section sits above its control.

A residual of the same kind survives the gauge and we do not resolve it. A dead latent — a decoder column whose encoder never fires on the data — still has a direction, still enters the row space, and still moves the leverage of every other feature, while changing nothing the model computes. So two functionally identical autoencoders can receive different scores here. Fixing that needs matched activation data to define the active support, which this section deliberately does not use; we return to it in Section 8.

What is and is not computed. Everything below comes from a decoder matrix. No model weights are loaded, no activations are collected, no autoencoder is trained, and no linear programme is solved. The cost is seconds per dictionary, which is why the section can span five model families rather than one. The price is that these are statements about the geometry a dictionary presents to a linear reader, not about what its features mean or how well it reconstructs; the reconstruction metrics its authors report are a different and complementary measurement.

Design. Four axes, chosen so that each isolates one variable (Table 2). *Scale*: Qwen-Scope residual dictionaries at $d = 2048$ and $d = 4096$ with load and recipe held at 16× and TopK. *Load*: EleutherAI's Pythia-160m dictionaries at one layer with k fixed at 64 and width varying, giving 5.3× to 85.3× at constant d. *Causal*: EleutherAI's SmolLM2-135M pair, one arm trained on the model and one on a randomly initialised copy of it, same recipe, same data, same ten layers. *Architecture*: the MLP down-projection of LFM2.5-350M, a hybrid convolution/attention model rather than a standard transformer.

Every code is compared against an i.i.d. Gaussian code of *its own* shape, evaluated at *its own* published operating sparsity. That matching is not cosmetic. Because $\sum_i h_i = d$, the mean leverage of any code is exactly d/F, so absolute quantities like the fraction of features a bound rules out are properties of the shape and not of the dictionary. Only the departure from a shape-matched control carries information about the code.

5.1 Near-floor geometry is generic

Each control is a single i.i.d. draw at its block's shape rather than a Monte Carlo ensemble. That is deliberate and it is defensible: by Proposition 5 the ratio is a function of the leverage profile, whose concentration at these dimensions is tight enough that the whole set of controls spans less than $1.0005 - 1.0000$ in R_{geom} across every shape and load we measure. A reader who wants the sampling spread rather than our word for it can generate it in seconds from the released script.

The 7 matched controls attain the floor: R_{geom} from 1.0000 to 1.0005 across every shape and every load from 4.5× to 85.3×, with leverage dispersion 0.005 to 0.020. This is Proposition 5 in action rather than a coincidence: an i.i.d. code concentrates its leverage tightly around d/F, and a code with equalised leverage sits at the floor by the identity.

The consequence for how such ratios are read is direct. A reported attainment ratio near one, under a pseudoinverse readout, is consistent with a code that has learned nothing at all. It becomes evidence about

training only when compared against a shape-matched baseline, and that comparison is the first thing we could not find in the work we build on.

It is worth seeing how little the ratio constrains a code, because the identity makes the extreme case constructible. Stack k copies of the same orthonormal basis, $\Phi = [I_d\ I_d\ \cdots\ I_d]$ with $F = kd$. Then $\Phi\Phi^\top = kI_d$, every leverage score is exactly $1/k = d/F$, and by Proposition 5 the code attains the floor exactly: $R_{\text{geom}} = 1$. Yet features i and $i + d$ have identical directions, so no readout of any kind — linear, affine or otherwise — can tell which of the two is active. A code can therefore sit exactly on the rank–trace floor and be maximally unidentifiable. Attaining the floor is not a certificate of anything; it is the statement that the unavoidable cross-talk is spread evenly, and evenness is compatible with duplication. This cuts both ways, and we take the cut: it is why we report the excess above a matched control as a measured difference in leverage dispersion, and not as a claim that one code is "better conditioned" than another.

5.2 No released dictionary we measured is at the floor

All 40 of the 40 dictionaries sit above their own control, at R_{geom} from 1.0055 to 1.2018. There is no exception across two decoder sites (residual stream and MLP), three training recipes (TopK, JumpReLU-style and a hybrid model's own projection), five model families, and a twentyfold range of load.

The direction is worth pausing on, because it is the opposite of the natural guess. One might expect a dictionary trained to represent a model's features to be at least as well arranged for linear readout as an unstructured code. It is not: a random code of the same shape carries strictly less cross-talk under the best linear readout each admits. Whatever these dictionaries optimise — sparse reconstruction of activations, under an ℓ_0 or TopK constraint — it is not the evenness of their own leverage, and the two objectives are not aligned.

By the identity, the excess has a single source: leverage dispersion, 0.090 to 0.469 against the controls' 0.005 to 0.020. A released dictionary allocates its directions unevenly with respect to the space it lives in, and a handful of features with small h_i dominate the harmonic sum that sets the ratio.

The second statistic agrees, in sparsity units. The diagnostic returns a failure threshold as well as a ratio: the smallest sparsity at which Corollary 34 guarantees that some feature cannot be recovered by any affine rule. Its absolute value is a property of the shape — at the operating sparsities these dictionaries are used at, the sufficient condition rules out essentially every feature of the controls too — so it carries information only as a matched difference. Matched, it is unanimous: all 40 of 40 dictionaries reach guaranteed failure *earlier* than the i.i.d. code of their own shape and operating sparsity, by 4 to 40 sparsity levels, with the trained thresholds running from 13 to 45. This is a different functional of the leverage distribution from R_{geom} — a worst-case statement about one feature rather than an average over all of them — and it points the same way, which is the main reason we report it.

Depth matters more than width, but not in one clean ordering. The excess is far from uniform across a model, and within a suite it varies by more than any width effect we measure below. In the three Qwen suites the pattern is clean: mid-depth dictionaries sit furthest from the floor and the deepest layer sampled sits closest — $R_{\text{geom}} = 1.2018$ at layer 17 of Qwen3-8B against 1.0178 at its own last layer. In the other two suites it is not. The deepest LFM2.5 layer is 1.0240 while the closest to the floor is the layer before it (1.0083), and the deepest SmolLM2 layer is 1.0319 against a minimum of 1.0105 at layer 3. So the safe statement is the one the data supports in every suite: the layer a dictionary is trained on changes its linear readability by far more than its width does, and the direction of that change is not the same in every family.

5.3 A randomly initialised control shows where the excess comes from

The obvious objection to the previous subsection is that training a TopK autoencoder might produce uneven leverage on any input, in which case the excess would say nothing about the model. An i.i.d. control cannot answer this, because it does not share the training procedure.

EleutherAI released the pair that does. Two dictionaries trained by an identical recipe on identical data, one on SmolLM2-135M and one on a *randomly initialised* copy of it, at the same ten layers and the same 64× expansion. The randomly initialised arm returns to the floor: $R_{\text{geom}} = 1.0038$ to 1.0065 with dispersion 0.060 to 0.078, against the trained arm's excess in all 10 paired layers. Its values are also nearly flat across depth, where the trained arm has clear structure.

Fitting a sparse autoencoder therefore does not, by itself, produce dispersed leverage. The excess tracks what the model represents. This is the strongest causal statement the section supports, and it rests on one model pair, which is the only such pair we are aware of — a limitation we return to in Section 8.

5.4 Load and dispersion come apart

Holding d and k fixed and varying only width, the ratio is flat while the dispersion grows: R_{geom} of 1.0674, 1.0749 and 1.0734 at loads of 5.3×, 42.7× and 85.3×, while $\text{cv}(h)$ climbs from 0.276 through 0.404 to 0.469.

Without the identity this reads as an anomaly. With it, it is a prediction: R_{geom} is set by $\sum_i h_i^{-1}$, a harmonic quantity dominated by the smallest leverage scores, whereas the coefficient of variation is a second-moment summary sensitive to the upper tail. Extra spread at higher load can therefore accumulate in a part of the distribution the ratio does not see. Across all 40 dictionaries the excess is ranked almost perfectly by the harmonic-to-arithmetic gap of the leverage and only well by the dispersion, which is the same statement measured rather than derived.

Two things follow for practice. Widening a dictionary at fixed d does not improve — and does not much worsen — how well it can be read linearly. And the coefficient of variation, the obvious summary to reach for, is the wrong one: it moves when the ratio does not.

6 What training buys: controlled campaigns

Released dictionaries tell us what trained codes look like; they cannot tell us what any particular choice during training does to them, because nothing about them was under our control. This section supplies the intervention. We train 400 small networks in which the loss, the width, the feature load and the training sparsity are set by us, measure the same statistics on the resulting codes, and retain every trained model so that later analyses need no retraining — three of the results below were computed months after the runs, from the saved weights alone.

6.1 The network against an affine probe of its own representation

The campaigns in Appendix F compare the network's thresholded output against *fixed* readouts of its code. That understates the baseline, and the audit that prompted this campaign said so. Stage A replaces it: 180 models at $d \in \{50, 100, 200\}$ with $F = 2d$, 20 seeds per cell, three code families (L^4, L^2, and an untrained random code), and an affine probe fitted over three families, two representations, three threshold policies and a six-point regularisation grid, all selected on validation with the test split opened once. `tests/test_probes.py` poisons the test set and asserts that no selected hyperparameter moves, so leakage is excluded structurally rather than by inspection.

Splits, and where they cannot be disjoint. Train, validation and test draw *disjoint supports*, which is a stronger guarantee than disjoint states and is what makes a fitted probe's test score meaningful. It cannot hold unconditionally: at $s = 1$ there are only F supports in total — 100, 200 and 400 at the three widths — against the thousands of states each split requests. The sampler therefore serves the test split first, caps the smallest sparsities at what the space contains, and distributes the remainder proportionally; it records which sparsities were exhausted and how many draws were rejected, and it raises rather than silently returning an empty split. At $F = 100$ the $s = 1$ test set holds 50 states, not 4096, and the sparsities $s \in \{1, 2\}$ are flagged exhausted. Every per-sparsity rate at those sparsities is therefore computed on fewer states than the nominal budget, and its Wilson interval is correspondingly wider.

The comparison has to be matched, and matching it changes the answer. A first pass compared the network at a fixed $\theta = 1/2$ against the best of all eighteen probe configurations. That stacks three advantages on the probe: two thirds of the configurations tune their threshold on validation while the network does not; two thirds read the *pre*-ReLU state Φb while the network's output layer reads $\mathrm{ReLU}(\Phi b)$; and $\theta = 1/2$ is not on a common scale across a ridge prediction, a logistic probability and a hinge margin. Removing all three — probe restricted to the post-ReLU state, one validation-selected global threshold for each side, differences paired within seed — gives Table 3.

Across the 120 trained models the two tie in 103, with 15 models where the network leads and 2 where the probe does. That single count needs breaking down before it is read, because 60 of the ties are L^2 cells in which both decoders sit at $s_{95} = 0$: those are joint failures, not agreements, and counting them as ties inflates the apparent evidence for equivalence. Among the 60 L^4 models the split is 43 ties, 15 to the network and 2 to the probe, and the difference grows with width.

The registered outcome was that a properly trained, validation-selected affine decoder would *match* the network. Under s_{95} it does at the smallest width and stops doing so as width grows. Under the other co-primary estimand it does not hold even there: the AUC difference at $d = 50$ is 0.0053 ([0.0038, 0.0068]) on 19 of 20 seeds, so the registered prediction fails at every width once the finer statistic is used, and what s_{95} shows at $d = 50$ is a difference below its own resolution rather than a match. We report that, and Section 6.3 states what the comparison can and cannot mean, because the probe class contains the network's own decoder and the gap therefore measures the fitting procedure rather than the expressiveness of affine maps.

The sign is set by the code, not by the protocol. The obvious objection to the paragraph above is that a protocol giving both sides a validation-selected threshold must end up favouring the network, which has one output layer to calibrate against the probe's eighteen configurations. The untrained arm answers it. Run through the identical procedure, an untrained code's own thresholded readout *loses* to the fitted affine probe by -1.00, -2.00, -2.30 and -2.60 sparsity levels at the four widths, on every one of the 20 models per cell at $d \leq 200$ and all 10 at $d = 400$, with AUC differences of -0.2122 to -0.1782 — two orders of magnitude larger than any L^4 cell and in the opposite direction. A procedure that systematically favoured the network could not produce that.

So across the three code families the same protocol gives three different answers, and which one it gives is decided by how the code was made: the network leads under L^4 and increasingly with width, neither decoder recovers anything under L^2, and the probe leads by a wide margin on a code that was never trained. Read as a decoder comparison this table says little; read as a property of the code, the ordering is the result.

The second estimand agrees, and the earlier disagreement was an artefact. The analysis plan named two co-primary estimands: s_{95} and the recovery AUC, the area under the exact-recovery curve over the evaluated sparsities. s_{95} is a floored crossing point and therefore blunt, so a difference of less than one level is invisible to it; the AUC sees the whole curve. On the same matched post-ReLU comparison the AUC is positive at every width and grows with it: 0.0053 (95% interval [0.0038, 0.0068]) on 19 of 20 seeds at $d = 50$; 0.0210 ([0.0193, 0.0226]) on 20 of 20 at $d = 100$; 0.0379 ([0.0365, 0.0394]) on 20 of 20 at $d = 200$; and 0.0699 ([0.0670, 0.0726]) on 10 of 10 at $d = 400$. Every cell is unanimous across seeds and no interval covers zero.

An earlier version of this section reported the opposite: an AUC that changed sign with width, which we described as the two estimands disagreeing and treated as the finding. That reading was produced by a defect in how the shared threshold was selected (Appendix G), which cost the network up to five sparsity levels at the widest setting. With the defect fixed the two estimands concur, and their concurrence is the more informative outcome: a difference visible to a blunt statistic and to a fine one, in the same direction, is harder to attribute to the choice of summary. Figure 1(a,b) shows both estimands at all four widths so that a reader can see the agreement rather than take our word for it.

The nonlinearity costs almost nothing on a trained code, and a great deal on an untrained one. Restoring the pre-ReLU probe — which reads the preactivation Φb, a state the network's output layer

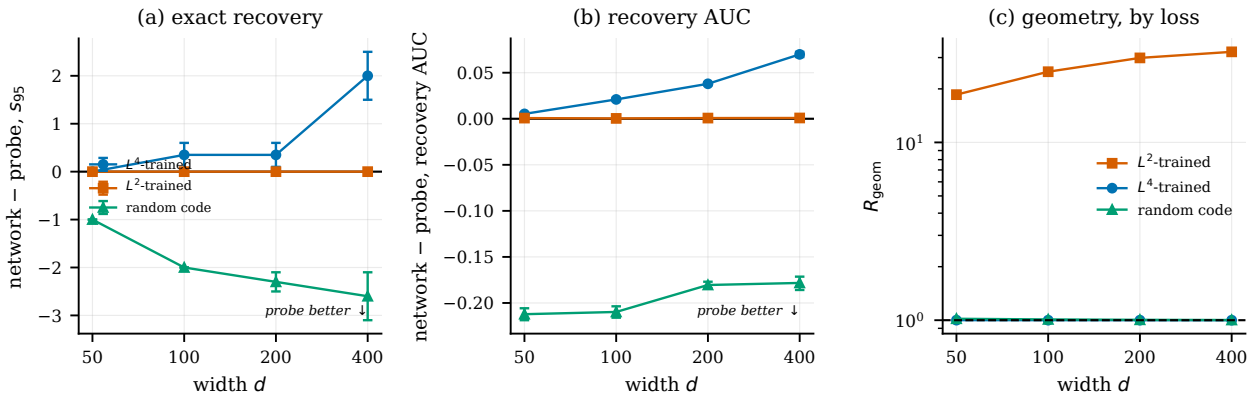

Figure 1: **The matched comparison, both estimands, and the geometry the loss produces.** Paired within seed; bars are bootstrap intervals over the models of a cell. Negative means the affine probe decodes better. **(a)** Under s_{95} the matched post-ReLU comparison separates the three code families in three directions: the network leads under L^4 by an amount that grows with width, L^2 is a flat zero because neither decoder recovers anything, and on an untrained code the probe leads by one to two and a half levels. The protocol is identical in all three. **(b)** The same models under the recovery AUC, which resolves differences smaller than one sparsity level and agrees with s_{95} at every width — an earlier version reported a sign reversal here, which was an artefact of the threshold defect (Appendix G). **(c)** R_{geom} by loss. L^2 leaves the interface one to two orders of magnitude above the rank–trace floor; L^4 and an *untrained* random code both sit on it, which is why proximity to the floor is not by itself evidence of learned structure.

never sees — is how we had previously located the effect, and on the L^4 codes it now finds very little. The pre-ReLU probe's advantage is -0.10 of a sparsity level at $d = 50$ with 18 of 20 seeds tied, -0.25 at $d = 100$, -0.25 at $d = 200$, and at $d = 400$ it reverses sign: the network leads even the pre-ReLU probe by 0.60 on 7 of 10 models. An earlier version of this paper read a full-level gap here as a measurement of what the ReLU discards; that reading was an artefact of the threshold defect and is withdrawn (Appendix G, item 2).

On the untrained arm the same comparison is an order of magnitude larger and unanimous: -2.45, -4.05, -6.55 and -10.90 levels to the pre-ReLU probe, growing steeply with width. So the pipeline of ReLU plus a single global threshold throws away a large and growing amount of affinely accessible information on a code that was never trained, and next to none on one trained under L^4. That is the surviving form of the claim, and it is a statement about the interaction between the code and the readout rather than about the nonlinearity alone.

The loss decides the interface. The three code families separate cleanly and by a wide margin, at every width:

	$d = 50$	$d = 100$	$d = 200$
R_{geom}, L^4	1.0006	1.0003	1.0002
R_{geom}, L^2	18.5635	24.9782	29.8734
R_{geom}, random	1.0187	1.0099	1.0049
$\kappa_{\min}$, L^4	4.4873	6.0155	8.0352
$\kappa_{\min}$, L^2	1.0000	1.0016	1.0146
$\kappa_{\min}$, random	3.1612	4.4289	6.2377

L^2 drives R_{geom} into the tens and rising with width, and its frontier collapses to 1. By Theorem 12 a feature is separable at s only when $\kappa_i > s$, so a frontier at 1 means the worst feature is separable at singletons and at nothing beyond them. Only at $d = 50$, where $\kappa_{\min} = 1.0000$ exactly, does the bound reach the degenerate case in which two features share a representation; at the larger widths $\kappa_{\min}$ is 1.0016, 1.0146 and 1.0653, so every feature is separable at $s = 1$ and fails at $s = 2$. The mechanism is a spread of column norms: a feature

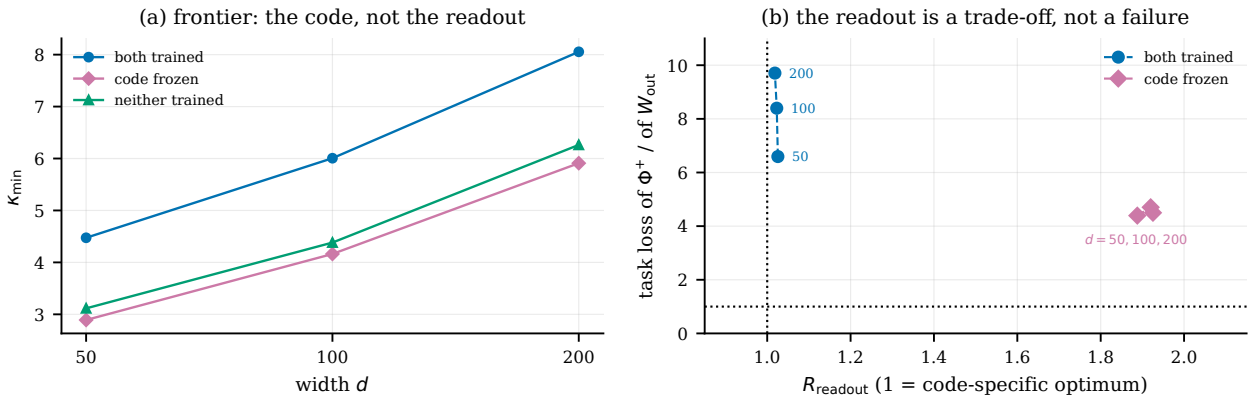

Figure 2: **Which half of the training does the work.** Medians over 10 models per arm and width. **(a)** The frontier tracks the code: training only the readout leaves $\kappa_{\min}$ where an untrained code puts it — below it, in fact — while training both lifts it at every width. **(b)** The same models placed by cross-talk against task loss. The horizontal axis is R_{readout}, so 1 is the best readout *of the code that arm was given*; the vertical axis is how much lower the trained readout's own training loss is than that optimum's. The frozen arm sits at nearly twice the cross-talk, which invites reading it as a failed optimisation, but it beats the cross-talk optimum on the loss it trained under, so the cross-talk is bought rather than lost. Its three widths coincide: neither coordinate moves as the problem grows. The jointly trained arm is better on both axes at once and pulls further ahead with width, so co-adaptation does not trade the two objectives against each other — it aligns them.

whose direction is a fraction of a typical length lies inside the convex hull of the others, and no duplicate is needed. L^4 does not do this, and its R_{readout} of 1.0263 says its own output layer is near-optimal for the code it built.

But near-floor geometry is generic. The untrained random code reaches $R_{\text{geom}} = 1.0187$ at $d = 50$ and its frontier is 6.2377 at $d = 200$, against L^4's 8.0352. So the distance from the rank–trace floor, which earlier work reads as a signature of learned structure, is what an i.i.d. code gives for free at this load; the trained code is better, by one to two sparsity levels, and that margin is the honest size of the effect.

And the margin comes from the code, not the readout. The random control differs from a trained model in two ways at once — its code is not trained *and* its readout is Φ^+ rather than learned — so it cannot say which half matters. E8 separates them with three arms on an identical budget, 10 seeds each: both trained; a random code held *frozen* with only W_{out} trainable; and neither trained.

	$d = 50$			$d = 100$			$d = 200$		
arm	R_{geom}	$\kappa_{\min}$	R_{ro}	R_{geom}	$\kappa_{\min}$	R_{ro}	R_{geom}	$\kappa_{\min}$	R_{ro}
both trained	1.0006	4.468	1.0258	1.0003	6.014	1.0228	1.0002	8.060	1.0185
frozen code	1.0427	2.880	1.8978	1.0209	4.122	1.9217	1.0097	5.874	1.9201
neither	1.0201	3.106	1.0000	1.0100	4.373	1.0000	1.0048	6.272	1.0000

R_{ro} abbreviates R_{readout}. The frozen arm is at $d \in \{50, 100\}$ from one run and $d = 200$ from a second, added after review asked for the third width; the two carry separate run records rather than being merged.

Training the readout alone leaves the frontier where an untrained code puts it, at all three widths: 4.122 against 4.373 at $d = 100$ and 5.874 against 6.272 at $d = 200$, while training both reaches 6.014 and 8.060. The gain is entirely in Φ. That is unsurprising for κ, which is a function of the code alone — the frozen and untrained rows must agree up to the column normalisation that distinguishes them, and their agreement is a consistency check rather than a result — but it is the point: whatever the training buys for support recovery, it buys by moving the code.

The readout column is the surprise, and it is stable across width. The frozen arm trained W_{out} for the full budget and ended at $R_{\text{readout}} = 1.9217$ at $d = 100$ and 1.9201 at $d = 200$ — nearly twice the cross-talk of the best readout *of the code it was given* — while the jointly trained arm reaches 1.0228 and 1.0185.

It is tempting to read that as gradient descent failing at the conditional problem, and it is not. The cross-talk optimum is attained by Φ^+ for *every* code, so a readout with R_{readout} near 1 always exists and its existence settles nothing; what matters is whether the arm's own objective prefers its W_{out} to that optimum. It does, decisively. Scored under the loss the arm trained on, in its own post-ReLU representation, on fresh draws from a seed no training used (`scripts/frozen_readout_tradeoff.py`), the frozen W_{out} beats Φ^+ by a factor of 4.70 at $d = 100$ and 4.50 at $d = 200$, on 10 of 10 models at each width. The cross-talk it gives up is bought, not lost. As a check on the scoring, the untrained arm — whose readout *is* Φ^+ — returns a ratio of 1.00. Figure 2(b) places both arms in the two coordinates at once, which is the only view in which the trade-off is visible as a trade-off.

So the two objectives genuinely pull apart on a fixed random code, and the contrast is about what co-adaptation does to them rather than about optimisation difficulty: the jointly trained arm attains $R_{\text{readout}} = 1.0185$ *and* a task advantage of 9.71 at $d = 200$, so it does not trade one against the other at all, while the frozen arm can only buy the second with the first. The joint arm's task advantage also grows with width, 6.59 to 8.40 to 9.71, where the frozen arm's is flat (4.39, 4.70, 4.50). Training the code appears to align the task objective with the cross-talk objective; freezing it leaves them in conflict that no amount of readout training resolves.

The frontier is now exact. Earlier we reported $\kappa_{\min}$ over a subset of features and called it an upper bound. Evaluating every feature of all 180 models shows the bound was tight: the subset overestimated by 0.0206 on average at $d = 200$ and never by more than a tenth of a level. Its *justification* is weaker than its accuracy, though. The subset was chosen by Theorem 32, and that theorem is sufficient rather than necessary: on trained codes it locates the true argmin in only 6 of 20 models at $d = 200$, where the true argmin sits at leverage rank 38 by median, outside the window. On untrained codes it succeeds every time. Training pulls the lowest-leverage feature and the worst-frontier feature apart, and the numbers above are the exact ones.

A fourth width, committed to in advance. Before running it we extrapolated the frontier from $d \in \{50, 100, 200\}$ and registered $\kappa_{\min} = 10.79$ at $d = 400$ in `configs/e7_stageC.json`. Then we ran the width — 10 models per arm at $F = 2d = 800$, same protocol — and measured 10.510, an error of -2.60% against the registered value.

Two caveats on that number, both of which make it weaker than it first reads. The registered extrapolation used the subset frontier, the only statistic available when it was written; refitting on the exact frontier gives $\kappa_{\min} \propto d^{0.4202}$ and 10.759, an error of -2.31%. We report the registered comparison as the primary one and the refit beside it, because the refit was computed after seeing the measurement and is the more flattering of the two; the three defensible pairings of predicted and measured all fall between -2.2% and -2.6%, so nothing here turns on the choice. And a prediction from three points fitting two parameters is a weak instrument: it tests that the growth is regular, not that any particular law holds. We read the result as a local rate rather than a law, because the exponent drifts downward across the four widths and Section F.4 explains why four points cannot separate the candidate shapes.

Two other things replicate at this width. The loss still decides the interface: L^2 reaches $R_{\text{geom}} = 32.3131$ with its frontier at 1.0653, against L^4's 1.0001 and 10.5098, and the untrained code again sits near the floor at 1.0025 with a frontier of 8.9069. And the matched decoder comparison continues in the direction the smaller widths established, more sharply: at $d = 400$ the network leads the affine probe by 2.00 of a sparsity level ([1.50, 2.50]) in 10 of 10 models, where the three smaller widths gave a tie or a third of a level. Under the conservative reading, in which the probe keeps the maximum over families evaluated on the test set, the difference is 2.00 — so the ordering at this width does not depend on which of the two defensible selection rules is used. Section 6.3 explains why this is a statement about supervised fitting and not about what an affine readout can represent.

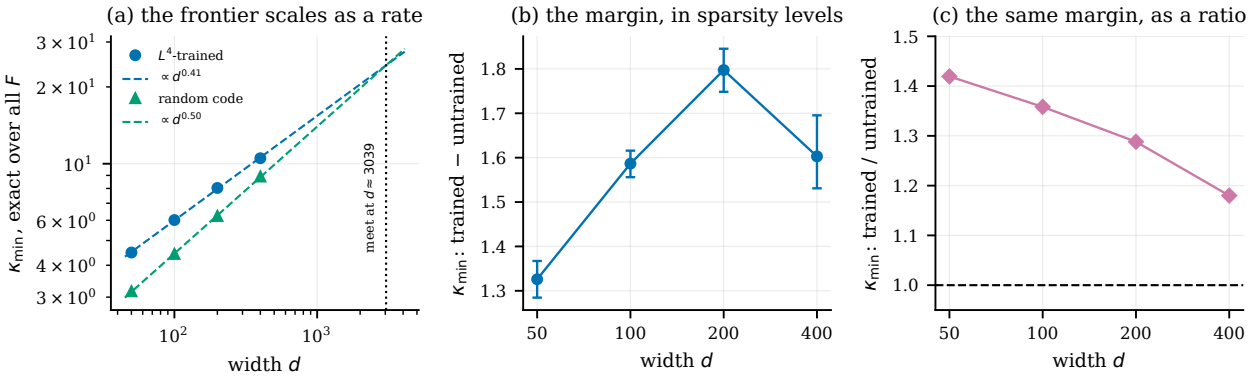

Figure 3: **The frontier's rate, and the margin it puts in question.** All values are the exact $\kappa_{\min}$ over every one of the F features, never the subset. **(a)** Both codes' frontiers are close to straight on these axes over the four widths, and the untrained slope is the steeper. The power-law fits meet near $d \approx 3\,039$, drawn to show what the two slopes imply and not offered as a prediction: it lies far outside the measured range, and four points do not identify the shape — other forms fit as well and move the meeting point by a factor of two. The robust statement is the ordering of the slopes, not where they cross. **(b)** The margin in sparsity levels, with two-sample bootstrap intervals over models; it peaks at $d = 200$ and the fall to $d = 400$ exceeds the sampling noise. **(c)** The same margin as a ratio, which falls at every step. The two panels are shown separately because they are the two honest readings of the same quantity and they do not agree until $d = 400$.

The margin over an untrained code is closing, and that is the finding of this width. Both frontiers grow with width, but not at the same rate: fitted over all four widths the trained code gives $d^{0.4101}$ and the untrained one $d^{0.4977}$ (Fig. 3a). The untrained exponent is the larger, so the advantage of training is a decreasing function of width in both readings of "advantage". As a ratio it falls throughout and faster each time: 1.4195, 1.3582, 1.2882, 1.1800. In absolute sparsity levels — the operationally meaningful unit, since the frontier licenses supports up to $\lceil \kappa_{\min} \rceil - 1$ — it rises to a peak at $d = 200$ and then falls: 1.3260, 1.5866, 1.7974, 1.6029. The fall is larger than the sampling noise: a two-sample bootstrap over models — the arms are separately trained, so the difference is between independent groups and not paired — gives a drop of 0.1940 with interval [0.0903, 0.2818], and the two widths' intervals do not overlap. Taken at face value the two fitted laws meet near $d \approx 3\,039$.

We state this rather than the extrapolation. Four widths spanning a factor of eight cannot establish where two rates cross, and $d \approx 3\,039$ is well outside the range measured. What the data does support is narrower and still consequential: *training buys frontier at every width we measured, and the amount it buys stops growing by* $d = 400$. A reader entitled to conclude that gradient descent finds structurally better codes would want that margin to be at least stable in width, and here it is not. Whether the trained code's deceleration reflects a real ceiling or the fixed optimisation budget — 50 000 steps at every width, so the largest models get the least optimisation per parameter — this campaign cannot say, and it is the first thing a larger budget should test.

And the shortcut has stopped being defensible. The same width shows the subset heuristic's justification collapsing, which is why every number in the two paragraphs above is the exact minimum over all F features. The mean overestimate grows 0.0025, 0.0118, 0.0206, 0.0398; the median leverage rank of the true argmin grows 5, 23, 38, 131; and at $d = 400$ the subset locates the true argmin in 2 of 10 trained models. The error is still small in absolute terms, but it is one-sided and it is asymmetric between the arms: untrained codes keep the true argmin at leverage rank 1, so their subset value *is* exact, while trained codes drift. A margin computed from subset values is therefore biased in favour of training by an amount that grows with exactly the variable under study — which is why we computed the exact frontier for all four widths before making the claim above, and why we would not trust the shortcut past $d = 200$ in future work.

6.2 A second campaign, at double the feature load

Everything above comes from one campaign at one feature load, $F = 2d$. Stage B is a separate campaign, run with its own seeds and its own run record, and it varies that load (Table 4). Its $p = 0.01$ cells sit at $F = 4d$ — twice stage A's load, at $d \in \{100, 200\}$ — and its $p = 0.02$ cells stay at $F = 2d$ and double the training sparsity instead. Every other setting matches stage A: the same optimiser, step count, batch size, probe budget and evaluation grid, verified against the two configuration files.

An earlier version of this section described the $p = 0.01$ cells as exact repeats of stage A and read their larger effects as a replication. That was wrong: they are a different design, and Appendix G records the error and what it cost.

At double the load the ordering holds and the frontier falls. The decoder comparison keeps its sign and grows: the network leads the affine probe by 0.70 of a sparsity level at $d = 100$ ([0.40, 1.00], 7 of 10 models) and 1.00 at $d = 200$ ([0.70, 1.30], 9 of 10), against 0.35 and 0.35 at the same widths and half the load. The untrained arm keeps its own opposite sign, -1.80 and -2.00 levels to the probe; and every L^2 cell has both decoders at $s_{95} = 0$, 40 models across 4 cells, the same joint failure stage A shows.

The frontier moves the other way, as it must: doubling the number of features in the same d costs recovery, and $\kappa_{\min}$ falls from 6.0155 to 4.3072 at $d = 100$ and from 8.0352 to 5.6212 at $d = 200$. So the two campaigns together say that the loss–geometry separation and the sign structure of the decoder comparison are not artefacts of $F = 2d$, while the absolute frontier is a strong function of the load. The comparison is across campaigns with different seeds, which is why we read the ordering and not the difference of the means.

Training sparsity, at matched load, buys nothing. Stage B's $p = 0.02$ cells are the only place any training variable other than the loss is varied, and they must be compared against stage A's cells at the *same* load, $F = 2d$ — not against stage B's own $p = 0.01$ cells, which are at twice that. Made correctly, the comparison gives nothing to the hypothesis that denser training states buy frontier. At $d = 50$ the two are indistinguishable, 4.4873 against 4.5074. At $d = 200$ the higher sparsity is *worse*, 7.6479 [7.5971, 7.7063] against 8.0352 [7.9971, 8.0787], with the per-model ranges disjoint.

An earlier version reported the opposite as "the cleanest small result in the campaign" by comparing across the load. We withdraw it. What remains is a null: over the two widths where a matched comparison exists, doubling the training sparsity does not improve the geometry the code presents — indistinguishable at $d = 50$ and worse at $d = 200$ — and the decoder ordering survives the change (L^4 at 0.00 and 0.40, L^2 still pinned at $\kappa_{\min} = 1.0031$). Both matched widths rest on 10 models against stage A's 20, and the direction at $d = 200$ comes from one width, so we read this as an absence of evidence for the withdrawn claim rather than as evidence for its converse.

6.3 What the decoder comparison can and cannot mean

The comparison in the previous subsections is easy to over-read, and the direction of the over-reading is the flattering one. This subsection states the ceiling on it and then measures where the comparison actually sits against that ceiling, because a ceiling one has only argued for is a caveat and a ceiling one has measured is a result.

The probe class contains the network's own decoder. The affine probe is fitted on the post-ReLU state $z = \mathrm{ReLU}(\Phi b)$ with a design matrix that appends an intercept and nothing else (`_design` in `src/lrtr/probes.py`), so its hypothesis class is every affine map of z — which includes W_{out}, the map the network's own output layer applies. Writing that map in the probe's own parameterisation,

$$W_{\text{net}} \;=\; \begin{bmatrix} W_{\text{out}}^{\top} \\ 0 \end{bmatrix} \;\in\; \mathbb{R}^{(d+1)\times F},$$

scoring W_{net} as a probe reproduces the network's own evaluation exactly. It follows that a probe configuration attaining the network's score *exists* for every model in every cell. Whenever the network scores higher, what

the gap measures is that our fitting and selection procedure did not find that map: a fixed ridge grid, a fixed number of steps, and 8,192 training states. It is not a statement that an affine readout cannot represent what the network computes, because it demonstrably can.

The selection misses it, and we can say so by how much. That argument bounds the comparison without telling us whether the bound is tight, so we tested it. The campaign selects a probe configuration by maximising a validation objective — the area under the per-sparsity validation recovery curve, stored for every configuration in the saved records — and W_{net} is a member of the class that objective ranges over. Evaluating the same objective at W_{net} therefore asks whether the selection missed something it had the data to find. It needs no refitting, so it runs over the released weights in seconds per model (`scripts/probe_ceiling.py`).

It did miss it, in every cell where the network leads and in every model of those cells. The validation gap between the network's own map and the selected probe is 0.0222 ([0.0195, 0.0249]) at $d = 100$, 0.0368 ([0.0351, 0.0386]) at $d = 200$ and 0.0715 ([0.0677, 0.0747]) at $d = 400$, all positive, and positive in 80 of 80 models across the 6 L^4 cells at $d \geq 100$ in both campaigns, at both feature loads (0.0261 and 0.0520 in stage B's $F = 4d$ cells).

The gap tracks the reported difference in size as well as in sign. It is largest at $d = 400$, the width where the network's lead is 2.00 levels, and smallest at $d = 50$ (0.0039, [0.0025, 0.0055]), the width where the lead is zero. It is not zero there, so we do not offer $d = 50$ as a null: a small failure to find the better map need not cost a whole sparsity level, and s_{95} is floored.

The L^2 cells are the internal control for the measurement itself. There neither decoder recovers anything and the network has no lead, and there the validation gap sits at essentially zero. So a positive gap is not something this analysis produces wherever it looks: it appears where the network leads, vanishes where nothing does, and reverses where the probe leads.

So the residual L^4 difference is a search failure, measured rather than assumed. A better member of the probe's own class was available, validation could see that it was better, and the procedure returned something else.

The control rules out the obvious objection to that. One could suspect this of being an artefact — that the network's own map simply always looks good on a validation objective computed from the network's own states. The untrained arm says otherwise, and by a wide margin in the opposite direction: there the selected probe beats W_{net} on validation by -0.2180 at $d = 50$, -0.1829 at $d = 200$ and -0.1813 at $d = 400$, and the selection fails to beat it in 0 of 90 models. On a code that was never trained the fitting procedure finds a far better affine map than the network's own readout, which is exactly what it fails to do on a trained one.

As a validity check, this analysis rebuilds each cell's split bundle from the campaign's own seed and recomputes both sides from the released weights. The differences it obtains reproduce the published ones exactly — a worst-cell discrepancy of 0.00 of a sparsity level across all 22 cells of the three campaigns, Tables 3 and 4 together. The check is stated over every cell rather than over the ones it happens to pass: an earlier version was scoped to Table 3 alone, which excluded the single cell where it then failed.

What the comparison does and does not support. It does not support "the network's nonlinearity buys expressive power": the class contains its decoder, and the gap is now *known* to be a search failure rather than merely arguably one. We report the L^4 differences of 0.35 to 2.00 and build nothing on them.

What the comparison does support is a property of the code, because the protocol's asymmetries are held fixed while the code varies. Both sides read the same state, both get one validation-selected global threshold, both are scored on the same held-out supports. Under that fixed protocol the sign of the difference is set by how the code was produced: positive and growing with width for L^4, exactly zero for L^2, and negative by one to two and a half levels for an untrained code. A protocol that systematically favoured the network could not produce a two-and-a-half-level loss on any arm.

Read that way the comparison says something we did not expect. Training does not only improve the geometry the code presents (Section 6); it also puts the network's own readout somewhere a supervised fit of the same class, on the same data, with a standard grid, does not reach — while on an untrained code that same fit comfortably outperforms the network's readout. The trained readout is not merely good; it is hard to find by fitting. That is a form of the co-adaptation E8 isolates, seen from the decoder's side rather than the code's.

The question this leaves open. Knowing that the selection misses W_{net} does not say why. Two mechanisms are consistent with it and this experiment does not separate them: the optimiser and grid may simply be too coarse to reach that region, or the probe's surrogate objective — ridge, logistic or squared hinge — may have its optimum somewhere other than at the map that maximises exact support recovery, in which case no amount of better optimisation of *that* objective would find it. Separating them needs a refit *started* at W_{net}: if the fit stays, the grid was too coarse; if it walks away and scores worse, the objective is misaligned. That is the expensive direction — the full grid and the margin fits for every model, where the measurement above needs neither — so `fit_probe` accepts a starting point and `probe_ceiling.py -with-refit` wires it up, and we report the question rather than a partial answer to it.

7 Background and related work

7.1 Computation in superposition

A network of hidden width d *represents* $F \gg d$ *features in superposition* if its activation $a(x) \in \mathbb{R}^d$ is approximately Φb for a feature-encoding matrix $\Phi \in \mathbb{R}^{d\times F}$ and a sparse Boolean b. Following Hänni et al. (2024), features $f_1, \ldots, f_F$ are ε-*linearly represented* by a if there is a readout $R \in \mathbb{R}^{F\times d}$ with $|r_k \cdot a(x) - f_k(x)| < \varepsilon$ for all k and x.

Hänni et al. construct an approximate-linear recursive template for sparse Boolean circuits, alternating a targeted superpositional AND layer with an error-correction layer, and certify emulation of circuits of width $\widetilde{O}(d^{3/2})$. Adler and Shavit Adler & Shavit (2024) give parameter-description lower bounds and explicit constructions for computing in superposition, obtaining a near-quadratic capacity by thresholding after each stage. Adler et al. Adler et al. (2025) reframe superposition combinatorially through feature channel coding, and Kong et al. Kong et al. (2025) show empirically that reducing inter-feature interference at fixed parameter count improves performance. This paper reproves none of these constructions; it identifies the interface-level reason the two regimes coexist.

7.2 Trained toy models of computation in superposition

A recent empirical line studies whether small trained networks actually compute in superposition. Braun et al. Braun et al. (2025) introduced the compressed-computation task — a width-50 network asked to compute 100 sparse ReLU features — as a testbed. Bhagat et al. Bhagat et al. (2026) argued that the standard L^2-trained model largely exploits a mixing term rather than genuinely computing in superposition, and Ferreira da Silva and Heimersheim Ferreira da Silva & Heimersheim (2026b) showed that training under an L^4 loss elicits a solution that does compute in superposition, assigning each feature a sparse codeword over neurons and decoding it with a pseudoinverse of the encoder — that is, a linear readout interface in the sense studied here. In parallel, Ferreira da Silva & Heimersheim (2026a) report perturbation-based evidence for feature-specific error correction in large language models, the mechanism whose certified tolerance drives the $d^{3/2}$ compatibility scale of the Hänni template. This debate is about which decoding interface a trained network maintains; the diagnostic proposed here is a direct instrument for it, and Section F.3 applies it to exactly the models under dispute.

7.3 Frame theory, Welch bounds and cross-Gramians

Classical Welch bounds Welch (1974) lower-bound the correlations of overcomplete unit-norm systems; the maximum-correlation form we use, $\max_{i\neq j} |\langle \phi_i, \phi_j\rangle| \geq \sqrt{(F-d)/(d(F-1))}$, appears for instance as Theorem 2.3 of Strohmer and Heath Strohmer & Heath (2003) in the study of Grassmannian frames. Wal-

dron Waldron (2003) showed that generalised Welch-bound-equality sequences are exactly tight frames, and Datta, Howard and Cochran Datta et al. (2012) derived the entire family of higher-order Welch bounds from a geometric rank/Grammian argument, of which the rank–trace computation behind Theorem 1 is the first-order instance; see also Klappenecker & Rötteler (2005) for the design-theoretic view. For pairs of sequences, Aceska and Kaczanowski Aceska & Kaczanowski (2022) introduced the cross-frame potential, and Christensen, Datta and Kim Christensen et al. (2020) proved Welch-type bounds for the maximum coherence of dual frame pairs, characterising equality via equiangular tight frames.

Theorem 1 sits in this lineage: it is a unit-diagonal cross-Gramian formulation assuming only $\operatorname{rank}(M) \le d$ and $M_{ii} = 1$ — in particular, no duality relation between the two sequences, which, as finite sequences, are of course frames for their spans. We include its short rank–trace proof only to keep the paper self-contained; the proof mechanism is standard in this literature, cf. Datta et al. (2012). The contribution here is the application to readout interfaces, the gain-normalised form (Corollary 2), and the diagnostic built on top of them.

7.4 Compressed sensing and sparse recovery

The threshold-recovery results used below are standard coherence-based and random-dictionary support-recovery arguments. Their role is to make explicit a distinction relevant to neural computation: small-error linear readout requires uniformly small coordinate error, whereas threshold recovery only requires the aggregate interference from the active support to remain below a margin.

The collision frontier of Section 4 sits in this neighbourhood and we place it carefully, because two established literatures ask adjacent questions.

The first is group testing. Recovering a sparse Boolean b from Φb is exactly *quantitative* group testing when Φ is a real measurement matrix, and the combinatorial theory of superimposed, disjunct and separable codes descends from Kautz and Singleton Kautz & Singleton (1964). That theory asks how many tests suffice to identify the *whole* support, by *any* decoder, usually for a matrix to be designed. We ask something weaker and more specific: for a matrix we are given, and one coordinate of it, is that coordinate's presence decidable by an *affine* function of Φb? The gap is real in both directions. A matrix can be identifiable in the group-testing sense while $\kappa_i \le s$, because $\kappa_i \le s$ only requires a collision between *convex combinations* of admissible states, not an exact coincidence of two integer ones. Conversely $\kappa_i > s$ certifies a linear probe with a margin, which identifiability alone does not supply. The restriction to affine decoders is not a weakness of the analysis but the object of interest, since linear probing is how features are read in practice.

The second is the null space property. Writing z' for z extended by zero at coordinate i, the constraint $\Phi_{-i} z = \phi_i$ says precisely that $z' - e_i \in \ker \Phi$, so κ_i is a statement about kernel vectors normalised at one coordinate — the same geometry as the null space property and as the neighbourliness of randomly projected polytopes Donoho & Tanner (2005). Two differences matter. The null space property is quantified over all supports and certifies recovery by ℓ_1 minimisation, a nonlinear decoder; κ_i is per-coordinate and certifies an affine one. And the null space property is hard to verify, which is why the literature tests it by semidefinite relaxation d'Aspremont & El Ghaoui (2011), whereas κ_i is the exact value of one linear programme — a consequence of asking the weaker, one-coordinate question rather than of a better algorithm. The box $\|z\|_\infty \le 1/\alpha$, which comes from bounded activations and has no counterpart in the null space property, turns out to bind rarely, so for most features the frontier is a minimum-ℓ_1-representation quantity and the tools of that literature apply directly.

7.5 Sparse autoencoders

Sparse autoencoder work scales dictionary learning to millions of latents Cunningham et al. (2023); Gao et al. (2024); Lieberum et al. (2024) and studies reconstruction–sparsity trade-offs Michaud et al. (2025). Such results motivate the importance of overcomplete internal representations, but dictionary size is a reconstructive quantity and does not measure the recursive Boolean interface capacities studied here. This paper answers one of the open questions listed by Sharkey et al. Sharkey et al. (2025) concerning what theoretical insight can be gleaned from computation performed natively in superposition.

Delineation from a prior sparse-autoencoder study. The closest prior work measures sparse autoencoders with an overlapping toolset and a different question. Borobia et al. (2026), in *Neurocomputing*, train TopK autoencoders on weight-pruned language models and ask which features survive compression, using reconstruction fidelity and feature-level survival statistics as the instrument. That paper is the reference point for the dictionary measurements here, and the division of labour between the two is worth stating plainly because the objects overlap and the quantities do not. This paper trains no autoencoder, prunes nothing, and measures no reconstruction. It takes decoder matrices as given — including ones that paper never examined, such as the randomly-initialised control of Section 5 — and computes a different quantity: the cross-talk a linear reader of that decoder must accept, its exact decomposition into leverage dispersion, and the sparsity at which affine recovery must fail. No model, dictionary, measurement or result is shared between the two, and neither paper's conclusions depend on the other's.

8 Limitations and what we withdrew

1. **One trained setting.** E5 covers a single toy task at $d = 50$, $F = 100$, with 5 seeds per condition. Nothing here is a claim about large trained language models.
2. **Out-of-distribution evaluation.** The diagnostic is defined on Boolean sparse states, while the toy models are trained on continuous sparse inputs. It therefore measures which interface the learned code *supports*, not what the network does on its native distribution.
3. **Constant sparsity in the application.** The Hänni-template comparison assumes $s = O(1)$, whereas the distributional results allow s up to $O(d/\log d)$; real features are continuous with heterogeneous sparsity.
4. **The floor bounds linear readouts only.** Nonlinear decoders and distribution-specific average-case mechanisms are outside its scope — which is the point of the separation, but also a limit on what the diagnostic can certify.
5. **Fit with few points.** The $c\,d/\ln d$ fit in E6 rests on 4 widths, on which a plain linear law fits nearly as well ($R^2 = 0.9636$ against 0.9882). The data are consistent with the predicted shape and do not discriminate it from the alternatives; the logarithm's base is not identifiable at all.
6. **s_{95} is a point estimate, not a certified threshold.** The trial budget shrinks with width, and at $d = 1\,024$ even a perfect run gives a Wilson lower bound of 0.929— below the 95% level being estimated. The reported thresholds are best estimates; certifying them at confidence would need more trials at the large widths.
7. **The linear side's non-exactness is the theorem, not a measurement.** Wherever we report that a linear readout "cannot be exact", that follows from Theorem 1 by construction for any rank-d unit-diagonal interface. What the experiments measure is the *magnitude* of that error for codes that were actually learned, and how far a threshold decoder gets before it bites.
8. **Correlated curve points.** The nested-support trick makes each point of $p_{\mathrm{rec}}(s)$ marginally unbiased but correlates points within a trial; the Wilson intervals are per-point and should not be read as a joint confidence band.
9. **Open log gap.** The Adler–Shavit capacity bracket retains a single logarithmic factor of slack, which is not resolved here.
10. **The leverage prediction is a sufficient condition, and its selection heuristic decays with width.** Corollary 34 predicts the L^2 collapse from leverage alone and holds at all three widths of Section 6. But it is applied at $h_{\min}$, and it is sufficient rather than necessary, which shows up in where the true worst feature sits: the lowest-leverage window contains the true $\arg\min \kappa$ in 6 of 20 L^4 models at $d = 200$ and 2 of 10 at $d = 400$, where the true $\arg\min$ sits at leverage rank 131 by median — against every model of every untrained code, at every width. Training pulls the two apart, and the gap widens with width, so the theorem localises the failure less and less well as the code improves. Nothing reported depends on the heuristic — the frontiers quoted are exact — but a reader should not take low leverage as a reliable pointer to the binding feature, and past $d = 200$ should not use it as a sampling window at all.

11. **The exact frontier does not extrapolate much further.** Every κ_i reported here is exact — all F features of every model at all four widths, including the 10 models per arm at $d = 400$, $F = 800$. An earlier version of this limitation said the $d = 400$ sweep would be unaffordable and that a fourth width would have to return to a subset estimate; it was run instead, in a few hours on ten CPU workers, and this entry is corrected rather than removed because the estimate was wrong in the direction that mattered. What remains true is the shape of the cost: one linear programme per feature — 0.32 s at $d = 200$, from the recorded sweep times — growing steeply in d while F grows with it, with the wall-clock of every campaign in Appendix F.5. A fifth width would be a different proposition, and the honest statement is that the statistic is exact everywhere we report it and that we do not know where it stops being affordable.

12. **The nonlinear witness is built but not yet used.** The extractor of Section C.1 is implemented and validated — every witness it returns is confirmed inseparable by a linear programme, and on small codes against brute-force enumeration — but two things are missing. The Radon compression to $d + 2$ states is not implemented, so the witnesses are valid and not minimal; and no result reported in this paper is derived from one, so the sharper claim it licenses is not made here.

13. **How much the ReLU costs is measurable in only three cells.** The decoder comparison says the ReLU costs an L^4 code almost nothing and an untrained code a great deal (Section 6). Quantifying that on the frontier rather than on a fitted probe is harder: κ is defined on Φb, and $\mathrm{ReLU}(\Phi b)$ is not a linear image of the state polytope, so the post-ReLU side must be sampled and tested rather than solved. Sampling can only refute separability, never confirm it, so it overestimates the frontier — here by 0.00 to 7.93 levels against the exact κ — and the overestimate cancels only in a difference taken with the same sampler and the same draws on both sides (1 500 draws per side, 30 features per cell, sampler ceiling $s \leq 12$).

    Three of the nine cells give a measurable difference and they agree with the decoder comparison: at $d = 50$ the ReLU costs the L^4 code 0.87 levels $[0.63, 1.10]$ on 22 of 30 features and the untrained code 2.03 $[1.77, 2.27]$ on 29; at $d = 100$ the untrained code loses 4.33 $[4.07, 4.57]$ on all 30. In 3 cells both sides reach the sampler ceiling, so the difference there is censoring and not measurement, and we report no value — including for every L^4 cell above $d = 50$, which is precisely where we would most want one. Every L^2 cell is a flat zero because its frontier is at 1 on both sides. A larger sampling budget would lift the ceiling and we have not spent one, so the claim that the cost is small on trained codes rests on $d = 50$ plus the decoder comparison, and not on the frontier at the two larger widths.

14. **A dictionary-only score is not quite a property of the autoencoder.** Every decoder is analysed under a declared unit-column gauge (Section 5), which removes the scaling freedom, but not the one that matters most in practice. A dead latent — a column whose encoder never fires on the data — still occupies a direction, still enters the row space, and still changes the leverage of every other feature, while leaving the function the autoencoder computes untouched. Two functionally identical dictionaries can therefore receive different scores here, and the dictionaries most likely to carry such columns are the widest ones, which is the axis Section 5 varies. Resolving it needs matched activation data to define the active support, which is exactly the input this section is built to avoid using, so it is a real limit on the method and not only on this application of it. What the section measures is the geometry the released matrix presents; whether that matrix has columns the model never uses is a separate question we do not answer.

15. **One control the analysis wants and does not have.** A random *tight frame*, whose R_{geom} is exactly 1 rather than the 1.0005 an i.i.d. code reaches at worst, would sharpen the claim that near-floor geometry is generic: as it stands the comparison is against a code that is close to the floor rather than on it, so a small residual advantage for the trained code could be conditioning rather than structure.

16. **The margin over an untrained code stops growing inside our own range.** This is the sharpest limitation on the headline claim, and it comes from our own data rather than from something we did not run. Section 6 reports the trained frontier at $d^{0.4101}$ and the untrained one at $d^{0.4977}$: the untrained exponent is larger, the ratio falls at every step, and the margin in sparsity levels peaks at $d = 200$ and falls by 0.1940 $[0.0903, 0.2818]$ at $d = 400$. "Training buys the code" therefore holds at every width we measured and is not established as a property that survives scale. Two candidate explanations are not separated here: a real ceiling on what the frontier can buy, or the fixed optimisation budget of 50 000

steps at every width, which gives the largest models the least optimisation per parameter. Distinguishing them needs a budget sweep at fixed width, which is cheap, and a fifth width, which is not.

9 Discussion

Four readings survive this work, and two that motivated it do not.

The floor is a target, not an obstacle — and reaching it means less than it looks. Gradient descent under the unit-diagonal constraint converges to a unit-norm tight frame (Appendix F), and a network trained on a task with no interference objective lands within 1.0006 of the same bound. But an untrained i.i.d. code reaches $R_{\text{geom}} = 1.0187$ with no training at all, and 7 shape-matched controls land between 1.0000 and 1.0005 across a twentyfold range of load. Proximity to the rank–trace floor is what the ensemble gives for free, not a signature of learned structure, and Proposition 5 says why rather than leaving it as an empirical coincidence: the ratio is exactly one when leverage is equalised, and an unstructured code equalises it. Any attainment ratio reported without a shape-matched baseline is uninterpretable, and we could not find that baseline in the work we build on.

No released dictionary we measured is at the floor, and the excess tracks what the model learned. All 40 of 40 decoders sit above their own control, at R_{geom} from 1.0055 to 1.2018, with no exception across five model families, three training recipes and two decoder sites. The direction is the opposite of the natural guess: a random code of the same shape carries strictly *less* cross-talk under the best linear readout each admits, so whatever sparse reconstruction optimises, it is not the evenness of its own leverage. The matched pair EleutherAI released settles where the excess comes from: an autoencoder trained by an identical recipe on a *randomly initialised* copy of the same model returns to the floor (1.0038–1.0065) in all 10 paired layers. Fitting a sparse autoencoder does not by itself produce dispersed leverage; representing a trained model's features does. For practice this reorders two intuitions: the layer a dictionary is trained on changes its linear readability by far more than its width does, though not in one ordering that holds across families; and the coefficient of variation — the obvious summary — is the wrong one, because it moves when the ratio does not.

The loss decides whether a linear interface exists at all. This is the largest effect we report and the one that replicates most widely. L^2 drives R_{geom} to 29.8734 at $d = 200$ and 32.3131 at $d = 400$ while collapsing its frontier to 1 — its worst feature is separable at singletons and at no larger sparsity — and L^4 stays at 1.0002 with a frontier that grows to 10.510. What distinguishes models is therefore not whether they approach the floor but whether the floor *binds* them, and Stage B reproduces the separation in an independent campaign at twice the feature load.

What training buys is the code, not the readout. A random code with only its readout trained keeps an untrained code's frontier, 4.122 against 4.373, while training both reaches 6.014. The frozen arm's own decoder ends at $R_{\text{readout}} = 1.9217$, near twice the closed-form cross-talk optimum for the code it was handed — and that is not a failed optimisation. It beats that optimum by 4.70 on the loss it was trained under, so on a fixed random code the task objective and the cross-talk objective genuinely diverge and the readout is right to follow the former. What the joint arm gains is that it need not choose: it holds $R_{\text{readout}} = 1.0228$ while beating the same baseline by 8.40. Training the code aligns the two objectives; freezing it leaves them in a conflict no amount of readout training resolves.

What does not survive, first: the decoder framing, in either direction. This work began with the claim that an L^4 network's thresholded output keeps working past where linear readouts of its own code collapse. Against the fixed readouts of Appendix F that appeared true; a matched comparison against a fitted probe was registered as the test, and the registered prediction was a tie. What the corrected comparison gives is neither: across 120 models the two tie in 103 — but 60 of those ties are L^2 cells where both decoders sit at $s_{95} = 0$, which is joint failure and not agreement — with 15 to the network, 2 to the probe, and a difference that grows with width to 2.00 levels in 10 of 10 models at $d = 400$.

We decline to claim that as a nonlinear advantage, and not merely on principle. The probe's hypothesis class contains the network's own output layer, so a probe attaining the network's score exists in every cell; writing that map in the probe's parameterisation and scoring it under the very objective the selection maximises

shows that the selection missed it in 80 of 80 models across every L^4 cell above the tie width, by 0.0222 to 0.0715 of validation objective. The gap is a search failure, measured. The untrained arm is the control that makes this a finding rather than a tautology: there the same selection beats the network's own map by -0.1813 and fails to do so in 0 of 90 models (Section 6.3).

What the comparison does support is a property of the code, because the protocol is held fixed while the code varies: the sign is positive under L^4, zero under L^2, and negative by up to -2.60 levels on a code that was never trained. A single cell of that table says little; the ordering across the three is the result. And it comes with a mechanism: on a trained code the network's own readout sits where a supervised fit of the same class on the same data does not reach, while on an untrained code that fit comfortably beats it. The trained readout is not merely better — it is hard to find by fitting, which is co-adaptation seen from the decoder's side.

And second: the representation gap. An earlier version located the effect in what the ReLU discards, reporting that a probe reading the preactivation beats the network by a full sparsity level. That was an artefact of the threshold-selection defect. On trained codes the pre-ReLU advantage is now at most a quarter of a level with most seeds tied, and at $d = 400$ it reverses. The surviving form is narrower and confined to the untrained arm, where the preactivation is more affinely decodable by -2.45 to -10.90 levels: the ReLU-plus-threshold pipeline throws away a large and growing amount on a code that was never trained, and next to none on one trained under L^4. Appendix G records both withdrawals with their causes, along with four others.

An instrument, and what it does not settle. This gives the debate in Bhagat et al. (2026); Ferreira da Silva & Heimersheim (2026b) something to measure. "Does this network compute in superposition?" becomes three questions with numerical answers: how far the code is from its own floor rather than from the ensemble's, how that distance decomposes into leverage dispersion, and, per feature, the largest sparsity at which any affine rule could succeed. The first two are statements about the row space under a declared unit-column gauge, and are invariant to any invertible transformation of the ambient coordinates — which is what makes them properties of the code rather than of a basis, and equally what stops them from being claims about numerical conditioning or noise robustness. None requires reverse-engineering a mechanism, and the third is exact rather than sampled — which matters, because this project's history is a series of empirical readings that reversed when an estimand or a baseline was tightened, while κ_i never moved.

The consequence question deserves stating precisely, because the honest answer is neither "none" nor "established". In operational units the diagnostic does say something: all 40 of 40 dictionaries reach guaranteed affine failure 4 to 40 sparsity levels earlier than an i.i.d. code of their own shape and operating sparsity. At a sparsity where a random code is still safe, a released dictionary already has a feature no affine rule can recover. That is a statement a practitioner can act on, and it is a worst-case guarantee rather than an average.

What it is not is an external endpoint. Both the ratio and the failure threshold are functionals of the same leverage distribution, so their agreement is a consistency check and not independent corroboration; and neither is measured against reconstruction quality, attribution accuracy, steering behaviour, or any other quantity an interpretability pipeline actually optimises. Whether a 40-level earlier failure boundary changes what such a pipeline recovers in practice is the question we would most want a reader to take next, and the one this paper does not answer.

For capacity theory, the interface distinction explains why the Hänni and Adler–Shavit results coexist rather than compete (Appendix D). It also cautions against reading sparse-autoencoder dictionary sizes as computation capacities: those are reconstructive quantities, and both halves of this paper show that reconstruction and linear decodability are governed by different criteria — most sharply in the dictionaries, which optimise the first and come out behind an unstructured code of their own shape on the second.

Acknowledgments

Withheld for anonymous review.

References

Roza Aceska and McKenna Kaczanowski. Cross frame potential. *Numerical Functional Analysis and Optimization*, 43(14):1663–1678, 2022. Preprint arXiv:2205.05613.

Micah Adler and Nir Shavit. On the complexity of neural computation in superposition, 2024. Version v3, 26 February 2026.

Micah Adler, Dan Alistarh, and Nir Shavit. Towards combinatorial interpretability of neural computation, 2025.

Jai Bhagat, Sara Molas-Medina, Giorgi Giglemiani, and Stefan Heimersheim. Compressed computation is (probably) not computation in superposition, 2026.

Hector Borobia, Elies Seguí-Mas, and Guillermina Tormo-Carbó. Feature survival under weight pruning: Sparse autoencoder diagnostics for compressed language models. *Neurocomputing*, pp. 134826, 2026. doi: 10.1016/j.neucom.2026.134826.

Dan Braun, Lucius Bushnaq, Stefan Heimersheim, Jake Mendel, and Lee Sharkey. Interpretability in parameter space: Minimizing mechanistic description length with attribution-based parameter decomposition, 2025.

Jack Capon. High-resolution frequency-wavenumber spectrum analysis. *Proceedings of the IEEE*, 57(8): 1408–1418, 1969.

Ole Christensen, Somantika Datta, and Rae Young Kim. Equiangular frames and generalizations of the Welch bound to dual pairs of frames. *Linear and Multilinear Algebra*, 68(12):2495–2505, 2020.

Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models, 2023.

Alexandre d'Aspremont and Laurent El Ghaoui. Testing the nullspace property using semidefinite programming. *Mathematical Programming*, 127(1):123–144, 2011. doi: 10.1007/s10107-010-0416-0.

Somantika Datta, Stephen D. Howard, and Douglas Cochran. Geometry of the Welch bounds. *Linear Algebra and its Applications*, 437(10):2455–2470, 2012. Preprint arXiv:0909.0206.

David L. Donoho and Jared Tanner. Neighborliness of randomly projected simplices in high dimensions. *Proceedings of the National Academy of Sciences*, 102(27):9452–9457, 2005. doi: 10.1073/pnas.0502258102.

Nelson Elhage, Tristan Hume, Catherine Olsson, et al. Toy models of superposition, 2022.

Francisco Ferreira da Silva and Stefan Heimersheim. Evidence for feature-specific error correction in LLMs, 2026a.

Francisco Ferreira da Silva and Stefan Heimersheim. Compressed computation under l^4 loss is likely computation in superposition, 2026b.

Leo Gao, Tom Dupré la Tour, Henk Tillman, et al. Scaling and evaluating sparse autoencoders, 2024.

Kaarel Hänni, Jake Mendel, Dmitry Vaintrob, and Lawrence Chan. Mathematical models of computation in superposition, 2024. Version v1, 10 August 2024.

William B. Johnson and Joram Lindenstrauss. Extensions of Lipschitz mappings into a Hilbert space. In *Contemporary Mathematics*, volume 26, pp. 189–206. American Mathematical Society, 1984.

William H. Kautz and Richard C. Singleton. Nonrandom binary superimposed codes. *IEEE Transactions on Information Theory*, 10(4):363–377, 1964. doi: 10.1109/TIT.1964.1053689.

Andreas Klappenecker and Martin Rötteler. Mutually unbiased bases are complex projective 2-designs. In *Proceedings of the IEEE International Symposium on Information Theory (ISIT)*, pp. 1740–1744, 2005.

Linghao Kong, Inimai Subramanian, Yonadav Shavit, Micah Adler, Dan Alistarh, and Nir Shavit. Expand neurons, not parameters, 2025.

Tom Lieberum, Senthooran Rajamanoharan, Arthur Conmy, et al. Gemma scope: Open sparse autoencoders everywhere all at once on Gemma 2, 2024.

Yizhou Liu, Ziming Liu, and Jeff Gore. Superposition yields robust neural scaling, 2025.

Eric J. Michaud, Liv Gorton, and Thomas McGrath. Understanding sparse autoencoder scaling in the presence of feature manifolds, 2025.

Lee Sharkey, Bilal Chughtai, Joshua Batson, et al. Open problems in mechanistic interpretability, 2025.

Thomas Strohmer and Robert W. Heath. Grassmannian frames with applications to coding and communication. *Applied and Computational Harmonic Analysis*, 14(3):257–275, 2003.

Roman Vershynin. *High-Dimensional Probability: An Introduction with Applications in Data Science*. Cambridge University Press, 2018.

Shayne Waldron. Generalized Welch bound equality sequences are tight frames. *IEEE Transactions on Information Theory*, 49(9):2307–2309, 2003.

Lloyd R. Welch. Lower bounds on the maximum cross correlation of signals. *IEEE Transactions on Information Theory*, 20(3):397–399, 1974.

A The Interface Diagnostic

A.1 Problem statement

Problem 21 (Interface diagnosis)**. Input:** a code $\Phi \in \mathbb{R}^{d\times F}$ with $F > d$ (or a trained network from which an effective code is read off); a readout family $\mathcal{G}$; a sparse-state model, here uniformly random supports of size s drawn from a grid $\mathcal{S}$; a threshold θ; a trial budget T; a confidence level $1-\alpha$.

Output: (i) the attainment ratio $r = \widehat{W}/W(F,d) \geq 1$ and the maximum-form ratio $r_{\max}$, together with its factorisation $R_{\text{geom}} \times R_{\text{readout}}$ against the code's own floor (Section 3.1); (ii) for each $s \in \mathcal{S}$, the empirical $p_{\text{rec}}(s)$ with a Wilson interval and the exact per-coordinate RMS error $\rho_{\text{lin}}(s)$ of the calibrated linear interface; (iii) the criterion-separation witness $(s_{95}, \rho_{\text{lin}}(s_{95}))$; (iv) for each feature in a chosen subset, the collision frontier κ_i and hence the largest sparsity at which any affine probe can detect that feature (Algorithm 1).

The output is a diagnosis, not a score to be optimised, and the four parts answer different questions. A code with r close to 1 is Welch-optimal in the mean-square sense, but only R_{readout} says anything about the decoder; a large s_{95} with $\rho_{\text{lin}}(s_{95})$ well above $d^{-1/2}$ says the code supports support recovery in a regime where analog reconstruction from the same map is useless; and κ_i is the only part that is per-feature and exact rather than aggregate and empirical, which is what makes it usable as a statement about an individual feature rather than about the code on average.

A.2 Algorithm 1: floor attainment

Step 3 is not a numerical guard but a semantic one: without it, an interface can be made to look arbitrarily good by shrinking G, which is exactly the confusion that Remark 3 dispels. By Theorem 1, a correct implementation always returns $r \geq 1$; we use that as a runtime assertion, and no violation was observed in 720 gain-normalised interfaces (Section F.1).

The mean-square statistic that the theorem actually constrains never requires the interface to be built. The maximum statistic does, and we say so rather than folding it into the same cost.

Algorithm 2 FLOORATTAINMENT — how close the readout interface sits to the floor

Require: code $\Phi \in \mathbb{R}^{d\times F}$, $F > d$; readout $G \in \mathbb{R}^{F\times d}$ or a rule in $\{\text{TIED}, \text{PINV}, \text{LSQ}\}$
Ensure: attainment ratio r (and $r_{\max}$ if requested), or DEGENERATE

$G \leftarrow \Phi^\top$, Φ^+, or the least-squares fit, per the rule
$D_{ii} \leftarrow \langle g_i, \phi_i\rangle$ for $i = 1, \dots, F$ ▷ the diagonal of $M = G\Phi$, in $O(Fd)$; M itself is not formed
if $\min_i |D_{ii}| < \epsilon_{\text{gain}}$ **then return** DEGENERATE ▷ some feature is read out with vanishing gain
end if
$\bar{G} \leftarrow D^{-1}G$ ▷ gain normalisation; see Corollary 2
$H \leftarrow \bar{G}^\top \bar{G}$; $\quad \Sigma \leftarrow \Phi\Phi^\top$ ▷ both $d \times d$; $O(Fd^2)$
$\widehat{W} \leftarrow \dfrac{\operatorname{tr}(H\Sigma) - F}{F(F-1)}$ ▷ exact, by Lemma 22
$W \leftarrow (F-d)/(d(F-1))$ ▷ Theorem 1
if the maximum form is requested **then**
    form $\widetilde{M} \leftarrow \bar{G}\Phi$ and set $\widehat{\mu} \leftarrow \max_{i\neq j} |\widetilde{M}_{ij}|$ ▷ $O(F^2 d)$ time, $O(F^2)$ memory
end if
return $r \leftarrow \widehat{W}/W$ (and $r_{\max} \leftarrow \widehat{\mu}/\sqrt{W}$ if computed)

Lemma 22 (Mean-square cross-talk from $d \times d$ statistics)**.** *Let $\bar{G} = D^{-1}G$ with $D_{ii} = \langle g_i, \phi_i\rangle \neq 0$, let $\widetilde{M} = \bar{G}\Phi$, and set $H = \bar{G}^\top\bar{G}$ and $\Sigma = \Phi\Phi^\top$, both in $\mathbb{R}^{d\times d}$. Then*

$$\sum_{i\neq j} \widetilde{M}_{ij}^2 \;=\; \left\|\bar{G}\Phi\right\|_F^2 - F \;=\; \operatorname{tr}(H\Sigma) - F.$$

Proof. $\widetilde{M}$ has unit diagonal by construction, so the F diagonal entries contribute exactly F to $\left\|\widetilde{M}\right\|_F^2$ and the remainder is the off-diagonal sum. Cyclicity of the trace gives $\left\|\bar{G}\Phi\right\|_F^2 = \operatorname{tr}(\Phi^\top\bar{G}^\top\bar{G}\Phi) = \operatorname{tr}(\bar{G}^\top\bar{G}\,\Phi\Phi^\top) = \operatorname{tr}(H\Sigma)$. □

Forming H and Σ costs $O(Fd^2)$ time and $O(d^2)$ memory, against $O(F^2d)$ and $O(F^2)$ for the direct route — a reduction of a factor F/d in time and F^2/d^2 in memory, with no approximation whatever. The identity is verified against the dense computation to relative accuracy 10^{-10} in the accompanying tests.

A.3 Algorithm 2: separation profile of a given code

Algorithm 3 is the diagnostic proper: it takes a code and a readout that already exist — a designed code, an optimised one, or the effective code of a trained network — and holds them fixed while the support varies. This is the only version that says anything about a particular object, and it is the one applied to trained networks in Section F.3.

The support-recovery side of Algorithm 3 is Monte Carlo over supports; the analog side is *exact* for the given code, computed once before the loop rather than estimated inside it. Total cost is $O(Fd^2 + TFd\,s_{\max})$ time and $O(Fd + d^2)$ memory: the $F \times F$ interface is never formed.

A.4 A scaling experiment on the random-code ensemble

Algorithm 4 is a different object and we keep it named apart. It redraws a fresh code at every trial, so what it characterises is the (d, F) random-code *ensemble*, not any individual code; it is the vehicle for the width-scaling study of Sections F.1 and F.4, and it is not a diagnosis of anything. Because the code is regenerated block by block from a counted seed and never stored, it reaches widths at which the code would not fit in memory.

Both algorithms share the same asymmetry: the support-recovery side is estimated, the analog side is computed. That is what makes the comparison a witness rather than two noisy curves — the analog error carries no Monte Carlo uncertainty, so any gap at s_{95} is attributable to the recovery side alone.

Algorithm 3 SeparationProfile — support recovery versus irreducible analog error, on a fixed code

Require: code $\Phi \in \mathbb{R}^{d\times F}$ and readout $G \in \mathbb{R}^{F\times d}$; sparsity grid $\mathcal{S}$; trials T; threshold θ; seed σ
Ensure: $\{(s, p_{\mathrm{rec}}(s), \mathrm{CI}, \rho_{\mathrm{lin}}(s))\}_{s\in\mathcal{S}}$ and the separation witness $(s_{95}, \rho_{\mathrm{lin}}(s_{95}))$

$s_{\max} \leftarrow \max \mathcal{S}$; $\bar{G} \leftarrow D^{-1}G$ as in Algorithm 2
$\|A\|_F^2 \leftarrow \mathrm{tr}(H\Sigma) - F$; $\|A\mathbf{1}\|^2 \leftarrow \left\|\bar{G}\Phi\mathbf{1}\right\|^2 - 2\,\mathbf{1}^\top\bar{G}\Phi\mathbf{1} + F$ ▷ Lemma 23; $O(Fd^2)$
$e_s \leftarrow \frac{1}{F}\left[(p-q)\,\|A\|_F^2 + q\,\|A\mathbf{1}\|^2\right]$, $p = \frac{s}{F}$, $q = \frac{s(s-1)}{F(F-1)}$, for each $s \in \mathcal{S}$ ▷ exact; Prop. 28
for $t = 1$ **to** T **do**
  draw nested supports $S_1 \subset \cdots \subset S_{s_{\max}}$ from seed (σ, t)
  $Z \leftarrow \bar{G}\,[\,x_1, \ldots, x_{s_{\max}}\,]$, $x_s = \sum_{j\in S_s}\phi_j$ ▷ $O(Fd\,s_{\max})$
  k_s += 1 for each s with $\mathbf{1}\{Z_{\cdot s} \geq \theta\} = \mathbf{1}_{S_s}$
end for
$p_{\mathrm{rec}}(s) \leftarrow k_s/T$ with Wilson interval; $\rho_{\mathrm{lin}}(s) \leftarrow \sqrt{e_s}$
$s_{95} \leftarrow \max\{s :\ p_{\mathrm{rec}}(s') \geq 0.95\ \forall s' \leq s\}$
return the profile and $(s_{95}, \rho_{\mathrm{lin}}(s_{95}))$

Algorithm 4 EnsembleScaling — the same two criteria, averaged over random codes

Require: d, F, sparsity grid $\mathcal{S}$, trials T, threshold θ, seed σ, block size B
Ensure: the ensemble-averaged profile and $(s_{95}, \rho_{\mathrm{lin}}(s_{95}))$

$s_{\max} \leftarrow \max \mathcal{S}$; $k_s \leftarrow 0$ for all s; $e_s \leftarrow 0$ for all s
for $t = 1$ **to** T **do**
  draw nested supports $S_1 \subset \cdots \subset S_{s_{\max}}$ from seed (σ, t)
  $X \leftarrow$ partial sums $[\,x_1, \ldots, x_{s_{\max}}\,]$, $x_s = \sum_{j\in S_s}\phi_j$ ▷ $d \times s_{\max}$
  $\Sigma \leftarrow 0_{d\times d}$, $v \leftarrow 0_d$
  for each block β of B columns of Φ **do**
    regenerate Φ_β from seed (σ, t, β) ▷ Φ is never stored
    $Z \leftarrow \Phi_\beta^\top X$; mark s as failed if $\mathbf{1}\{Z_{\cdot s} \geq \theta\}$ disagrees with $\mathbf{1}_{S_s}$ on this block
    Σ += $\Phi_\beta\Phi_\beta^\top$; v += $\Phi_\beta\mathbf{1}$
  end for
  k_s += 1 for each surviving s
  $\|A\|_F^2 \leftarrow \|\Sigma\|_F^2 - F$; $\|A\mathbf{1}\|^2 \leftarrow v^\top\Sigma v - 2\,\|v\|^2 + F$ ▷ Lemma 23
  e_s += $\frac{1}{F}\left[(p-q)\,\|A\|_F^2 + q\,\|A\mathbf{1}\|^2\right]$, $p = \frac{s}{F}$, $q = \frac{s(s-1)}{F(F-1)}$ ▷ Prop. 28
end for
$p_{\mathrm{rec}}(s) \leftarrow k_s/T$ with Wilson interval; $\rho_{\mathrm{lin}}(s) \leftarrow \sqrt{e_s/T}$
$s_{95} \leftarrow \max\{s :\ p_{\mathrm{rec}}(s') \geq 0.95\ \forall s' \leq s\}$
return the profile and $(s_{95}, \rho_{\mathrm{lin}}(s_{95}))$

Lemma 23 (Sufficient statistics of the tied interface)**.** *Let Φ have unit-norm columns, $M = \Phi^\top\Phi$, $A = M - I_F$, $\Sigma = \Phi\Phi^\top \in \mathbb{R}^{d\times d}$ and $v = \Phi\mathbf{1} \in \mathbb{R}^d$. Then*

$$\|A\|_F^2 = \|\Sigma\|_F^2 - F, \qquad \|A\mathbf{1}\|_2^2 = v^\top\Sigma v - 2\,\|v\|_2^2 + F.$$

Proof. By cyclicity of the trace, $\left\|\Phi^\top\Phi\right\|_F^2 = \mathrm{tr}((\Phi^\top\Phi)^2) = \mathrm{tr}((\Phi\Phi^\top)^2) = \|\Sigma\|_F^2$; the diagonal of M is all ones and contributes F, so subtracting F leaves the off-diagonal energy, which is $\|A\|_F^2$. For the second identity, $A\mathbf{1} = \Phi^\top\Phi\mathbf{1} - \mathbf{1} = \Phi^\top v - \mathbf{1}$, hence $\|A\mathbf{1}\|^2 = v^\top\Phi\Phi^\top v - 2\mathbf{1}^\top\Phi^\top v + F = v^\top\Sigma v - 2\,\|v\|^2 + F$, using $\mathbf{1}^\top\Phi^\top v = v^\top v$. □

Lemma 23 is the reason the diagnostic scales: it replaces an $F \times F$ object by a $d \times d$ one, and combined with Proposition 28 it yields the average linear error at every sparsity without a single Monte Carlo draw.

A.5 Hyperparameters, complexity and cost

The diagnostic has five hyperparameters: the readout rule, the threshold θ (default $1/2$, the natural midpoint for unit-norm codes and Boolean states), the sparsity grid $\mathcal{S}$, the trial budget T, and the block size B, which trades memory for the number of regeneration passes. Only T affects the statistical resolution; Section F.1 reports the threshold sensitivity of the conclusions.

Algorithm 2 costs $O(Fd^2)$ time and $O(d^2)$ memory for the ratio r, by Lemma 22. The maximum-form ratio $r_{\max}$ is the one exception in the whole method: no $d \times d$ reduction of a maximum is available to us, so obtaining it exactly costs $O(F^2 d)$ time and $O(F^2)$ memory, and it is computed only when asked for. Since the floor of Theorem 1 is a statement about the mean square, nothing about the floor comparison is lost by omitting it; we report $r_{\max}$ at the moderate widths where it is affordable and drop it beyond. Algorithm 3 costs $O(Fd^2 + T\,F\,d\,s_{\max})$ time and $O(Fd + d^2)$ memory. Algorithm 4 costs $O(T\,F\,d\,s_{\max})$ time — one pass of code regeneration and one $B \times d \times s_{\max}$ product per block — and $O(Bd + d^2)$ memory, independent of F. At $d = 1\,024$ and $F = 1\,048\,576$ the code would occupy 4 GB in single precision; the streaming implementation peaks near $B\,d \cdot 4$ bytes. Measured wall-clock times are in Table 10.

A.6 Applicability, edge cases and failure modes

Conditions of applicability. The diagnostic assumes (i) $F > d$, otherwise the floor is vacuous and Algorithm 2 refuses to run; (ii) unit-norm code columns, or a declared normalisation, since the threshold $\theta = 1/2$ is calibrated to unit gain; (iii) a sparse-state model that is exchangeable across coordinates, which is what makes the closed form of step 3 valid.

Edge cases. At $s = 1$ the linear error is still non-zero while threshold recovery succeeds whenever $\mu < \theta$; this is the smallest instance of the separation. When $F/d \to 1$ the floor tends to zero and the diagnosis becomes uninformative — correctly so, since a nearly complete code has no interference to speak of.

Failure modes. Three are worth naming. (a) *Degenerate gain*: if some $|M_{ii}|$ is numerically zero the calibration is undefined and Algorithm 2 returns DEGENERATE rather than a number; this is not a corner case but the expected outcome when a readout is fitted without the diagonal constraint (Section F.2). (b) *Uncovered feature*: a least-squares readout fitted on states in which some feature never appears has an identically zero row, which triggers (a); the implementation therefore enforces coverage of the fitting set. (c) *Vacuous profile*: if the coherence of the code exceeds θ, $p_{\mathrm{rec}}(1)$ is already below 0.95 and $s_{95} = 0$; the certificate is then empty, and the diagnostic reports that rather than extrapolating.

A.7 Implementation and traceability

Table 5 maps every quantity in Problem 21 to the function that computes it. The correctness of the implementation is checked by a test suite that verifies the theorems numerically — equality for tight frames, no violations for random interfaces, the closed forms against Monte Carlo, and the streaming trial against an explicit dense evaluation of the same code.

B Threshold recovery and the separation

The floor rules out small-error ε-linear readout with $\varepsilon = o(d^{-1/2})$ when $F \gg d$. It does not rule out thresholded Boolean recovery, because thresholding does not require every inactive score to be small — only that the aggregate interference stay below the margin.

Theorem 24 (Threshold recovery under coherent superposition)**.** *Let Φ have unit-norm columns with coherence μ, let b be s-sparse, let $x = \Phi b + \eta$ with $\left\|\Phi^\top \eta\right\|_\infty \le \nu$, and let $Q(z)_i = \mathbf{1}\{z_i \ge 1/2\}$. If $s\mu + \nu < 1/2$ then $Q(\Phi^\top x) = b$, with margin $\tau = \frac{1}{2} - (s\mu + \nu) > 0$.*

Proof. For $i \in S = \mathrm{supp}(b)$ we have $z_i = 1 + \sum_{j \in S,\, j \ne i} \langle \phi_i, \phi_j \rangle + \langle \phi_i, \eta \rangle \ge 1 - (s-1)\mu - \nu \ge \frac{1}{2} + \tau$. For $i \notin S$, $z_i \le s\mu + \nu = \frac{1}{2} - \tau$. □

Corollary 25 (Random codes at quadratic load)**.** *Let $F = \lfloor d^2 \rfloor$ and let $\phi_1, \dots, \phi_F$ be independent uniform unit vectors in $\mathbb{R}^d$. Then $\mu \le 6\sqrt{\log d/d}$ with probability at least $1 - d^{-5}$ for all sufficiently large d, and on that event the threshold decoder recovers every s-sparse Boolean state whenever $\left\|\Phi^\top \eta\right\|_\infty < \frac{1}{2} - 6s\sqrt{\log d/d}$.*

Proof. For independent uniform unit vectors u, v, the inner product $\langle u, v\rangle$ has the law of the first coordinate of a uniform point on S^{d-1}, whose tail satisfies $\Pr\{|\langle u, v\rangle| > t\} \le 2\exp(-(d-1)t^2/2)$ for $0 < t < 1$ (see e.g. (Vershynin, 2018, Thm. 3.4.6)). Taking a union bound over the $\binom{F}{2}$ unordered pairs and using $2\binom{F}{2} \le F^2 \le d^4$,

$$\Pr\{\mu > t\} \;\le\; d^4 \exp\!\Big(-\tfrac{(d-1)t^2}{2}\Big).$$

With $t = 6\sqrt{\log d/d}$ we have $\frac{(d-1)t^2}{2} = 18(1-1/d)\log d \ge 9\log d$ for $d \ge 2$, so $\Pr\{\mu > t\} \le d^4 e^{-9\log d} = d^{-5}$. The recovery claim is then Theorem 24 with $\mu \le t$. □

The worst-case coherence bound is pessimistic for a *random* support, which is the case the diagnostic measures. The following average-case statement is the one used in Algorithms 3 and 4.

Theorem 26 (Random-support threshold recovery)**.** *Let Φ have independent uniform unit-vector columns and let S be a uniformly random support of size s, independent of Φ. For $\delta \in (0,1)$ there is a universal $C > 0$ such that, with probability at least $1 - \delta$,*

$$\max_{i \in [F]} \Big| \sum_{j \in S,\, j \ne i} \langle \phi_i, \phi_j \rangle \Big| \;\le\; C\sqrt{\frac{s\log(F/\delta)}{d}},$$

and on that event thresholding $\Phi^\top(\Phi \mathbf{1}_S + \eta)$ at $1/2$ recovers $\mathbf{1}_S$ whenever $\left\|\Phi^\top \eta\right\|_\infty + C\sqrt{s\log(F/\delta)/d} < 1/2$.

Proof. Conditionally on S and on ϕ_i, the summands are independent and each is distributed as the first coordinate of a uniform point on S^{d-1}, hence sub-Gaussian with variance proxy $O(1/d)$ (Vershynin, 2018, Thm. 3.4.6). The sum of at most s of them is sub-Gaussian with proxy $O(s/d)$, so $\Pr\{|I_i| > t \mid S\} \le 2\exp(-c_0 d t^2/s)$. A union bound over $i \in [F]$ and the stated choice of t give the claim; the recovery statement follows as in Theorem 24. Full details are in Appendix M. □

Remark 27 (Which randomness the probability is over)**.** The guarantee of Theorem 26 is *annealed*: the probability is taken over the joint draw of the code Φ and the support S. It is therefore the right statement for Algorithm 4, which redraws the code at every trial, and it is not directly the statement for Algorithm 3, which holds one code fixed. A quenched version — Φ fixed, probability over S alone — follows from concentration for sampling without replacement, but it is not free: it needs hypotheses on the particular Φ (bounded coherence and bounded row sums of $|\Phi^\top\Phi| - I$), which a given trained code may or may not satisfy. We do not assume them. What the fixed-code diagnostic reports is therefore an empirical recovery rate with a Wilson interval, and the theorem is quoted as the ensemble-level expectation it is, not as a guarantee about the code in hand. The analog side has no such caveat: Proposition 28 is exact for every fixed Φ.

We now give the average linear-readout error under the *same* distribution, which is what allows the two criteria to be compared without a distributional mismatch.

Proposition 28 (Average linear energy floor, uniform random supports)**.** *Let $F > d$, let $M = G\Phi \in \mathbb{R}^{F\times F}$ satisfy $\operatorname{rank}(M) \le d$ and $M_{ii} = 1$ for all i, and set $A = M - I_F$. Let $S \subseteq [F]$ be a uniformly random support of size s with $1 \le s \le F/2$ and let $b = \mathbf{1}_S$. Then*

$$\mathbb{E}_S \left\|Ab\right\|_2^2 \;\ge\; \frac{s(F-s)}{F(F-1)} \left\|A\right\|_F^2 \;\ge\; \frac{s(F-s)(F-d)}{d(F-1)}. \tag{8}$$

In particular $\frac{1}{F}\mathbb{E}_S \left\|Ab\right\|_2^2 \ge \frac{s(F-d)}{2d(F-1)}$, which is $\Omega(s/d)$ whenever $F/d \to \infty$.

Proof. For a uniformly random size-s support, $\mathbb{E}[b_i] = s/F$ and $\mathbb{E}[b_i b_j] = \frac{s(s-1)}{F(F-1)}$ for $i \neq j$. Hence $\mathbb{E}[bb^\top] = (p-q)I_F + q\,\mathbf{1}\mathbf{1}^\top$ with $p = s/F$ and $q = \frac{s(s-1)}{F(F-1)}$, and

$$p - q = \frac{s(F-1) - s(s-1)}{F(F-1)} = \frac{s(F-s)}{F(F-1)}.$$

Therefore $\mathbb{E}_S \|Ab\|_2^2 = \operatorname{tr}(A^\top A\, \mathbb{E}[bb^\top]) = (p-q)\|A\|_F^2 + q\|A\mathbf{1}\|_2^2 \geq (p-q)\|A\|_F^2$. Since A has zero diagonal and $\operatorname{rank}(M) \leq d$ with unit diagonal, Theorem 1 gives $\|A\|_F^2 \geq F(F-d)/d$, which yields the second inequality. For $s \leq F/2$ we have $F - s \geq F/2$, and dividing by F gives the per-coordinate form. □

Corollary 29 (Separation at quadratic load, one sparse-state model)**.** *Let $F = d^2$ and let Φ have independent uniform unit-vector columns. There is a universal $c > 0$ such that, for $1 \leq s \leq c\,d/\log d$ and a uniformly random support S of size s, thresholding $\Phi^\top \Phi \mathbf{1}_S$ at $1/2$ recovers $\mathbf{1}_S$ exactly with probability at least $1 - d^{-10}$ for all sufficiently large d. For the* same *sparse-state model, every unit-diagonal rank-d linear readout has average per-coordinate squared error at least $\frac{s(F-d)}{2d(F-1)} = \Omega(s/d)$, and hence RMS error $\Omega(\sqrt{s/d})$.*

Proof. Apply Theorem 26 with $\delta = d^{-10}$, so $\log(F/\delta) = 12 \log d$; the interference bound is $C\sqrt{12 s \log d / d}$, which is below $1/2$ as soon as $s \leq c\,d/\log d$ with $c < 1/(48C^2)$. The second statement is Proposition 28 with $F = d^2$. □

Remark 30 (What the separation compares)**.** The two sides compare different *interface criteria* on the same input distribution: expected squared linear readout error versus exact-recovery probability of a threshold decoder. Neither criterion dominates the other in general. The content of Corollary 29 is that at quadratic feature load one of them is attainable exactly and the other is provably not, in the same regime — which is precisely the quantity Algorithm 3 reports. Corollary 29 is stated for uniform supports; the Bernoulli variant, with the same conclusion, is in Appendix M.

C The hierarchy is an implication, and only one way

We now have three criteria on the same code: analog reconstruction (Section 3.1), affine support recovery (Section 4), and whatever a trained nonlinear decoder achieves. Calling them a hierarchy is only justified if one implies another. It does, in one direction, and the implication runs through leverage — the same statistic that Theorem 6 showed governs the analog level.

Lemma 31 (Leverage is redundancy, exactly)**.** *If Φ has full row rank and $h_i < 1$ then $\min\{\|z\|_2^2 : \Phi_{-i} z = \phi_i\} = h_i/(1-h_i)$.*

Proof. The minimum-norm solution gives $\min \|z\|_2^2 = \phi_i^\top (\Phi_{-i}\Phi_{-i}^\top)^{-1}\phi_i$, and $\Phi_{-i}\Phi_{-i}^\top = \Sigma - \phi_i\phi_i^\top$. Sherman–Morrison gives $(\Sigma - \phi_i\phi_i^\top)^{-1} = \Sigma^{-1} + \Sigma^{-1}\phi_i\phi_i^\top\Sigma^{-1}/(1-h_i)$, so the value is $h_i + h_i^2/(1-h_i) = h_i/(1-h_i)$. □

So a feature with low leverage is reproducible by the others with small coefficients, with no detour through coherence. That is the missing step: it converts a statement about the analog geometry into a bound on the affine frontier.

Theorem 32 (Low leverage forces the frontier down)**.** *For every i with $h_i \leq \frac{1}{2}$,*

$$\kappa_i \;\leq\; 1 + \sqrt{\frac{(F-1)\,h_i}{1-h_i}}. \tag{9}$$

Proof. Take the z of Lemma 31, so $\Phi_{-i} z = \phi_i$ and $\|z\|_2 = \sqrt{h_i/(1-h_i)}$. Then $\|z\|_1 \leq \sqrt{F-1}\,\|z\|_2$ and $\max(\sum_j z_j^+, 1 + \sum_j z_j^-) \leq 1 + \|z\|_1$. For z to be *admissible* it must also satisfy the box, and $h_i \leq 1/2$ gives $\|z\|_2 \leq 1$ and hence $\|z\|_\infty \leq 1$. So this z witnesses the right-hand side. □

Remark 33 (The hypothesis is necessary, not cosmetic)**.** Above $h_i = 1/2$ the minimum-ℓ_2 representation need not satisfy the box, and the inequality is then not merely unproved but false. Take unit-norm columns in $\mathbb{R}^2$,

$$\phi_1 = (1, 0), \qquad \phi_2 = \left(\tfrac{1}{4}, \tfrac{\sqrt{15}}{4}\right), \qquad \phi_3 = \left(\tfrac{1}{4}, -\tfrac{\sqrt{15}}{4}\right).$$

Here $h_1 = 8/9$, so the right-hand side of (9) equals 5. But the second coordinate forces $z_2 = z_3$ and the first then forces $z_2 = z_3 = 2$, so the only representation of ϕ_1 by the other two columns violates $\|z\|_\infty \le 1$: the programme (6) is infeasible and $\kappa_1 = +\infty$. An earlier version of this work stated (9) for all $h_i < 1$; that was wrong, and the counterexample is in the test suite.

Corollary 34 (A failure threshold with no decoder in it)**.** *If*

$$h_i \;\le\; \min\left\{\frac{1}{2},\; \frac{(s-1)^2}{(F-1)+(s-1)^2}\right\}$$

then feature i is not *affinely separable at sparsity s.*

Proof. Solve $1 + \sqrt{(F-1)h/(1-h)} \le s$ for h, intersect with the hypothesis of Theorem 32, and apply Theorem 12. □

The cap only binds for $s - 1 > \sqrt{F-1}$; below that the second expression is already at most a half and the statement is the unrestricted one. Dropping it makes the corollary false, and by the same mechanism as Remark 33: with $a = (1, 0)$, $b = (-\frac{25}{44}, \frac{\sqrt{1311}}{44})$ and $\phi_i = (-\frac{17}{40}, \frac{\sqrt{1311}}{40})$, all unit-norm in $\mathbb{R}^2$, the unique representation is $z = (\frac{1}{5}, \frac{11}{10})$, so $\kappa_i = +\infty$ and the feature is separable at every sparsity — yet $h_i = 5/9$ lies below the uncapped $s = 3$ threshold of 2/3.

Corollary 34 is a statement about the code alone: no training, no decoder, no probe. It predicts affine collapse from a single leverage score. Its cost is looseness — the $\sqrt{F-1}$ step is lossy, and Section F.3 reports a case where it predicts failure at $s = 11$ against a measured frontier of 4. It is a sufficient condition for collapse, not an estimate of the frontier.

Proposition 35 (The converse is false)**.** *Let $\Phi = [\, I_d \;\; I_d \,] \in \mathbb{R}^{d \times 2d}$. Then $h_i = 1/2$ for every i, so leverage is perfectly uniform and $W_\star(\Phi) = W(F, d)$: the analog geometry is optimal. Yet $\kappa_i = 1$ for every i, the smallest value the frontier can take, so no feature is separable at any positive sparsity.*

Proof. $\Sigma = 2I_d$, so $h_i = \phi_i^\top \phi_i / 2 = 1/2$ and Theorem 6 gives equality. Each column i has a duplicate j with $\phi_j = \phi_i$; taking $z = e_j$ gives $\Phi_{-i} z = \phi_i$ with $\sum_j z_j^+ = 1$, $\sum_j z_j^- = 0$ and $\|z\|_\infty = 1$, so $\kappa_i \le \max(1, 1) = 1$. The reverse holds for every code and every feature, since $z^- \ge 0$ forces the objective to be at least $1 + \sum_j z_j^- \ge 1$. Hence $\kappa_i = 1$. Theorem 12 requires $\kappa_i > s$, and $1 > 1$ is false, so feature i is separable at *no* positive sparsity: the largest separable sparsity is $\lceil \kappa_i \rceil - 1 = 0$. Directly: the singleton state e_i and the singleton state e_j have identical representations, so no rule of any kind can tell them apart. □

Together, Theorem 32 and Proposition 35 say:

> Bad analog geometry *implies* a bad affine frontier. Good analog geometry does *not* imply a good one.

That is a strict one-way hierarchy, with the forward arrow proved and the converse refuted by an explicit code, and it is what makes the word "hierarchy" literally true here rather than a label attached to three separate measurements.

C.1 The nonlinear level: a witness, not a frontier

The third level has no frontier to compute. "All nonlinear decoders" has no capacity without a restricted class, and restricting it leads to sign-rank, which we do not open. What it does have is a finite certificate, and it comes free from Proposition 18: the states appearing in (7) are explicit vectors that *no* affine rule can

label correctly, and Radon's theorem says the obstruction compresses to at most $d+2$ of them. A trained network either resolves them or does not, with margin

$$\gamma_{\text{net}} = \min\Big(\min_k N_i(x_k^+) - \theta,\ \min_l \theta - N_i(x_l^-)\Big),$$

which is a sharper report than a recovery curve: not "a solver says the hulls intersect somewhere among the $\binom{F}{s}$ admissible states" but a named, listed set that any third party can rebuild from Φ and check.

The extraction is implemented (`lrtr.affine_frontier.radon_witness`) and validated by asking a linear programme for an affine separator of exactly the extracted states and requiring it to be infeasible; on small codes it is also checked against brute-force enumeration of every Boolean state. Two honest qualifications. The compression to $d+2$ is *not* implemented, so the witness we return is valid but not minimal — on the codes we have examined it comes out at about $d+2$ states already, which is small against $\binom{F}{s}$ but is not a handful, and the per-state margin γ_{net} is therefore a minimum over that many states. And no result in this paper is yet derived from it; the chain $G_1 \to G_2 \to G_3$ is complete as a construction, and reporting witnesses for trained models is future work rather than a claim made here.

D Application: two frameworks, two invariants

We now apply the interface distinction to the two frameworks whose apparent tension motivated this work. All statements imported from the source papers were checked against arXiv:2408.05451v1 and arXiv:2409.15318v3; the audit is recorded in Appendix L.

Lemma 36 (Computation-layer interface; restated from (Hänni et al., 2024, Thm. 11))**.** *Fix $s = O(1)$ and suppose the targeted superpositional AND primitive of Hänni et al. is applied in a regime satisfying its stated graph-balancing, sparsity, norm and incoming-interference hypotheses. Then the computation layer produces outgoing readout error $\varepsilon_{\text{comp}}(d) = \widetilde{O}(\sqrt{s^2/d}) = \widetilde{O}(d^{-1/2})$.*

Lemma 37 (Correction-layer tolerance; restated from (Hänni et al., 2024, Thm. 21))**.** *The correction layer admits incoming error $\varepsilon_{\text{in}} < K(d)\, d^{1/4}/(F^{1/2}s^{1/4})$ with $K(d) = \text{polylog}(d)$, and produces $\varepsilon_{\text{out}} = O((\log d)\sqrt{s/d})$ and $\mu_{\text{out}} = \widetilde{O}(\sqrt{s/d})$.*

Proposition 38 (Template compatibility threshold; derived from Lemmas 36–37)**.** *Fix $s = O(1)$. The published Hänni computation–correction template is certified in the regime $F \lesssim \widetilde{O}(d^{3/2})$, and its recursive construction emulates circuits of width m using width $d = \widetilde{O}(m^{2/3}s^2)$, giving a matching certified scale $\widetilde{\Theta}(d^{3/2})$. This is a template-specific certificate, not an impossibility result for other implementations.*

Proof. The correction theorem applies after the computation layer when $\varepsilon_{\text{comp}}(d) \lesssim \varepsilon_{\text{corr,max}}(F, d, s)$, i.e. $\widetilde{O}(d^{-1/2}) \lesssim K(d)\, d^{1/4}/F^{1/2}$ for $s = O(1)$. Suppressing polylogarithmic factors, $F^{1/2} \lesssim d^{3/4}$, hence $F \lesssim \widetilde{O}(d^{3/2})$. Conversely, inverting $d = \widetilde{O}(m^{2/3}s^2)$ at $s = O(1)$ gives $m = \widetilde{\Omega}(d^{3/2})$. □

In the Adler–Shavit framework, exact Boolean state is restored by thresholding after every stage. Their parameter-description lower bound $\Omega(m' \log m')$, converted to neurons under their architectural assumption of $\Theta(n^2)$ parameters with $O(1)$ average description length each, gives $F_{\text{rec}}^{\text{AS}}(n) \le O(n^2/\log n)$; their explicit construction uses $n = O(\sqrt{m'}\log m')$ neurons, giving $F_{\text{rec}}^{\text{AS}}(n) \ge \Omega(n^2/\log^2 n)$. The remaining logarithmic gap is open.

The two regimes do not contradict each other, and the reason is now precise. The Hänni template maintains an approximate ε-linear interface, so it must feed its residual error into a correction theorem whose tolerance is $\widetilde{O}(d^{1/4}/F^{1/2})$; matching that tolerance to the unavoidable $d^{-1/2}$ cross-talk scale of Theorem 1 gives the $d^{3/2}$ threshold. Adler–Shavit instead threshold, leaving the small-error class entirely, so Corollary 10 does not apply to them. In the vocabulary of Section A: the first framework operates where the diagnostic's attainment ratio matters, the second where its criterion-separation witness does.

E Experimental setup

Six campaigns validate the diagnostic, from controlled random codes to a trained network. E1–E3 establish that the implementation measures what the theory describes; E4 tests whether *learned* codes attain the floor; E5 applies the diagnostic to trained models; E6 tests how the recovery threshold scales.

Environment. All experiments run on CPU only, with no CUDA device. Python 3.12.10, NumPy 2.4.4, SciPy 1.17.1, PyTorch 2.13.0+cpu on a 14-core machine; thread counts are pinned and two cores are reserved for the system. Every campaign writes a run record with the exact command, seeds, configuration, environment, wall-clock duration and exit status. Total measured compute is 343.8 minutes (Table 10).

E1 — floor on random codes. $d \in \{16, 32, 64, 128\}$, $F/d \in \{2, 4, 8, 16\}$, 15 independent codes per configuration, three readouts (tied, pseudoinverse and an i.i.d. Gaussian readout as an adversarial control), and a harmonic tight frame at each (d, F) as the equality witness.

E2 — recovery and separation at $F = d^2$. $d \in \{64, 128, 256\}$, 200 trials per configuration, thresholds $\theta \in \{0.4, 0.5, 0.6\}$ (3 values) as a sensitivity ablation.

E3 — average linear energy. $d \in \{32, 64, 128\}$ at $F = 8d$, sparsities $s \in \{1, 2, 4, 8, 16, 32, 64, 128\}$, energies computed in closed form under both the Bernoulli and the uniform-support model, with a 20 000-draw Monte Carlo cross-check of the closed form.

E4 — optimised codes. $d \in \{16, 32, 64\}$, $F/d \in \{2, 4, 8, 16\}$ (12 configurations), Adam at learning rate 10^{-2} for 20 000 steps, 168 runs in total across four variants: *free* (G and Φ independent), *tied* ($G = \Phi^\top$), *smooth-max*, and the *uncalibrated* ablation in which the diagonal constraint is removed.

E5 — trained toy model. The compressed-computation task of Braun et al. (2025): $d = 50$, $F = 100$, inputs with each coordinate active with probability $p \in \{0.01, 0.02\}$ and value $\sim U[-1, 1]$, target $\mathrm{ReLU}(x)$, $\hat{y} = W_{\text{out}}\mathrm{ReLU}(W_{\text{in}}x)$ trained by Adam for 50 000 steps under an L^2 and an L^4 loss, 5 seeds each, plus an i.i.d. random-code control at the same (d, F). Diagnosis uses 400 evaluation trials per sparsity and three linear readouts, including a least-squares readout fitted on the evaluation distribution — deliberately the strongest linear baseline we can give the competing criterion.

E6 — scaling. $d \in \{128, 256, 512, 1024\}$ at $F = d^2$, up to $F = 1\,048\,576$ features. The code is never materialised: each trial regenerates it in blocks from a counted seed sequence and evaluates every sparsity in one pass using nested supports.

Statistics. Recovery probabilities carry Wilson 95% intervals. Group comparisons in E5 use a two-sided exact Mann–Whitney U test with Cliff's δ as effect size. The scaling law in E6 is a one-parameter least-squares fit of $s_{95} = c\,d/\ln d$, the shape being predicted by the theory and only the constant free; with 4 points the fit is reported as a consistency check, not as an estimate with inferential force.

Reproducibility. Every figure and table in this paper is generated from the raw result files by a script, and every measured number in the text is a macro emitted from those same files; a checker regenerates the macros, compares them against the file on disk, and fails the build on any mismatch, on a macro the text uses but does not define, and on a macro defined but never used. We are explicit about the limit of that guarantee: it certifies that the *values* match the saved results, not that the sentences around them draw the right conclusion. Prose claims still need a reader. Commands are in Appendix K.

A note on E6's linear side. To keep the cost of the largest widths manageable, the exact linear-energy statistic is computed on every fifth trial rather than every trial, so its averages rest on fewer samples than the recovery curves (10 trials against 50 at $d = 1\,024$). The recovery probabilities use the full budget.

F Earlier campaigns: E1–E6

The six campaigns below precede the ones in Section 6 and are reported here rather than in the body for two reasons. They establish that the diagnostic measures what the theory says it measures, which is a precondition for the body's results rather than a result itself; and their decoder comparisons use a fixed threshold on one side, an asymmetry the body's campaigns were designed to remove (Appendix G, item 1). Their geometry and frontier numbers are unaffected by that asymmetry, because neither involves a selected threshold.

One provenance wart applies to E1–E6 and we would rather name it than have it found: they were run before the run-record policy that stamps every result with the commit and the working-tree status of the code that produced it, so their records cite the repository's initial commit. Every campaign from E7 onward carries exact provenance. The saved artefacts of E1–E6 are released unchanged.

F.1 E1–E3: the diagnostic measures what the theory describes

Across 16 configurations, 15 trials each and three readouts — 720 calibrated interfaces in total — the floor was violated 0 times. This is a correctness check on the implementation rather than evidence for Theorem 1, and we report it as such.

The attainment ratio of the tied readout falls from 1.999 at the lowest feature load to 1.063 at the highest (Table 6, Fig. 4a): a random code is close to Welch-optimal in the mean-square sense once F/d is large, since $\mathbb{E}\langle\phi_i, \phi_j\rangle^2 = 1/d$ while $W(F, d) \to 1/d$ from below. At $d = 128$, $F = 2\,048$ the tied ratio is 1.0662 and the pseudoinverse ratio is 1.0010, the latter being the closer of the two at high load. In all 16 configurations the harmonic tight frame attains the floor to within 4e-16 of exact equality, confirming Proposition 4 at machine precision. The Gaussian control, by contrast, reaches ratios as large as 3.4e+08: the bound holds for it too, but nothing forces an arbitrary readout to be anywhere near the floor, and the diagnostic's attainment ratio is therefore informative across seven orders of magnitude rather than a formality.

Fig. 4b shows threshold recovery at $F = d^2$. The transition moves right with d, from $s_{95} = 1$ at $d = 64$ to $s_{95} = 2$ at $d = 256$. At $d = 256$ and $s = s_{95}$, thresholding is exact in at least 95% of trials while the calibrated linear interface retains a per-coordinate RMS error of 0.0884, which is 1.41 times the $d^{-1/2}$ scale and above its own theoretical floor of 0.0882. That pair of numbers is the criterion-separation witness of Problem 21, measured.

The threshold is a hyperparameter, and $\theta = 1/2$ is not a privileged choice. At $d = 256$ the recovery threshold moves from $s_{95} = 1$ at $\theta = 0.4$ to $s_{95} = 2$ at $\theta = 0.5$ and $s_{95} = 4$ at $\theta = 0.6$. Raising it helps here because, with unit-norm codes and Boolean states, active scores concentrate near one while inactive scores concentrate near zero, so a higher cut trades false positives for false negatives favourably in this regime; the individual interference terms are of course signed. Two limits on how far this ablation goes: the three thresholds re-decode the *same* trials rather than independent replications, so they show the sensitivity of the decision rule and not of the sampling; and at the smallest width the effect is not merely numerical — at $d = 64$, $\theta = 0.4$ no sparsity reaches the 95% level at all, so no certificate exists there.

E3 confirms the energy scale (Fig. 4c). Over 24 configurations the ratio $E/(s/d)$ lies in $[0.9735, 1.0088]$ — the s/d reference scale is essentially exact for random codes — with 0 violations of either energy floor, and the closed forms agree with a 20 000-draw Monte Carlo estimate to within a relative 0.0013.

Measured against the *proved* uniform-support floor of Proposition 28, however, the ratio lies in $[1.1395, 2.2165]$: the bound is correct but conservative by a factor between one and two in this regime. That is expected and worth stating plainly, since it bounds what the diagnostic can certify. Two steps of the proof give the slack: the non-negative $q\,\|A\mathbf{1}\|^2$ term is discarded, and $\|A\|_F^2$ is replaced by its worst case $F(F-d)/d$ rather than the value realised by the code at hand. The looser the feature load, the larger the second effect — here $F = 8d$, so $(F-d)/F = 7/8$ alone accounts for part of the gap. Algorithm 3 therefore reports the *exact* energy computed from the code's own frame operator, and uses the floor only as the guarantee that the quantity can never be zero.

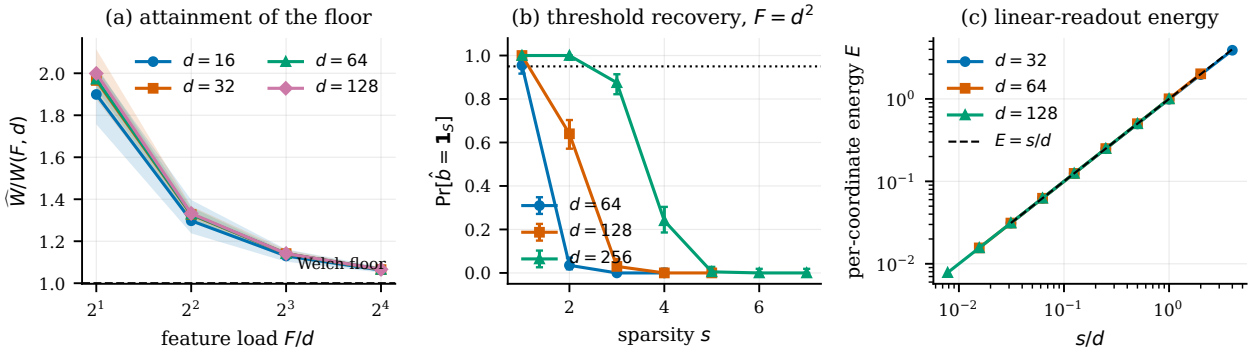

Figure 4: **The Interface Diagnostic on random codes.** (a) Attainment ratio $\widehat{W}/W(F,d)$ of the calibrated tied readout versus feature load; shaded bands are the min–max range over trials, the dashed line is the floor of Theorem 1. (b) Exact threshold recovery at $F = d^2$ with Wilson 95% intervals; the dotted line marks the 0.95 level defining s_{95}. (c) Average per-coordinate linear energy under uniformly random supports against the s/d reference. Generated by `scripts/make_figures.py` from `results/e1`, `results/e2` and `results/e3`.

F.2 E4: learned codes attain the floor

Gradient descent under the unit-diagonal constraint drives the interface to the floor (Table 7, Fig. 5). Over 60 runs of the free variant, spanning all 12 configurations, the attainment ratio reaches a mean of 1.0006 and never exceeds 1.0176, with 0 runs below the floor. The tied variant is sharper still: over 36 runs, mean ratio 1.0000 with a maximum of 1.0000 (0 below the floor), and a relative tight-frame residual of 0.0013 on average (worst case 0.0016). In other words, the optimiser recovers the equality case of Proposition 4: a unit-norm tight frame emerges from gradient descent rather than being constructed.

The smooth-max variant is the one that does *not* reach its target, and the reason is structural rather than a tuning failure. Over 36 runs it attains a mean mean-square ratio of 1.5050 (worst case 3.5230, 0 below the floor) and drives the maximum off-diagonal entry only to 3.125 times $\sqrt{W(F,d)}$. Equality in the maximum form requires an equiangular tight frame, and those exist only for special (d, F) — none of our 12 configurations is guaranteed to admit one. The maximum form of the bound is therefore approached but not attained here, which is the expected outcome and not evidence against the bound.

The uncalibrated ablation is the most informative of the four, and its outcome is more extreme than we expected. Removing the diagonal constraint does not lead the optimiser to a code with low interference: it leads it to the trivial solution. Across all 36 runs the *raw* off-diagonal energy collapses to 6.61e-09 — the readout is driven to zero — and with it the diagonal gains, whose smallest absolute value reaches 0 in single precision. In 25 of the 36 runs the calibration is therefore undefined and Algorithm 2 returns DEGENERATE; in the remaining runs the calibrated interface still respects the floor, and in no run did it fall below (0 violations). An unconstrained optimiser reports spectacular apparent cross-talk while destroying the interface it was supposed to build. This is Remark 3 as an experiment, and it is the reason step 4 of Algorithm 2 exists and reports a status rather than a number.

F.3 E5: the separation inside a trained network

Table 8 and Fig. 6 report the diagnostic applied to the compressed-computation models. Three findings stand out; one of them we did not anticipate, and one prediction we had made was not borne out.

The L^4-trained network attains the floor. Its calibrated linear interface has attainment ratio 1.0006 under the pseudoinverse readout (range $[1.0005, 1.0006]$ over 5 seeds) and 1.0216 under the fitted least-squares readout (range $[1.0206, 1.0223]$), against a floor of $W = 0.01010$. The L^2 baseline sits at 28.8135 and the random-code control at 1.0550. E4 had shown the floor to be reachable by direct optimisation of the cross-talk; here it is approached by a network optimising a task loss that never mentions interference.

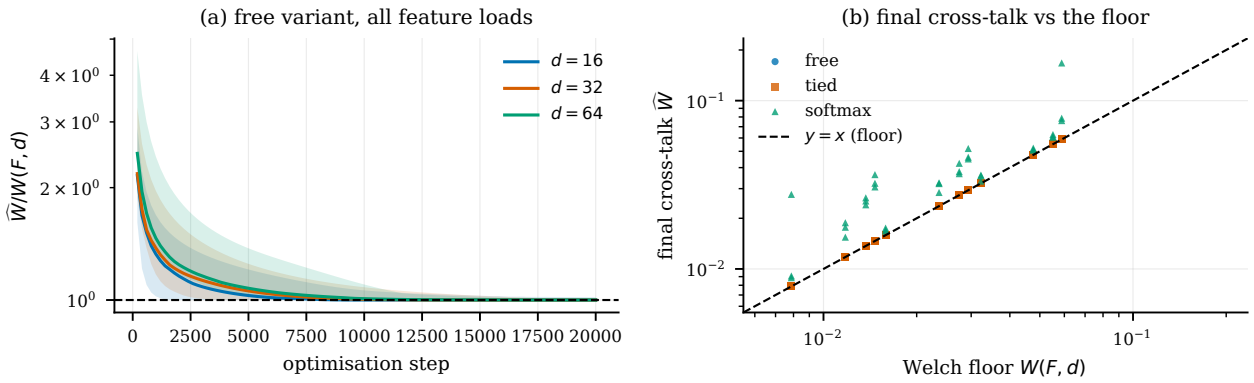

Figure 5: **Optimised codes attain the Welch floor.** (a) Attainment ratio during optimisation of the free variant; bands are min–max over configurations and seeds. (b) Final cross-talk against the floor across every configuration and variant; the dashed line is equality. Generated by `scripts/make_figures.py` from `results/e4`.

Two caveats on the statistics, both cutting against reading too much into them. First, the comparison with the random code is readout-dependent: L^4 is closer to the floor than an i.i.d. code of the same shape under the pseudoinverse (1.0006 versus 1.0170) and under least squares, but its own trained decoder is *farther* from the floor (1.0260) than either. That last figure has no counterpart in the control and we do not pair it with one: the random code has no trained decoder, so its W_{out} column is by construction the pseudoinverse and would be compared against a different object. Across the readouts that do admit a matched comparison, the trained code is comparable to a random one on this axis, not better. Second, every pairwise test returns $p = 0.0079$ with $|\delta| = 1$, which is exactly the smallest two-sided p an exact Mann–Whitney can produce at $n = 5$ per group: within-group variance is negligible, so the test reports "no overlap" and nothing more. The effect sizes are what separate the cases, and they differ by orders of magnitude — 28.8135 versus 1.0216 for L^2 against L^4, a few percent for L^4 against random. No multiplicity correction is applied across the nine tests, and at this n none would survive one.

The separation is real but much narrower than we predicted, and the s_{95} statistic runs against us. Our pre-registered expectation was that the network's thresholded output would remain exact over a strictly wider sparsity range than its best linear readout. The measurement contradicts that at the sparsities where both work. Ranking the three linear readouts by exact-recovery rate at each s and taking the best of them, the network is behind up to $s = 4$: at $s = 3$ both are exact, and at $s = 4$ the network recovers in 0.844 of trials while the calibrated W_{out} interface recovers in 0.985. On the s_{95} statistic the linear readout is strictly ahead – $s_{95} = 4$ against the network's 3 on all 5 seeds at $p = 0.010$, and ahead on 3 of 5 seeds (tied on the remainder) at $p = 0.020$ (Table 8). The preregistered comparison is not supported.

The network only leads further out, where both decoders are already unreliable: at $s = 5$, 0.745 versus 0.294, and at $s = 6$, 0.426 versus 0.136. E5 therefore shows the network extending the usable sparsity range past the point where the thresholded linear readouts of its own code have collapsed, not dominating them everywhere.

The floor statement is on firmer ground, but it is worth being exact about what it is. At every sparsity analog reconstruction carries irreducible error: at $s = 3$, where the network is exact in 1.000 of trials, the best linear readout still has per-coordinate RMS error 0.1733 against its floor of 0.1714. That non-zero error is Theorem 1 restated for this code, not an independent measurement — any rank-d unit-diagonal interface has it by construction. What E5 measures is its *size* for a code that gradient descent actually produced, and how far the thresholded decoder gets before that error stops it.

The L^2 model is maintaining a linear interface. This is the finding we did not plan for. In the L^2 models the ReLU *clips* only 4.6% of preactivations — the macro is the negative fraction, so the unit is very nearly linear — the learned decoder sits at relative distance 0.011 from the pseudoinverse of the encoder,

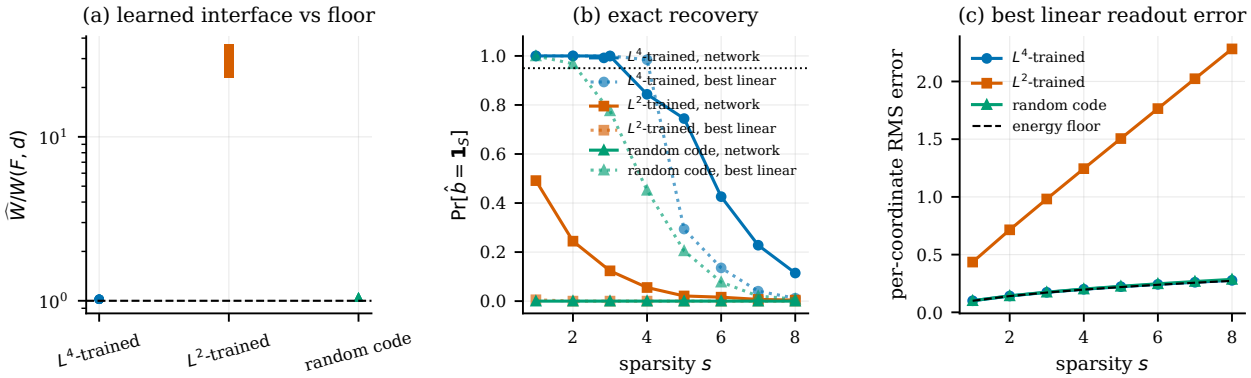

Figure 6: **Interface separation inside a trained toy model of computation in superposition.** A width-50 network trained to compute 100 sparse ReLU features (setup of Braun et al. (2025); L^4 variant of Ferreira da Silva & Heimersheim (2026b)); one point per seed. (a) Attainment ratio of the learned calibrated interface against the floor of Theorem 1. (b) Exact recovery by the network's thresholded output (solid) versus the best linear readout of the same code (dotted). (c) Per-coordinate RMS error of the best linear readout against the energy floor of Proposition 28. Generated by `scripts/make_figures.py` from `results/e5`.

and the network's thresholded decision agrees with that of a plain linear readout on 99.1% of evaluation states, which the two preceding facts are what make possible. The L^4 models are elsewhere: the ReLU clips 48.6% of preactivations, decoder at relative distance 0.726, and agreement with the linear readout down to 21.8%. The diagnostic thus reproduces, from interface measurements alone and without any mechanistic reverse-engineering, the distinction that Bhagat et al. (2026) and Ferreira da Silva & Heimersheim (2026b) reached by other means. It also explains why the L^2 model is bound by Theorem 1 at all: a network that keeps an effectively linear readout *is* a linear readout for these inputs, and the best affine readout of that code sits at 28.8135 times the floor. That number is the fitted least-squares probe's, not the network's own: by the factorisation above the network's own readout is $R_{\text{geom}} \times R_{\text{readout}}$, which is a third smaller. Every ratio quoted in this subsection names its readout, because the two differ by more than the effect under discussion. Its thresholded output recovers even a single active feature only 0.491 of the time.

The finding replicates at a second training sparsity. Everything above is measured at $p = 0.010$. Repeating the whole campaign at $p = 0.020$ (ablation A5, 5 fresh seeds per condition) reproduces both halves: the L^4 models sit at 1.0217 against the floor and agree with a linear readout on only 22.3% of states, while the L^2 models sit at 45.1973 and agree 99.7% of the time. The interface distinction is therefore not an artefact of one input sparsity.

What does not favour our hypothesis. Three things. The L^4 models have a *worse* mean squared error on the training task than the L^2 models (1.19e-03 versus 8.30e-04), as expected since they optimise a different objective: the interface advantage is not a task-performance advantage, and we do not claim it is. The s_{95} prediction above failed. And the random-code control is a reminder that a good interface is not by itself evidence of computation: its *linear* readout recovers exactly with probability 0.698 at $s = 3$, better than the L^2 network, simply because an i.i.d. code is already near-Welch-optimal (ratio 1.0550). What the random control lacks is the nonlinear stage; the "network" column for that row is not meaningful and is reported only for completeness.

What the attainment ratio was measuring

The ratios above are referenced to $W(F, d)$, and Section 3.1 showed that such a ratio conflates two questions. Applying the factorisation (5) to the same models settles which of the two the reported numbers answered, and the answer is uncomfortable: one of them, entirely.

Under the calibrated pseudoinverse the readout factor is 1 to six decimals in every cell — 1.000000 for L^4, 1.000000 for L^2, 1.000000 for the random control — which is not a measurement but Theorem 6 restated, since the calibrated pseudoinverse *is* the code-specific optimum. The whole ratio is therefore R_{geom}: 1.0006 against a reported 1.0006 for L^4, 19.0052 against 19.0052 for L^2, 1.0170 against 1.0170 for the random code. So the claim that the L^4 model comes "within a few parts in a thousand of the floor" is a statement about the evenness of its leverage profile, and contains no information about its decoder. We restate it that way.

Asking the decoder question separately reverses the ordering. Measured against the best readout of its *own* code, the L^4 network's trained output layer sits at $R_{\text{readout}} = 1.0254$ while the L^2 network's sits at 1.0053 — so on this criterion the L^2 model has the better decoder and the worse code, the opposite of the reading the single ratio invites. The random control's own readout gives exactly 1.0000, which is a consistency check rather than a finding: that baseline sets $W_{\text{out}} = \Phi^+$ by construction, so its "own decoder" is the code-specific optimum by definition, and any other value would indicate a bug.

The mechanism is visible in the leverage profiles, and it is the quantity Corollary 34 needs. The L^4 code spreads leverage almost uniformly, from 0.4801 to 0.5192 with coefficient of variation 0.0169; the random code is comparable at 0.3927 to 0.6263 (coefficient of variation 0.0918); the L^2 code is not, running from 0.0092 to 0.9942 with coefficient of variation 0.8673. That dispersion is exactly the excess of Proposition 8, and it grows with the training density: at $p = 0.02$ the L^2 geometry factor rises to 29.1032.

Two limits on what this licenses. The factorisation is arithmetic on Theorem 6 and needs no new assumption, but it reinterprets an existing measurement rather than adding evidence — five seeds at one width remain five seeds at one width. And 0.0092 is the number Corollary 34 consumes: with $s = 2$ the threshold $\min\{\frac{1}{2}, (s-1)^2/((F-1)+(s-1)^2)\}$ is exactly $1/F$, so at $F = 100$ it is 0.01, and the measured minimum leverage of the L^2 code falls below it. The theory therefore predicts that this code loses affine separability at $s = 2$ — from a single leverage score, with no decoder and no probe in the argument. The corollary is applied at the minimum leverage, which is where it is strongest and, being below a half, also where its hypothesis holds. Whether that prediction survives at other widths is the question Section 8 flags as open.

F.4 E6: a scalability stress test, not a scaling law

This campaign was designed to measure how the recovery threshold scales, and it does not have the resolution to do that. We report it for what it does establish — that the measurement itself runs at a width and feature count where the code cannot be materialised — and withdraw the scaling-law reading.

Scaling to $d = 1\,024$ at $F = 1\,048\,576$ features lets the recovery threshold be traced over a factor of eight in width (Table 9, Fig. 7). On the grid, s_{95} grows from 1 at $d = 128$ to 8 at $d = 1\,024$.

That grid statistic needs care, because it floors the crossing and floors it unevenly. The true 95% crossing lies between grid points, and the gap is proportionally largest where the transition is sharp relative to the spacing — at $d = 128$ the interpolated crossing is 1.18 against a reported 1. Interpolating instead of flooring changes the headline: growth over the sweep falls from 8.00 to 6.89, against 5.60 for an exact $d/\ln d$ law. The floored figure therefore overstates the growth by roughly a third, purely as an artefact of the grid, and we quote the interpolated values from here on.

Four widths then cannot identify a law. On the interpolated crossings a one-parameter fit of $s_{95} = c\,d/\ln d$ gives $c = 0.0563$ with $R^2 = 0.9882$ over 4 points, but a plain linear law $c\,d$ reaches $R^2 = 0.9636$ and $c\,d/\ln^2 d$ reaches $R^2 = 0.9630$ on the same four points. The three are indistinguishable on this evidence, the base of the logarithm is not identifiable at all, and we therefore claim no scaling law — neither for $d/\ln d$ nor against it. A previous version of this work drew a $d/(16 \ln d)$ reference curve here; it has been withdrawn, because its constant is inherited from the earlier literature rather than derived anywhere in this work, and because it sat *above* the measured thresholds at every width and so was not the conservative reference its form suggests. Corollary 29 fixes only the shape $s \le c\,d/\log d$, for an unspecified universal c.

A second qualification. s_{95} is a point estimate: at $d = 1\,024$ the campaign runs 50 trials, so even a flawless 50-for-50 result gives a Wilson lower bound of only 0.929. No sparsity at that width can be certified at the 95% level with 95% confidence on this budget; the reported thresholds are best estimates, and a claim at confidence would need more trials.

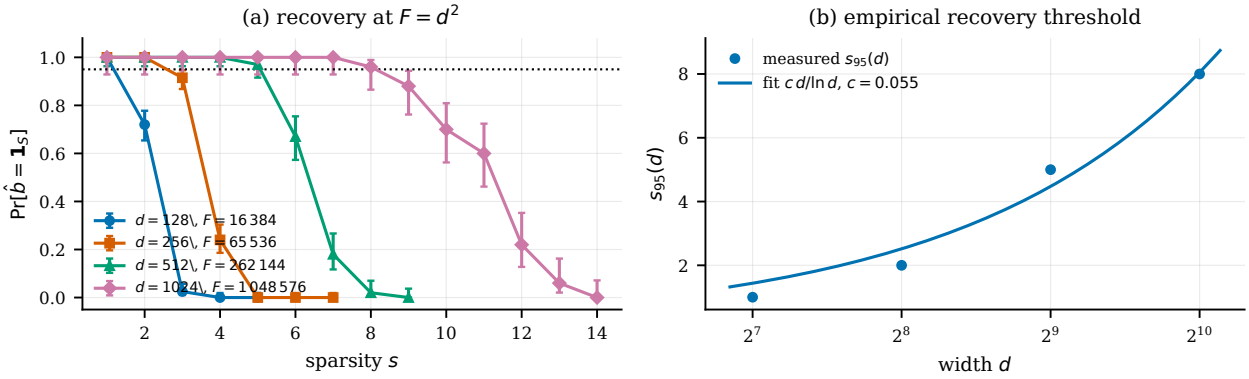

Figure 7: **Threshold recovery at quadratic feature load, up to** $d = 1\,024$ ($F = 1\,048\,576$). (a) Empirical exact-recovery probability with Wilson 95% intervals; a fresh random code is drawn per trial. (b) The empirical threshold $s_{95}(d)$ with the fitted $c\,d/\ln d$ curve shown as a fit only: four widths do not distinguish it from $c\,d$ or from $c\,d/\ln^2 d$, and no scaling law is claimed. Generated by `scripts/make_figures.py` from `results/e6`.

The whole campaign took 29.9 minutes on CPU, with $d = 1\,024$ accounting for 21.3 of them over 50 trials. Materialising the code at that size would have required 4 GB; the streaming implementation of Algorithm 4 keeps the peak footprint proportional to Bd.

F.5 Cost and traceability

Every campaign ran on CPU alone; Table 10 reports the measured wall clock, which totals 343.8 minutes across the six. Table 11 closes the loop between the claims above and the artefacts behind them: for each experimental claim it names the figure or table that shows it, the raw result file the numbers come from, and the script and configuration that produce that file. Together with the automatic check that every measured number in this manuscript matches the saved results (Appendix K), this is what makes the reported quantities auditable rather than merely stated.

Table 2: The Interface Diagnostic on 40 released dictionaries, grouped by the axis each block isolates, with every i.i.d. control matched to the shape and operating sparsity of the block it judges. s is the dictionary's own published operating sparsity. The *trained against randomly initialised* block is a matched intervention rather than an observation: both arms come from the same recipe, data and layers, and differ only in whether the underlying model was trained. Generated from `results/sae/derived/sae_axes.json`; the SHA-256 of every checkpoint analysed is recorded there.

code		d	F	F/d	s	R_{geom}	cv(h)
Scale: d varies, load fixed at 16×							
Qwen3-1.7B layer1		2048	32768	16.0	50	1.1371	0.356
Qwen3-1.7B layer18		2048	32768	16.0	50	1.1218	0.274
Qwen3-1.7B layer27		2048	32768	16.0	50	1.0055	0.102
Qwen3-1.7B layer9		2048	32768	16.0	50	1.1849	0.331
Qwen3-8B L17		4096	65536	16.0	50	1.2018	0.352
Qwen3-8B L30		4096	65536	16.0	50	1.0178	0.126
Qwen3-8B _L4		4096	65536	16.0	50	1.0812	0.262
Qwen3.5-2B layer1		2048	32768	16.0	50	1.0336	0.180
Qwen3.5-2B layer16		2048	32768	16.0	50	1.0798	0.259
Qwen3.5-2B layer23		2048	32768	16.0	50	1.0306	0.187
Qwen3.5-2B layer8		2048	32768	16.0	50	1.1704	0.356
i.i.d. 2048x32768	control	2048	32768	16.0	50	1.0001	0.008
i.i.d. 4096x65536	control	4096	65536	16.0	50	1.0000	0.005
Load: F/d varies, d and k fixed							
Pythia-160m 4k		768	4084	5.3	64	1.0674	0.276
Pythia-160m 32k		768	32768	42.7	64	1.0749	0.404
Pythia-160m 65k		768	65536	85.3	64	1.0734	0.469
i.i.d. 768x4084	control	768	4084	5.3	64	1.0005	0.020
i.i.d. 768x32768	control	768	32768	42.7	64	1.0001	0.008
i.i.d. 768x65536	control	768	65536	85.3	64	1.0000	0.005
Trained model against a randomly initialised one							
SmolLM2 random L0		576	36864	64.0	64	1.0038	0.060
SmolLM2 random L12		576	36864	64.0	64	1.0063	0.077
SmolLM2 random L15		576	36864	64.0	64	1.0063	0.077
SmolLM2 random L18		576	36864	64.0	64	1.0064	0.078
SmolLM2 random L21		576	36864	64.0	64	1.0063	0.077
SmolLM2 random L24		576	36864	64.0	64	1.0065	0.078
SmolLM2 random L27		576	36864	64.0	64	1.0065	0.078
SmolLM2 random L3		576	36864	64.0	64	1.0045	0.066
SmolLM2 random L6		576	36864	64.0	64	1.0057	0.074
SmolLM2 random L9		576	36864	64.0	64	1.0061	0.076
SmolLM2 trained L0		576	36864	64.0	64	1.0512	0.307
SmolLM2 trained L12		576	36864	64.0	64	1.0302	0.248
SmolLM2 trained L15		576	36864	64.0	64	1.1160	0.351
SmolLM2 trained L18		576	36864	64.0	64	1.1351	0.397
SmolLM2 trained L21		576	36864	64.0	64	1.0253	0.198
SmolLM2 trained L24		576	36864	64.0	64	1.0355	0.279
SmolLM2 trained L27		576	36864	64.0	64	1.0319	0.211
SmolLM2 trained L3		576	36864	64.0	64	1.0105	0.177
SmolLM2 trained L6		576	36864	64.0	64	1.0216	0.190
SmolLM2 trained L9		576	36864	64.0	64	1.0207	0.198
i.i.d. 576x36864	control	576	36864	64.0	64	1.0001	0.007
Hybrid convolution/attention MLP code							
LFM2.5-350M L0 (conv)		1024	4608	4.5	64	1.0110	0.101
LFM2.5-350M L1 (conv)		1024	4608	4.5	64	1.0453	0.247
LFM2.5-350M L10 (attn)		1024	4608	4.5	64	1.0510	0.239
LFM2.5-350M L11 (conv)		1024	4608	4.5	64	1.0330	0.204
LFM2.5-350M L12 (attn)		1024	4608	4.5	64	1.0119	0.122
LFM2.5-350M L13 (conv)		1024	4608	4.5	64	1.0247	0.196
LFM2.5-350M L14 (attn)		1024	4608	4.5	64	1.0083	0.090
LFM2.5-350M L15 (conv)		1024	4608	4.5	64	1.0240	0.167
LFM2.5-350M L2 (attn)		1024	4608	4.5	64	1.0179	0.133
LFM2.5-350M L3 (conv)		1024	4608	4.5	64	1.0183	0.158
LFM2.5-350M L4 (conv)		1024	4608	4.5	64	1.0605	0.262

Table 3: The matched comparison: network minus best post-ReLU affine probe in s_{95}, paired within seed, with a bootstrap interval over the models of each cell (20 at $d \leq 200$, 10 at $d = 400$). Both decoders read the same state and each gets one validation-selected global threshold. The sign is a property of the code and not of the protocol: positive for L^4 and growing with width, exactly zero for L^2 — where both decoders are at $s_{95} = 0$ and the zero is a joint failure rather than an agreement — and strongly negative for an untrained code, under the identical procedure. Section 6.3 states what the comparison can and cannot mean. Generated from `results/e7/derived/` and `results/e7_stageC/derived/`.

code	d	mean difference	95% interval	net / tie / probe	AUC difference
L^4	50	+0.00	[+0.00, +0.00]	0 / 20 / 0	+0.0053
L^4	100	+0.35	[+0.05, +0.60]	9 / 9 / 2	+0.0210
L^4	200	+0.35	[+0.10, +0.60]	6 / 14 / 0	+0.0379
L^4	400	+2.00	[+1.50, +2.50]	10 / 0 / 0	+0.0699
L^2	50	+0.00	[+0.00, +0.00]	0 / 20 / 0	+0.0007
L^2	100	+0.00	[+0.00, +0.00]	0 / 20 / 0	+0.0004
L^2	200	+0.00	[+0.00, +0.00]	0 / 20 / 0	+0.0007
L^2	400	+0.00	[+0.00, +0.00]	0 / 10 / 0	+0.0008
untrained	50	-1.00	[-1.00, -1.00]	0 / 0 / 20	-0.2122
untrained	100	-2.00	[-2.00, -2.00]	0 / 0 / 20	-0.2098
untrained	200	-2.30	[-2.50, -2.10]	0 / 0 / 20	-0.1805
untrained	400	-2.60	[-3.10, -2.10]	0 / 0 / 10	-0.1782

Table 4: Stage B: a separate campaign with its own seeds and run record. The F/d column is the one to read first. The $p = 0.01$ rows are at $F = 4d$, twice stage A's load; the $p = 0.02$ rows are at $F = 2d$ and vary the training sparsity instead. "mean difference" is network minus best post-ReLU affine probe in s_{95}, paired within seed, and $\kappa_{\min}$ is the exact minimum over all F features. Any comparison against stage A must hold F/d fixed; doing otherwise produced a claim we withdraw in Appendix G. Generated from `results/e7_stageB/derived/`.

code	d	F/d	p	mean difference	95% interval	net / tie / probe	$\kappa_{\min}$
L^4	100	4	0.01	+0.70	[+0.40, +1.00]	7 / 3 / 0	4.3072
L^4	200	4	0.01	+1.00	[+0.70, +1.30]	9 / 1 / 0	5.6212
L^4	50	2	0.02	+0.00	[+0.00, +0.00]	0 / 10 / 0	4.5074
L^4	200	2	0.02	+0.40	[+0.10, +0.70]	4 / 6 / 0	7.6479
L^2	100	4	0.01	+0.00	[+0.00, +0.00]	0 / 10 / 0	1.0003
L^2	200	4	0.01	+0.00	[+0.00, +0.00]	0 / 10 / 0	1.0076
L^2	50	2	0.02	+0.00	[+0.00, +0.00]	0 / 10 / 0	1.0000
L^2	200	2	0.02	+0.00	[+0.00, +0.00]	0 / 10 / 0	1.0031
untrained	100	4	0.01	-1.80	[-2.00, -1.50]	0 / 0 / 10	3.3999
untrained	200	4	0.01	-2.00	[-2.00, -2.00]	0 / 0 / 10	4.7230

Table 5: Each step of the method and the function that implements it. Paths are relative to the repository root.

quantity	where defined	implementation
$W(F, d)$	Eq. (2)	`lrtr/codes.py:welch_floor`
$D^{-1}M$	Cor. 2	`lrtr/interface.py:unit_diagonal`
$\widehat{W}, \widehat{\mu}, r, r_{\max}$	Alg. 2	`lrtr/interface.py:crosstalk_stats`
Algorithm 2	Sec. A	`lrtr/diagnostic.py:interface_floor_diagnostic`
$\Sigma, v \mapsto \|A\|_F^2, \|A\mathbf{1}\|^2$	Lem. 23	`lrtr/diagnostic.py:tied_energy_moments`
$\frac{1}{F}\mathbb{E}_S \|A\mathbf{1}_S\|^2$	Prop. 28	`lrtr/interface.py:linear_energy_uniform`
energy floor	Eq. (8)	`lrtr/interface.py:energy_floor_uniform`
threshold decoder Q	Thm. 24	`lrtr/threshold.py:threshold_decode`
fixed-code trial	Alg. efalg:two, l. 5–7	`lrtr/threshold.py:recovery_trial_fixed`
streaming trial	Alg. efalg:ensemble, l. 6–10	`lrtr/threshold.py:recovery_trial_streaming`
Algorithm efalg:two	Sec. efsec:method	`lrtr/diagnostic.py:fixed_code_separation_profile`
Algorithm efalg:ensemble	Sec. efsec:alg-ensemble	`lrtr/diagnostic.py:random_code_scaling_experiment`
$\mathrm{tr}(H\Sigma) - F$	Lem. eflem:suffstat	`lrtr/interface.py:crosstalk_mean_sq`
leverage form of r	Prop. efprop:leverage	`lrtr/interface.py:crosstalk_mean_sq_pinv`
s_{95}	Alg. efalg:two, l. 9	`lrtr/threshold.py:s95_from_curve`

Table 6: E1: attainment ratio $\widehat{W}/W(F, d)$ of the calibrated tied readout for random unit-norm codes, averaged over 15 trials. Values above one are required by Theorem 1; the approach to one with growing feature load is the content of the measurement.

	F/d			
d	2	4	8	16
16	1.899	1.298	1.130	1.063
32	1.966	1.327	1.141	1.065
64	1.972	1.329	1.141	1.066
128	1.999	1.332	1.141	1.066

Table 7: E4: attainment of the floor by gradient-optimised codes, 168 runs over 12 configurations. "Below floor" counts runs whose calibrated interface fell under the theoretical bound; it must be zero. The uncalibrated ablation has no meaningful calibrated ratio by construction.

variant	runs	mean $\widehat{W}/W$	max $\widehat{W}/W$	below floor	extra
free (G, Φ independent)	60	1.0006	1.0176	0	–
tied ($G = \Phi^\top$)	36	1.0000	1.0000	0	$\rho_{\max} = 0.0016$
smooth-max	36	1.5050	3.5230	0	max ratio = 3.12
uncalibrated (ablation)	36	n/a	n/a	0	raw ratio $\geq 1.5e-16$, $\min \lvert M_{ii}\rvert = 0.0e+00$

Table 8: E5: the diagnostic applied to trained models, averaged over 5 seeds at training sparsity $p = 0.010$. "Agrees with linear" is the fraction of evaluation states on which the network's thresholded decision coincides with that of a plain linear readout — the quantity that identifies which interface the model maintains.

model	$\widehat{W}/W$	s_{95} (network)	s_{95} (best linear)	ReLU clipped (%)	agrees with linear (%)
L^4-trained	1.022	3.0	4.0	48.6	21.8
L^2-trained	28.814	0.0	0.0	4.6	99.1
random code	1.055	0.0	2.0	50.0	13.8

Table 9: E6: recovery threshold and measured cost by width.

d	$F = d^2$	trials	s_{95}	$d/(16 \ln d)$	wall clock (min)
128	16 384	200	1	1.65	1.5
256	65 536	200	2	2.89	2.2
512	262 144	100	5	5.13	4.9
1024	1 048 576	50	8	9.23	21.3

Table 10: Measured computational cost, read from the run records. E1–E6 are CPU-only and hide any accelerator that is present, so they cannot silently become irreproducible on a CPU-only machine. E7 and E8 train on a GPU; every number this paper draws from them is then recomputed from their saved weights on a CPU, so no claim here depends on access to an accelerator. *jobs* counts models for the GPU campaigns and configured jobs for the others. †Stage A was resumed, and a run record times only the final invocation, so this figure is the last leg rather than the campaign.

campaign	question	jobs	workers	device	wall clock (min)
E1	floor on random codes	–	1	CPU	0.6
E2	recovery at $F = d^2$	–	1	CPU	67.5
E3	average linear energy	–	1	CPU	0.2
E4	optimised codes	168	12	CPU	169.6
E5	trained toy model	25	6	CPU	76.1
E6	scaling to $d = 1024$	–	4	CPU	29.9
E7-A	trained-network grid	180	8	CUDA	306.1†
E7-B	feature load and sparsity	100	10	CUDA	193.5
E7-C	$d = 400$ frontier rate	30	10	CUDA	103.8
E8	frozen code vs frozen readout	60	10	CUDA	77.0
E8	the same at $d = 200$	30	8	CUDA	123.2
total					1147.5

Table 11: Traceability: each experimental claim, the figure or table that shows it, the raw result file it comes from, and the script and configuration that produce it. Reproduction commands are in Appendix K.

claim	figure / table	raw result file	script (config)
The floor is respected by every calibrated interface	Tab. 6, Fig. 4a	`results/e1/raw/e1_rows.jsonl`	`experiments/e1_welch_floor.py` (`configs/e1.json`)
Threshold recovery succeeds where linear readout cannot be exact	Fig. 4b	`results/e2/raw/e2_profile.jsonl`	`experiments/e2_threshold_recovery.py` (`configs/e2.json`)
Average linear energy follows the s/d scale	Fig. 4c	`results/e3/raw/e3_rows.jsonl`	`experiments/e3_linear_energy.py` (`configs/e3.json`)
Learned codes attain the floor	Tab. 7, Fig. 5	`results/e4/raw/e4_runs.jsonl`	`experiments/e4_optimized_codes.py` (`configs/e4.json`)
A trained network attains the floor and separates the two criteria	Tab. 8, Fig. 6	`results/e5/raw/e5_runs.jsonl`	`experiments/e5_trained_toy.py` (`configs/e5.json`)
$s_{95}(d)$ grows with the $d/\ln d$ scale	Tab. 9, Fig. 7	`results/e6/raw/e6_summary.jsonl`	`experiments/e6_threshold_scaling.py` (`configs/e6.json`)

G Readings we withdrew, and what caused each

This appendix exists because the alternative is worse. Over the course of this work eight readings of our own measurements turned out not to survive a check, and in every case the reading we had chosen was the more interesting of the available ones. The last two were found by an external adversarial review of the submitted manuscript, after the other six had been written up here, and a second round of that review — of the fixes for the first two — found a ninth defect, in the audit tooling itself. That sequence is the most useful thing this appendix can tell a reader about how much to trust the rest. Listing them costs us the appearance of a clean result and buys the reader the only thing that makes the surviving claims worth anything: a record of how hard they were pushed. The register in `docs/known_defects.md` carries the code-level detail; what follows is the manuscript-level consequence of each.

1. "The two estimands disagree" (withdrawn; cause: a defective threshold search). The analysis plan named two co-primary estimands, s_{95} and the recovery AUC. An earlier version of Section 6 reported that they disagreed — s_{95} a tie at every width, the AUC favouring the network at $d = 50$ and the probe at $d \in \{100, 200\}$ — and treated the disagreement as the finding, on the grounds that a blunt statistic and a fine one seeing different things is informative.

The cause was in `select_thresholds`. For the `global` and `per_feature` policies it searched a grid of quantiles of the validation scores and returned the best point on that grid, without ever scoring θ_{fixed}, the value it was meant to improve on. On these score distributions the quantile grid could and did miss, so a decoder could be evaluated at a threshold that loses to the default on the very validation objective the search maximises. It cost the network up to five sparsity levels at $d = 400$. The fix scores θ_{fixed} first, adds a uniform grid to the quantile grid, and asserts the post-condition that the returned threshold is at least as good as θ_{fixed} on validation; `tests/test_threshold_selection.py` re-implements the old search and requires it to fail a bimodal case the new one passes. Both sides of the comparison moved when it was refitted, which is the reason we trust the correction: it was not a change that could only help one arm.

With the defect fixed the two estimands agree at all four widths and the disagreement was an artefact. We report the agreement, and this paragraph, rather than quietly substituting one for the other.

2. "The ReLU discards affinely decodable information" (withdrawn; same cause). A probe fitted on the preactivation Φb once beat the network by a full sparsity level, and we read that as a measurement of what the nonlinearity throws away — a claim we liked, because it reframed a decoder comparison as a statement about representations. It does not survive the same fix. The pre-ReLU probe's advantage is now -0.10 at $d = 50$ ([-0.25, 0.00], 18 of 20 seeds tied), -0.25 at $d = 100$ and -0.25 at $d = 200$, and at $d = 400$ it reverses: the network leads the *pre*-ReLU probe by 0.60 on 7 of 10 models. A quarter of a sparsity level, with most seeds tied and the sign flipping at the largest width, does not support a claim about what a nonlinearity discards.

What does survive is confined to the untrained arm, where the effect is an order of magnitude larger and unanimous: the pre-ReLU probe beats the network by -2.45, -4.05, -6.55 and -10.90 levels at the four widths, on every one of the 10 and 20 models per cell. On an untrained code the thresholded post-ReLU readout is far worse than an affine read of the preactivation, and training closes almost all of that gap. We state it in that scoped form and note that it is a comparison across two stages of one pipeline, not a decoder comparison (Section 6.3).

3. "The frozen-code arm is a failed optimisation" (withdrawn; cause: scoring an arm under an objective it was not trained on). In E8 the arm with a frozen random code trains only W_{out} and ends at $R_{\text{readout}} \approx 1.9201$, nearly twice the cross-talk of the best readout of the code it was given. Since Φ^+ attains that optimum for every code, the obvious reading is that gradient descent failed at a convex-looking problem.

We checked it instead of asserting it. Scored under the loss the arm actually trained on, in its own post-ReLU representation, on fresh draws from a seed no training used, the frozen W_{out} *beats* Φ^+ by a factor of 4.50

at $d = 200$ on 10 of 10 models. The cross-talk is bought, not lost, and the correct reading is about what co-adaptation does to two objectives rather than about optimisation difficulty. Section 6 now says that.

4. "The advantage of training grows with width" (withdrawn; cause: choosing the summary after seeing the result). Both the trained and the untrained frontier grow with width, so at three widths it was natural to report that training's advantage grows too — and as a ratio over $d \in \{50, 100, 200\}$ it does. The fourth width breaks it. In absolute sparsity levels, the operationally meaningful unit, the margin is 1.3260, 1.5866, 1.7974, 1.6029: it peaks at $d = 200$ and falls, by more than the sampling noise (a two-sample bootstrap over the independently trained arms gives 0.1940 with interval [0.0903, 0.2818]). As a ratio it falls at every step.

When we first found the fourth width's numbers we reassured ourselves with the ratio, which was the reading that preserved the claim. That is the same move as picking a summary statistic after seeing the result, and we record it as such in `docs/decision_log.md` (D20). The manuscript now leads with the reading that does not survive and states the surviving claim in its narrowed form: training buys frontier at every width we measured, and the amount it buys stops growing by $d = 400$.

5. "The subset frontier is a justified shortcut" (withdrawn; cause: a sufficient condition used as if it were necessary). Evaluating κ_i for every feature costs F linear programmes, so early campaigns evaluated a subset chosen by Theorem 32 and reported the minimum over it as an upper bound. The bound is accurate — it overestimates by 0.0398 on average even at $d = 400$ — but its justification is not: the theorem is sufficient for a feature to be hard, not necessary, so the subset need not contain the true argmin. On untrained codes it always does. On trained codes it drifts, and it drifts with width: the median leverage rank of the true argmin runs 5, 23, 38, 131, and at $d = 400$ the subset locates the argmin in 2 of 10 trained models.

The bias is one-sided and asymmetric between the arms, so a margin computed from subset values is biased in favour of training by an amount growing with the variable under study. Every frontier number in this paper is therefore the exact minimum over all F features. We would not use the shortcut past $d = 200$ again, and we report the accuracy and the invalidity together because the first without the second is how a shortcut survives.

6. Three theorem statements, false as written (corrected; cause: a proof establishing less than its statement claimed). An external adversarial audit found that Theorem 32 and Corollary 34 were stated for all $h_i < 1$ while their shared proof establishes them only for $h_i \leq 1/2$, with explicit unit-norm counterexamples at $d = 2$, $F = 3$; and that a proposition concluded a feature with $\kappa_i = 1$ is separable at $s = 1$ when the frontier statement requires $\kappa_i > s$, so the correct answer is that it is separable at no positive sparsity. The statements are corrected here and the counterexamples are regression tests (`tests/test_arrow_counterexamples.py`).

No measurement was affected: the corollary is only ever applied at $\min_i h_i$, and the largest $\min_i h_i$ over all 180 stage A models is 0.4885, inside the valid region. That is why the tests passed — they sampled only the region where the statement is true, which is a gap in the tests and not a false test. An earlier preprint carries the incorrect statements and will be corrected with an explicit erratum rather than a silent replacement.

7. "Stage B repeats stage A's protocol exactly, and denser training states buy frontier" (withdrawn; cause: comparing two cells without checking they had the same shape). Stage B was described in the submitted manuscript as an independent replication of two stage A cells plus a clean one-variable sparsity contrast. Both halves were false. Its $p = 0.01$ cells are at $F = 4d$, twice stage A's load, so they are not repeats of anything; and the difference in effect size between the two campaigns, which we attributed to the imprecision of a cell mean over ten seeds, is a design difference. Worse, the "cleanest small result in the campaign" — that $\kappa_{\min}$ rises by a third with the training sparsity — compared an $F = 4d$ cell against an $F = 2d$ one. Made at matched load the comparison reverses: 7.6479 [7.5971, 7.7063] at $p = 0.02$ against 8.0352 [7.9971, 8.0787] at $p = 0.01$, per-model ranges disjoint, in the direction opposite to the claim.

The claim is withdrawn and Section 6.2 now reports the null. What survives, correctly described, is a genuine load axis: the loss–geometry separation and the sign structure of the decoder comparison hold at twice the feature load. The structural cause is worth more than the correction: neither `primary_comparison.json` nor `all_feature_frontier.json` recorded F, so the number distinguishing the two designs was absent from the files every table is generated from, and no check could have caught the error. Both now record it, and Table 4 carries it as a column.

8. "The ceiling analysis evaluates the same objective the selection maximised" (corrected; cause: the seventh instance of the trap in items 1 and 5). Section 6.3 scores the network's own map under the campaign's probe-selection objective. The campaign maximises a normalised trapezoid over the validation recovery curve; the script computed the plain mean over grid points, and its own documentation asserted that the mean *was* the selection objective. On a uniform integer grid the two differ by endpoint reweighting of the same order as the smallest gap the analysis reports.

Recomputed under the correct functional the result survives and strengthens — the gaps grow, the count is unchanged — so nothing in Section 6.3 is withdrawn. Two things move. The $d = 50$ cell is no longer indistinguishable from zero, so it is no longer offered as a consistency check. And the L^2 cells, which under the wrong functional showed a systematic positive gap, sit at essentially zero, which is the better outcome: the gap is positive exactly where the network leads, absent where neither decoder recovers anything, and strongly negative where the probe leads. We record it because a corrected measurement that happens to help is exactly the kind a reader should be told about.

What the pattern says. Five of the nine defects behind them are the same trap in five different code paths: an operating point or an objective chosen by maximising something, with no check that it is the thing it claims to be. Two are a reading chosen from among several that the data permitted, selected after the fact for being the stronger claim. One — item 7 — is a comparison made without checking that the two things compared had the same shape, which is the same failure at the level of experimental design rather than of code. We mention the count because a reader deciding how much weight to put on this paper's surviving claims should know both the rate at which we found our own errors and the direction they pointed. Only the first pattern is one that more tests would have caught; the last was caught by recording the variable that distinguished the designs, which is now done.

H Reproducibility statement

Everything below is anonymised for review. An anonymised archive of the repository accompanies this submission; the public repository, DOI and citation metadata are supplied at camera-ready.

What is released. The code that produced every number, the configuration of every campaign, the raw JSON result of every model, the trained weights of all 400 networks in the controlled campaigns, and the sources of this manuscript. Nothing in the paper is computed by a script that is not released, and no measured number in the text is typed by hand: every one is a macro generated from `results/` by `scripts/make_numbers.py`, and `scripts/check_manuscript_numbers.py` regenerates every generated table and every macro definition from `results/` and exits non-zero on any mismatch. It is run as part of the release check and its failure modes are tested: injecting a wrong literal into a generated table, or an undefined macro of a generated family into the source, both make it fail.

Provenance. Every campaign from Section 6 onward writes a run record carrying the exact command line, the seeds, the resolved configuration, the environment, the wall-clock duration, the exit status, and the commit hash together with the working-tree status of the code that produced it — with the list of modified files when the tree was dirty. The earlier campaigns of Appendix F predate that policy and cite the repository's initial commit, which is stated there rather than hidden. Every third-party dictionary analysed in Section 5 is recorded with the SHA-256 of the checkpoint file actually read.

Cost. The full pipeline is reproducible on commodity hardware; the released cost table (Appendix F.5) gives the measured wall-clock and device of every campaign. The dictionary measurements of Section 5 cost seconds per dictionary and need no accelerator, which is why a reader can check the paper's main empirical claim without rerunning any training.

What a reader should be able to check independently. Three things, in increasing cost. The identity of Proposition 5 against the implementation, from the released `results/sae/derived/sae_axes.json` alone. The dictionary measurements, by downloading the listed checkpoints and running one script. And the controlled campaigns, either from the released weights — which is how four of the analyses in Section 6 were computed, months after the runs — or by retraining from the recorded seeds. The probe-ceiling analysis of Section 6.3 is the cheapest of the four and the most useful to re-run adversarially: it recomputes both sides of the paper's own decoder comparison from the released weights and the campaign's own seeds, so reproducing Table 3 to within 0.00 of a sparsity level is a check on that comparison as much as on the ceiling.

Known defects. A register of every defect found during this work, what each invalidated and what was done about it, is released with the code. The manuscript-level consequences are in Appendix G. Two of the entries invalidated readings that appear in an earlier preprint of this work, which will carry an explicit erratum rather than a silent replacement.

I Open problem

Corollary 10 shows that no reset staying inside the small-error ε-linear class can improve the worst-case cross-talk scale beyond $d^{-1/2}$ when $F \gg d$, while Theorem 26 shows thresholded recovery survives to $s = O(d/\log d)$ at quadratic load. Any interpolation beyond the Hänni template must therefore leave the small-error linear class. This makes one question concrete.

Conjecture 39 (Generic robust threshold reset)**.** Fix $s = O(1)$. There are constants $c, C, K > 0$ such that for every d and every $F \le c\, d^2/\log^C d$ there exist a code $\Phi \in \mathbb{R}^{d\times F}$, an input readout R_{in}, a width-$\widetilde{O}(d)$ reset module $\mathcal{R}_\Phi : \mathbb{R}^d \to \mathbb{R}^d$ and a scoring map S such that, for every s-sparse b and every x with $\|R_{\mathrm{in}}x - b\|_\infty \le K\, d^{1/2}/(F^{1/2}s^{1/4})$, the output $z = \mathcal{R}_\Phi(x)$ is composable with the next stage and satisfies $b_i = 1 \Rightarrow (Sz)_i \ge 3/4$ and $b_i = 0 \Rightarrow (Sz)_i \le 1/4$.

The incoming-error tolerance is the essential clause: a reset theorem for clean inputs $x = \Phi b$ alone would not support recursive composition. By Corollary 10 any proof must use an explicitly nonlinear or thresholded decoding mechanism, as Adler–Shavit do.

J Conclusion

We proposed the Interface Diagnostic, a computable procedure that decides which decoding interface an overcomplete code or a trained network maintains, and how far it is from the best possible one. It rests on a rank–trace floor for unit-diagonal linear readouts, in a gain-normalised form that bounds the interference-to-gain ratio, and on a separation between the linear and threshold criteria proved under one and the same sparse-state model. Its mean-square statistics run in $O(Fd^2)$ time and $O(d^2)$ memory and reach $F \approx 10^6$ features on a laptop CPU.

Two additions make the diagnosis about a code rather than about an ensemble. Computing a code's own floor in closed form splits any attainment ratio into a geometry term and a decoder term, and the split is deflationary: under the calibrated pseudoinverse the decoder term is one by construction, so the ratio a reader is invited to read as a decoder result is nothing of the kind. The trained L^4 network's 1.0006 and the L^2 baseline's factor 28.8135 are statements about leverage profiles. And for support recovery, one linear programme per feature returns the largest sparsity at which any affine probe can detect that feature, with a certified margin when it succeeds and a checkable certificate when it fails. Leverage links the two: low leverage forces the affine frontier down, with no decoder in the argument, and an explicit code refutes the converse, so the hierarchy holds in one direction only.

Applied to 180 models, the instrument returns one result that resists summary and two that do not. The first: given the same representation and the same threshold treatment, the comparison between the network and a fitted affine probe has no single sign. It is positive under L^4 and grows with width to 2.00 levels in 10 of 10 models at $d = 400$; it is exactly zero under L^2, where both decoders fail together; and it is negative by up to -2.60 levels on a code that was never trained. The 103 ties out of 120 are therefore not evidence of equivalence — 60 of them are joint failures — and the L^4 lead is not evidence of nonlinear expressive power either, since the probe class contains the network's own decoder (Section 6.3). What the comparison identifies is the code, not the decoder. The first positive: what separates models is the loss, not the architecture — L^2 pushes R_{geom} to 29.8734 and collapses its frontier to 1, while L^4 holds 1.0002 and a frontier of 8.0352. The second: that separation is bought by training the code and not the readout, since a frozen random code with a fully trained readout keeps an untrained code's frontier.

Two things we decline to claim. There is no scaling law here: the threshold-recovery campaign reaches $d = 1\,024$ at about a million features, which establishes that the measurement scales, but four widths do not distinguish $c\,d/\ln d$ from the alternatives. And proximity to the rank–trace floor is not evidence of learned structure, because an untrained code already achieves it.

The broader lesson is that capacity questions for overcomplete representations are not settled by counting features, nor by measuring distance to a bound that random codes already meet. They depend on which decoding interface a particular code maintains, and on which representation is being read — both of which are now measurable, per feature and exactly, for the code in hand.

K Reproduction

From a clean checkout:

```
python -m venv .venv && . .venv/bin/activate     # Windows: .venv\Scripts\activate
python -m pip install -r requirements.txt
python -m pytest -q tests/                       # 370 tests
bash scripts/run_all.sh                          # Windows: scripts\run_all.ps1
```

Individual campaigns, each of which writes `results/<id>/run_record.json` with the exact command, seeds, environment, duration and exit status:

```
python experiments/e1_welch_floor.py
python experiments/e2_threshold_recovery.py
python experiments/e3_linear_energy.py
python experiments/e4_optimized_codes.py
python experiments/e5_trained_toy.py
python experiments/e6_threshold_scaling.py
```

Adding `-smoke` to any of them runs a reduced configuration end to end in seconds. Figures, tables and the generated numbers are rebuilt with `scripts/make_figures.py`, `scripts/make_tables.py` and `scripts/make_numbers.py`; `scripts/check_manuscript_numbers.py` verifies that every measured number in this manuscript matches the saved results and exits non-zero otherwise. E6 writes results per width and per chunk and resumes an interrupted campaign without recomputing completed trials.

L Verification of imported statements

Every statement imported from the two source frameworks was checked against the published version. Table 12 records the audit.

Table 12: Audit of imported statements.

source	as printed there	as used here
(Hänni et al., 2024, Thm. 11)	targeted superpositional AND, precision $\varepsilon_{\text{in}} \to \varepsilon_{\text{out}}$ with $\varepsilon_{\text{out}} = \widetilde{O}(\sqrt{s^2/d})$	Lemma 36, $\varepsilon_{\text{comp}}(d) = \widetilde{O}(d^{-1/2})$ at $s = O(1)$
(Hänni et al., 2024, Thm. 14)	$\varepsilon^{(1)} = \widetilde{O}(\max\{s\mu, \sqrt{s\varepsilon}, \sqrt{s/d}\})$, $\mu^{(1)} = \widetilde{O}(\max\{\sqrt{1/d}, \mu\})$	quoted in Sec. D only to note it does not by itself preserve the $\widetilde{O}(d^{-1/2})$ invariant
(Hänni et al., 2024, Cor. 15)	relaxes the near-sphere hypothesis of Thm. 14 at the cost of depth $1 \to 3$	not used for a capacity claim
(Hänni et al., 2024, Thm. 21)	tolerance $\varepsilon < K\, d^{1/4}/(m^{1/2}s^{1/4})$, output $\varepsilon^{(1)} = O(\log(d)\sqrt{s}/\sqrt{d})$, $\mu^{(1)} = \widetilde{O}(\sqrt{s}/\sqrt{d})$	Lemma 37 with m in the role of F
(Hänni et al., 2024, Thm. 8)	width $d = \widetilde{O}(m^{2/3}s^2)$, depth $2L$	Proposition 38, inverted at $s = O(1)$
Adler & Shavit (2024)	$\Omega(\sqrt{m'}\log m')$ neurons and $\Omega(m'\log m')$ parameters; construction with $O(\sqrt{m'}\log m')$ neurons; at most $O(n^2/\log n)$ features with n neurons	Sec. D; the parameter-to-neuron conversion is flagged as an architectural assumption of that model
(Strohmer & Heath, 2003, Thm. 2.3)	$\max_{i\neq j} \lvert\langle\phi_i, \phi_j\rangle\rvert \geq \sqrt{(F-d)/(d(F-1))}$ for unit-norm systems	the Gram case of Theorem 1; the reduction is noted in Sec. 7.3

M Auxiliary proofs

M.1 Sub-Gaussian interference

Let $u, u_1, \ldots, u_m$ be independent uniform unit vectors in $\mathbb{R}^d$. Conditionally on u, each $X_j = \langle u, u_j\rangle$ is distributed as the first coordinate of a uniform point on S^{d-1}, hence has mean zero and is sub-Gaussian with variance proxy C_0/d for a universal C_0 (Vershynin, 2018, Thm. 3.4.6). The X_j are conditionally independent, so their sum is sub-Gaussian with proxy $C_0 m/d$ and $\Pr\{|\sum_j X_j| > t\} \leq 2\exp(-c_0 dt^2/m)$ with $c_0 = 1/(2C_0)$. This is the estimate used in Theorem 26.

M.2 The Bernoulli variant of the energy floor

Under $b_i \sim \text{Bernoulli}(p)$ independently, $\mathbb{E}[bb^\top] = p(1-p)I_F + p^2\mathbf{1}\mathbf{1}^\top$, so $\mathbb{E}_b \|Ab\|^2 = p(1-p)\|A\|_F^2 + p^2\|A\mathbf{1}\|^2 \geq p(1-p)F(F-d)/d$ by Theorem 1. With $p = s/F$ and $s \leq F/2$ this gives $\frac{1}{F}\mathbb{E}_b\|Ab\|^2 \geq \frac{s(F-d)}{2dF} = \Omega(s/d)$, matching Proposition 28 up to the $(F-1)$ versus F denominator. Both variants are computed by the released code and both are reported in E3.

N The nuclear-norm inequality

For $M \in \mathbb{R}^{F\times F}$ with singular value decomposition $M = \sum_{k=1}^r \sigma_k u_k v_k^\top$,

$$|\operatorname{tr}(M)| = \Big|\sum_k \sigma_k\, u_k^\top v_k\Big| \leq \sum_k \sigma_k |u_k^\top v_k| \leq \sum_k \sigma_k = \|M\|_*\,,$$

and by Cauchy–Schwarz on $(\sigma_1, \ldots, \sigma_r)$, $\|M\|_*^2 \leq r\sum_k \sigma_k^2 = r\,\|M\|_F^2$. With $r \leq d$ this gives $|\operatorname{tr}(M)|^2 \leq d\,\|M\|_F^2$, as used in Theorem 1. Note that the identity $\operatorname{tr}(M) = \sum_k \sigma_k$ holds only for positive semidefinite matrices and is *not* used.

O Reference scales

The scales appearing in this literature count different objects and should not be read as a capacity ordering. Writing $F_{\mathrm{CS}}(d,\alpha) = d\,g(\alpha)$ with $g(\alpha) = 1/((1-\alpha)\ln(1/(1-\alpha)))$ for compressed-sensing-style linearly decodable storage, $F^{\mathrm{H}}_{\mathrm{cert}}(d) = \widetilde{\Theta}(d^{3/2})$ for the Hänni template certificate, $L_{\mathrm{AS}}(d) = d^2/\log^2 d$ and $U_{\mathrm{AS}}(d) = d^2/\log d$ for the two sides of the Adler–Shavit bracket, and $N_{\mathrm{JL}}(d;\varepsilon) = \exp(\Theta(d\varepsilon^2))$ for Johnson–Lindenstrauss packing Johnson & Lindenstrauss (1984), one has

$$F_{\mathrm{CS}} \ll F^{\mathrm{H}}_{\mathrm{cert}} \ll L_{\mathrm{AS}} \lesssim F^{\mathrm{AS}}_{\mathrm{rec}} \lesssim U_{\mathrm{AS}} \ll N_{\mathrm{JL}}$$

as $d \to \infty$. This does not mean that recursive computation beats storage: N_{JL} counts passively packable states, F_{CS} counts linearly decodable features through a compressed bottleneck, and the middle two count recursively computable Boolean features under two different published frameworks. Michaud et al. Michaud et al. (2025) give a complementary reason why an observed dictionary size may fall below the template scale: if feature manifolds consume capacity, the relevant comparison is the effective dimension rather than the raw feature count.

P Sparse-autoencoder context

As background only, Borobia et al. (2026) train TopK sparse autoencoders ($k = 64$, expansion $8d$) on three language models and observes no dead features at $F = 8d$, placing those dictionaries well below both the $d^{3/2}$ template scale and the near-quadratic Adler–Shavit scale. Sparse-autoencoder dictionary size is a reconstructive quantity and is not a measurement of recursive computation capacity; these numbers are not used in any proof or experiment of the present paper, and are mentioned only to indicate where current empirical dictionaries sit relative to the scales of Appendix O. Liu et al. Liu et al. (2025) and Elhage et al. Elhage et al. (2022) provide further context on why strong superposition regimes are of interest.